

Documentation

Trinity C2



17.08.2026

James Marshall (ST10434366), Lyle William Richards (ST10322521), Sky Martin
(ST10286905), Connaire Myall (ST10297849), Morné Viljoen (ST10374994)

0x12Dark Development

Contents

1. Introduction	1
1.1. Overview	1
1.1.1. Core Functions	1
1.1.2. Malleability & Extensibility	1
1.2. Business Context	2
2. Requirements	3
2.1. Communication Protocols and Standards	3
2.2. Agent Management	3
2.3. Command System	3
2.4. File Operations	3
2.5. TOR Integration	4
2.6. Encryption and Security	4
2.7. Audit System	4
2.8. Log Management	4
2.9. Extensibility	4
2.10. AI Integration	4
2.11. Interface Requirements	4
2.11.1. GUI	4
2.11.2. C2 Server API	5
2.11.3. CLI	5
2.12. Performance Requirements	5
2.12.1. C2 Server Requirements	5
3. User Roles	6
3.1. User Role Identification	6
3.2. Primary User Role - Operator	7
3.3. Responsibilities of the Operator	7
3.4. Secondary User Role - System Administrator	7
3.4.1. Administrator Responsibilities	7
3.5. User Permissions and Access Levels	8
3.6. Principle of Least Privilege	8
3.6.1. Benefits of RBAC	8
4. User Stories	9
4.1. US-001 User Authentication	9
4.2. US-002 Access Control	9
4.3. US-003 User Management	9
4.4. US-004 Client Management	9
4.5. US-005 Engagement Creation	9

4.6.	US-006 Scope Management	10
4.7.	US-007 Asset Inventory	10
4.8.	US-008 Asset Status Monitoring	10
4.9.	US-009 Task Management	10
4.10.	US-010 Team Collaboration	11
4.11.	US-011 Findings and Reporting	11
4.12.	US-012 Evidence Management	11
4.13.	US-013 Report Generation	11
4.14.	US-014 Audit Logging	12
4.15.	US-015 Notifications	12
4.16.	US-016 Dashboard and Metrics	12
4.17.	User Experience Journey Map	13
4.18.	User Experience Journey Map Matrix	14
5.	Work Breakdown Structure	19
5.1.	Task 1	19
5.2.	Task 2	25
6.	Non-functional Requirements	41
6.1.	Overview	41
6.2.	Performance	41
6.3.	Reliability and Availability	41
6.4.	Scalability	42
6.5.	Security	42
6.6.	Usability	43
6.7.	Maintainability	43
6.8.	Compatibility and Portability	44
6.9.	Accessibility	44
6.10.	Logging and Auditability	45
6.11.	Summary	45
7.	Analysis Artifacts	47
7.1.	Domain Models - Agent	47
7.1.1.	Agent Domain Model Explanations	47
7.2.	Domain Models - Team server	48
7.2.1.	Team server Domain Model Explanations	49
7.3.	Design Artefacts - Team server Implementation Classes	50
7.4.	Design Artefacts - Agent Implementation Classes	50
8.	Implementation	51
8.1.	Operator task issuing workflow	51
8.2.	Operator task queue retrieval workflow	53

8.3.	Operator task cancellation workflow	54
8.4.	Operator login workflow	56
8.5.	Agent initial checkin workflow (Static AES256 encryption)	59
8.5.1.	Agent Initial Check-in Sequence Explanation	59
8.6.	Agent fully-established checkin workflow (Static AES256 encryption) .	62
8.6.1.	Agent Fully-Established Check-in Explanation	62
8.7.	Agent direct system call command execution workflow	65
8.8.	Agent indirect system call command execution workflow	66
8.9.	Team server HTTP/S listener creation workflow	66
8.9.1.	Team Server HTTP/S Listener Creation Explanation	67
8.10.	Team server HTTP/s listener deletion workflow	67
8.10.1.	Team Server HTTP/S Listener Deletion Explanation	68
8.11.	Team server HTTP/s listener update workflow	68
8.11.1.	Team Server HTTP/S Listener Update Explanation	69
8.12.	Team server TCP listener creation workflow	70
8.12.1.	Team Server TCP Listener Creation Explanation	71
8.13.	Team server TCP listener update workflow	71
8.13.1.	Team Server TCP Listener Update Explanation	72
8.14.	Team server TCP listener deletion workflow	73
8.14.1.	Team Server TCP Listener Deletion Explanation	73
8.15.	Payload generation workflow	74
8.16.	Deployment Plan	76
9.	Data Schema	77
9.1.	Overview	77
9.2.	Technologies	77
9.2.1.	Relational (PostgreSQL)	77
9.2.2.	Flexible Storage (JSONB)	77
9.2.2.1.	Alternatives	77
9.2.3.	File Storage	78
9.3.	Proposed Entity Relationship Diagram (ERD)	79
9.3.1.	User and Access Control	79
9.3.2.	Campaign and Operators	80
9.3.3.	Listeners	80
9.3.4.	Host Information	80
9.3.5.	Campaign, Payloads and Agents	80
9.3.6.	Agent Information	81
9.3.7.	Tasks and Results	81
9.3.8.	File Transfers	81
9.4.	Checkin JSON schema	81
9.4.1.	Intital Checkin message request	82
9.4.1.1.	Request	82

9.4.1.2.	Response	82
9.4.2.	Fully-established Checkin message request	82
9.4.2.1.	Request	82
9.4.2.2.	Response	83
9.4.3.	Fully-established Checkin message with command execution output	83
9.4.3.1.	Request (Success state)	83
9.4.3.2.	Request (Failure state)	84
9.4.3.3.	Request (Non-failure/Non-success)	84
9.5.	Command JSON schema	84
9.5.1.	Help command	85
9.5.2.	Spawn PowerShell	85
9.5.3.	Spawn Shell	85
9.5.4.	Download	85
9.5.5.	Cancel Download	85
9.5.6.	Get Downloads	86
9.5.7.	Get Active Downloads	86
9.5.8.	Upload	86
9.5.9.	Kill Process	86
9.5.10.	RunAs	87
9.5.11.	Run	87
9.5.12.	RunU	87
9.5.13.	Escalate	87
10.	Architecture Artifacts	89
10.1.	Design patterns	89
10.1.1.	Observer	89
10.1.2.	Abstract Factory	90
10.1.3.	Strategy	91
10.2.	Architecture patterns	92
10.2.1.	Peer-to-peer model	92
10.2.2.	Client-Server model	93
11.	Security	96
11.1.	Component Security Architecture	96
11.1.1.	Agent Payload Security	96
II.I.I.1.	Security Considerations & Evasion	96
II.I.I.2.	Network Security	96
II.I.I.3.	Vulnerabilities & Potential Threats	97
11.1.2.	Team Server API Security	97
II.I.2.1.	Security Considerations & Injection Protection ...	97
II.I.2.2.	Network Security	97

II.I.2.3. Vulnerabilities & Potential Threats	98
11.1.3. Client Application Security (Blazor Web Interface)	98
II.I.3.1. Security Considerations & XSS Mitigation	98
II.I.3.2. Network Security	98
II.I.3.3. Vulnerabilities & Potential Threats	98
11.2. Authentication & Session Management	98
11.3. Password Security & Credential Storage	99
11.4. Authorisation & Access Control	99
11.5. API Security & Input Validation	100
11.6. Storage Security	100
11.7. Network Security & Evasion	100
11.8. Application of Security Principles (CIA Triad)	101
11.9. Threat & Risk Mitigation Matrix	102
12. DevOps	104
12.1. DevOps Approach	104
12.1.1. Branching Strategy	104
12.1.2. Development Workflow	106
12.1.3. GitHub Actions Pipeline	107
12.1.4. Automated Testing and Quality Assurance	109
I2.I.4.1. Agent Build and Tests	IO9
I2.I.4.2. C# Server Unit Tests	II0
I2.I.4.3. Database and Persistence Tests	II0
I2.I.4.4. Server API Tests	III
I2.I.4.5. Integration Testing	II2
12.1.5. Pull Request Validation	113
12.1.6. Testing Strategy and Non-Functional Requirements	113
12.1.7. Expected Testing Outcome	114
13. Running Costs	115
13.1. Infrastructure and Hosting	115
13.1.1. VPS Hosting Cost Comparison	115
13.1.2. Agent Growth and Infrastructure Requirements	116
13.1.3. Cost at Different Deployment Scales	117
13.2. Software and Development Tools	117
13.3. Maintenance and Operational Costs	118
13.4. Cost Optimisation	118
13.5. Summary	118
14. Change Management	119
14.1. Change Request Process	119
14.2. Version Control and Branching Strategy	119

14.3. Documentation Changes	120
14.4. Change Impact and Risk Assessment	120
14.5. Change Implementation and Testing	120
14.6. Change Tracking and Review	121
15. Prototype	122
15.1. Login	122
15.2. Dashboard	123
15.3. Deployment	124
15.4. Clients	125
15.5. Campaigns	126
15.6. Tasks	127
15.7. Files	128
15.8. Logs	129
15.9. Reports	130
15.10. Settings	131
16. Appendix A - Declaration of Authenticity	132
17. Appendix B - Declaration of Authenticity	134
18. Appendix C - Declaration of Authenticity	136
19. Appendix D - Declaration of Authenticity	138
20. Appendix E - Declaration of Authenticity	140
21. AI Declaration	142
21.1. Disclosure of AI Usage	142
References	144

1. Introduction

The purpose of this document is to define and describe the requirements, constraints, development scope, implementation, and maintenance considerations of the ‘**Trinity**’ post-exploitation framework developed for Dark Development.

1.1. Overview

Trinity is a malleable and extensible post-exploitation framework that supports four core functions: **weaponisation**, **persistence**, **privilege escalation**, and **data exfiltration** on Windows-based systems. The framework operates as a client-server model tailored for team-based offensive campaigns, comprising three main components:

1. **Command and Control (C2) Server**
2. **Web-based Client Interface**
3. **Agent Payload**

1.1.1. Core Functions

1. **Weaponisation** — The embedding of the agent payload with/in a selected binary format for execution on target endpoint(s).
2. **Persistence** — The maintenance of a foothold on compromised endpoint(s) enabling automatic re-execution in the eventuality of system shutdown or reboot.
3. **Privilege Escalation** — The acquirement of higher-level permissions on a system from a prior position of unprivileged access.
4. **Data Exfiltration** — The collection and transmission of data from compromised endpoint(s) to the command and control channel.

1.1.2. Malleability & Extensibility

The framework is considered **malleable** as operators are able to customise network communication protocols (e.g., HTTP, DNS, TOR, and SMB) and process injection routines of the agent, providing flexibility in terms of adapting to the requirements and constraints of the target operating environment.

Extensibility of agent capabilities is supported through a custom object file loader and the spawn process capability of the agent that enables third-party .NET tooling to be run in a newly created process. This enables the addition of new agent functionality and behaviour during runtime without termination. Trinity provides flexibility, both in its potential configurations and evasive options to execute offensive actions, to allow adaptation to the circumstances and objectives of your organisation.

1.2. Business Context

As it applies to this project, **Dark Development** is an organisation that provides consulting services for offensive security infrastructure, malware research, and penetration testing.

2. Requirements

2.1. Communication Protocols and Standards

1. HTTP REST API (Primary) (C# Web API project)
2. WebSocket Support (Real-time)
3. P2P Communication (SMB)
4. Message Format Standardisation
5. TOR network support

2.2. Agent Management

1. Unlimited Concurrent Agents
2. UUID-based Identification
3. Multi-factor Agent Identification (hostname, IP, username)
4. Agent Status Tracking (online / offline / lost)
5. Customisable agent check-ins
6. Default agent behaviour:
 - Command execution
 - List processes
 - Persistence using Windows tasks
 - Local user enumeration
 - File upload, explore, write
 - Privilege escalation
 - API Hashing and string encryption

2.3. Command System

1. One-off Commands:
 - Direct or indirect system calls (if possible)
 - Default kernel32.dll
2. Interactive Shell Support
3. Command Status Tracking (Pending / Running / Success / Failed / Timeout)
4. Large Output Handling (chunk streaming)
5. Buffer Management for keystrokes
6. Command automation / scripting

2.4. File Operations

1. File Upload / Download
2. Chunked File Transfer
3. Progress Tracking

2.5. TOR Integration

1. TOR Proxy Support
2. Onion Service Setup
3. TOR Circuit Management

2.6. Encryption and Security

1. TLS/SSL Encryption
2. Payload Obfuscation (AES256 Encryption)
3. Certificate Pinning

2.7. Audit System

1. Command Logging
2. Result Tracking
3. Error Logging
4. User Activity Monitoring

2.8. Log Management

1. 30-day Retention Policy
2. External Log Export (Discord / Telegram bots)
3. JSON Report Generation
4. Real-time Log Streaming

2.9. Extensibility

1. Plugin System (Extensible using object files)
2. Custom Command Modules
3. Third-party Integrations

2.10. AI Integration

1. OpenAI / Claude API Support
2. Local AI (Ollama)
3. OpenRouter Integration (*Future*)
4. AI-assisted Command Generation (*Future*)

2.11. Interface Requirements

2.11.1. GUI

- Agent management interface
- Agent builder interface
- Operator management interface
- Listener management interface

2.11.2. C2 Server API

- RESTful HTTP API & WebSocket endpoint interfaces.

2.11.3. CLI

- There is no command-line interface.

2.12. Performance Requirements

2.12.1. C2 Server Requirements

Component	Specification
Processor	1 gigahertz (GHz) or faster with 2 or more cores on a compatible 64-bit processor or System on a Chip (SoC)
RAM	4 gigabytes (GB)
Storage	10 GB or larger storage device
Supported OS	Debian 12+, Ubuntu 24.04 LTS+, Windows 7+

3. User Roles

3.1. User Role Identification

User role identification was performed through analysis of the project requirements, system architecture, and design artefacts developed during the analysis phase of the project. The objective of user role identification is to determine which individuals interact directly with the system, what responsibilities they perform, and what level of access is required for each role. Clearly defining user roles assists with system security, simplifies permission management, and provides a foundation for implementing Role-Based Access Control (RBAC) throughout the application.

Analysis of the current prototype identifies the **Operator** as the primary authenticated user of the Command and Control Framework. The Operator is responsible for interacting with the system through the graphical user interface, managing deployed agents, configuring listeners, generating payloads, creating operational tasks, and monitoring returned results.

Although the current implementation focuses primarily on the Operator, several additional entities participate in the operation of the framework:

Entity	Classification	Description
Operator	Human User	The primary authenticated user responsible for operating the C2 Framework, managing deployed agents, generating payloads, configuring listeners, creating tasks, and monitoring execution results.
Agent	Software Component	A deployed software implant that establishes communication with the C2 server, receives assigned tasks, executes supported commands, and returns execution results.
Listener	Software Component	A communication component responsible for accepting incoming HTTP or TCP connections from deployed agents and forwarding requests to the server.
Payload	Software Component	A generated executable that contains the configuration required for an agent to establish communication with the C2 server.

3.2. Primary User Role - Operator

The Operator is the primary authenticated user of the C2 Framework and serves as the central point of interaction between the user and the system. The Operator is responsible for controlling and monitoring the framework throughout an authorised cybersecurity assessment or testing engagement.

The Operator does not communicate directly with deployed agents. Instead, all communication occurs through backend services and APIs that manage interaction between the GUI and the C2 server.

3.3. Responsibilities of the Operator

- **Authentication:** Authenticate securely before accessing operational features.
- **Agent Management:** View connected agents, retrieve detailed agent info, monitor communication, update intervals, manage callback host info, and remove inactive agents.
- **Payload Generation:** Create payload UUIDs, generate encryption keys, apply C2 communication profiles, configure callbacks, and build deployable binaries.
- **Listener Configuration:** Create, configure, start, stop, and monitor HTTP/TCP listeners.
- **Task Management:** Assign instructions (command execution, file transfers, assembly execution) to specific agents.
- **Command Execution:** Issue shell commands, PowerShell scripts, file operations, process management, and assembly loading.
- **Operational Monitoring:** Continuously evaluate system performance, agent check-ins, task completion, and output logs.

3.4. Secondary User Role - System Administrator

A dedicated System Administrator role maintains overall system health, availability, user account lifecycle, and infrastructure stability without directly participating in operational targeting.

3.4.1. Administrator Responsibilities

- Creating and managing Operator accounts
- Assigning and modifying user permissions
- Managing authentication settings and server configuration
- Monitoring application performance and system logs
- Performing database maintenance, backup, and recovery operations
- Managing software updates and deployments

3.5. User Permissions and Access Levels

System Function	System Administrator	Operator
User Authentication	✓	✓
View Dashboard	✓	✓
Manage User Accounts	✓	✗
Configure Authentication	✓	✗
Configure Server	✓	✗
Manage Agents	✓	✓
Generate Payloads	✓	✓
Configure Listeners	✓	✓
Create Tasks	✓	✓
Execute Commands	✓	✓
View Results	✓	✓
Access System Logs	✓	Limited
Database Maintenance	✓	✗
Backup and Recovery	✓	✗
Software Updates	✓	✗

3.6. Principle of Least Privilege

[!IMPORTANT] The implementation of Role-Based Access Control (RBAC) enforces the **Principle of Least Privilege**, ensuring users receive only the permissions necessary to perform assigned duties.

3.6.1. Benefits of RBAC

- Improved security through controlled access to system functionality.
 - Simplified management of user permissions.
 - Increased accountability by associating actions with authenticated users.
 - Reduced administrative overhead when managing user accounts.
 - Improved scalability for introducing future roles without architecture re-designs.
-

4. User Stories

4.1. US-001 User Authentication

As a security consultant, I want to securely authenticate to the platform using my assigned credentials so that I can access the client engagement and assessments that are appropriate for my role. The system shall validate the users credential before granting access and establish a secure session. Users shall be directed to a personalized dashboard displaying their assigned engagements, pending tasks, recent activity and notifications. Failed authentication attempts shall return an appropriate error message without revealing sensitive system information, while all login attempts, whether successful or not, will be recorded within the audit log.

4.2. US-002 Access Control

As a system Administrator, I want to assign specific roles and permissions to platform users so that each individual can only access the features and information required to perform their responsibilities. The system shall support multiple predefined roles such as Administrator, Security Consultant, Project Manager, Analyst and Client Representative. Each role shall have configurable permissions that determines which modules, reports, settings and engagement information may be viewed or modified. Permission changes will take immediate effect.

4.3. US-003 User Management

As a System Administrator, I want to manage user accounts throughout their lifecycle so that access to the platform remains secure and current. The system shall allow authorised administrators to create new accounts; modify existing user information, reset passwords, temporarily disable accounts when needed. User records shall include contact information, assigned roles, organisation details and account status.

4.4. US-004 Client Management

As a Security Consultant, I want to maintain client profiles so that all engagement related information is stored in a central location. The platform shall allow authorised users to record client contact information, organisational details, contractual information, primary points of contact and engagement history. Client records should support multiple concurrent engagements whilst ensuring that data remains logically separated between organisations.

4.5. US-005 Engagement Creation

As a Penetration Tester, I want to create new assessment engagement so that testing activities, documentation, assigned personnel, findings and deliverables are

organised under a single project. When creating an engagement, the platform will allow users to define engagement names, associated clients, project objectives, authorised testing periods, assigned team members, project status and supporting documentation. Each engagement will be uniquely identifiable and accessible only to authorised personnel assigned to the project.

4.6. US-006 Scope Management

As a project manager, I want to document the agreed assessment scope so that all project participants understand the systems, applications and environments authorised for testing. The platform shall allow authorised users to record target assets, IP ranges, web applications, cloud environments, assessment objectives, exclusions and testing windows. Scope modifications shall require appropriate permissions and be version controlled to maintain a historical record of all changes throughout the engagements lifecycle.

4.7. US-007 Asset Inventory

As a security consultant, I want to maintain a complete inventory of authorised client assets participating in an engagement so that project information can remain safe, organised and accessible. The platform shall record details such as hostnames, operating systems, IP addresses, hardware specifications, software versions, ownership information and associated engagement details. Asset records shall support searching, filtering and categorisation, which will allow consultants to quickly locate specific systems during large scale assessments.

4.8. US-008 Asset Status Monitoring

As a Security Analyst, I want to monitor the operational status of authorised assets throughout an engagement so that I can quickly identify systems requiring attention. The platform shall provide a dashboard displaying each registered assets availability, communication status, recent activity and last recorded interaction. Status information should update automatically and provide visual indicators highlighting changes that require analyst review, improving situational awareness during ongoing engagements.

4.9. US-009 Task Management

As a Penetration Tester, I want to create, assign, prioritise and track assessment tasks so that project activities are organised and progress can be monitored effectively. The platform shall support assigning tasks so that project activities are organised and progress can be monitored effectively. The platform shall support assigning tasks to individual consultants, recording due dates, tracking completion status, attaching supporting documentation and maintaining comments associated with each given task. Managers should be able to monitor overall

project progress while consultants maintain visibility of their individual responsibilities.

4.10. US-010 Team Collaboration

As a Security Consultant, I want to collaborate with other members of the assessment team so that findings, observations and project updates can be shared efficiently throughout the engagement. The platform shall support shared notes, comments, task discussions, document attachments and internal notifications, whilst maintaining an audit trail of all collaborative activities. The functionality improves communication between project members and ensures important information is available to the entire assessment team.

4.11. US-011 Findings and Reporting

As a Security Consultant, I want to document identified security findings throughout an engagement so that vulnerabilities can be tracked, reviewed and included within the final assessment report. Each finding will include a title, description, affected assets, supporting evidence, severity rating, business impact, remediation recommendations and current remediation status. Findings should remain editable until final report approval, whilst maintaining version histories to preserve accountability.

4.12. US-012 Evidence Management

As a Security Consultant, I want to attach supporting evidence to assessment findings so that observations can be validated and presented clearly to the client. The platform shall support uploading screenshots, exported reports, log files, diagrams, documents and other supporting artefacts while maintaining secure storage and ensuring that evidence remains associated with the correct engagement and findings through the project.

4.13. US-013 Report Generation

As a Penetration Tester, I want to generate professional assessment reports so that project findings can be delivered to the clients in a consistent and structured format. The platform shall automatically compile engagement information, executive summaries, findings, supportive evidence, security ratings, remediation recommendations and appendices into a report that can be reviewed prior to publication. Reports should support multiple export formats and remain archived for future reference.

4.14. US-014 Audit Logging

As a System Administrator, I want all significant platform activity to be recorded within an audit log so that administrative actions and project changes can be reviewed when required. The system will record authentication events, account changes, engagement modifications, report generation, user management actions and other significant activities while protecting the integrity of audit records from unauthorised modifications.

4.15. US-015 Notifications

As a Security Consultant, I want to receive notifications regarding important project events so that I remain informed about changes affecting my engagements. Notifications should include task assignments, project updates, report approvals, approaching deadlines and administrative announcements. Users should be able to configure notification preferences according to their individual responsibilities.

4.16. US-016 Dashboard and Metrics

As a project manager, I want a comprehensive dashboard summarising engagement activity so that I can monitor project progress, consultant workloads and overall assessment status from a single location. The dashboard shall display active engagements, pending tasks, findings by severity, upcoming deadlines, team activity and project milestones using intuitive visualisations that assist managements decision making.

4.17. User Experience Journey Map



Figure 1: User Experience Journey Map diagram.

4.18. User Experience Journey Map Matrix

Stage	User Goal	Related User Story	User Actions	System Response	User Experience	Opportunities for Improvement
Access Platform	Securely access the application	US-001, US-002	Opens the application and enters credentials.	Authenticates the user, validates permissions, loads personalised dashboard.	"I want to access my projects quickly and securely."	Fast login, MFA, password recovery, clear error messages.
Manage Users	Ensure the correct personnel have access	US-003	Administrator creates or updates user accounts and assigns permissions.	User accounts are created and appropriate permissions applied.	"Everyone should only have access to what they need."	Bulk user import, role templates, activity history.
Manage Clients	Organise client information	US-004	Creates a client profile and stores organisation details.	Client profile is saved and available for future engagements.	"All client information should be in one place."	Client search, tagging, document storage.
Create Engagement	Start a new as-	US-005	Creates a new engagement	Engagement work-	"Everything for this	Engagement tem-

Stage	User Goal	Related User Story	User Actions	System Response	User Experience	Opportunities for Improvement
	assessment		ment, assigns consultants, defines objectives and timelines.	space is generated and linked to the client.	project should be organised together."	plates and duplication of previous projects.
Define Assessment Scope	Record authorised assessment boundaries	US-006	Adds target assets, authorised systems, timelines and engagement notes.	Scope information is stored and associated with the engagement.	"Everyone needs to understand what is in scope."	Visual scope builder and change history.
Build Asset Inventory	Organise target assets	US-007	Adds or imports authorised assets into the engagement.	Assets are categorised and displayed in the inventory.	"I need a complete inventory before beginning work."	CSV import, automatic categorisation, filtering.
Monitor Assets	Observe asset availability	US-008	Views the asset dashboard and checks system status.	Displays connectivity, health information and last activity.	"I need to know which systems are currently available."	Real-time updates and status indicators.

Stage	User Goal	Related User Story	User Actions	System Response	User Experience	Opportunities for Improvement
Manage Assessment Tasks	Organise assessment activities	US-009	Creates tasks, assigns consultants and monitors completion.	Tasks appear on team dashboards with deadlines and priorities.	“The whole team should know what they’re responsible for.”	Kanban board, calendar view and work-load balancing.
Collaborate with Team	Share information efficiently	US-010	Posts comments, uploads files and discusses findings with teammates.	Notifications are sent and discussion history is recorded.	“Communication should happen inside the platform.”	Mentions, threaded discussions and document pre-views.
Record Findings	Document discovered vulnerabilities	US-011	Creates findings, assigns severity and documents recommendations.	Findings are linked to the engagement and tracked through their lifecycle.	“I need to keep detailed records while I work.”	Severity calculators and vulnerability templates.
Upload Support-	Preserve supporting doc-	US-012	Uploads screenshots, docu-	Evidence is securely stored	“Everything supporting a	Drag-and-drop uploads

Stage	User Goal	Related User Story	User Actions	System Response	User Experience	Opportunities for Improvement
ing Evidence	umentation		ments and logs to findings.	and associated with the relevant finding.	finding should stay together.”	and version control.
Generate Reports	Produce deliverables	US-013	Selects report options and generates the final report.	Platform compiles engagement data into a structured report.	“Generating a report should require manual formatting.”	Custom updates, branding and PDF previews.
Review Audit Logs	Verify platform activity	US-014	Searches audit logs and reviews user activity.	Platform displays a searchable audit trail.	“I need confidence that all activity is traceable.”	Advanced filtering and export functionality.
Receive Notifications	Stay informed	US-015	Reviews notifications regarding engagements, tasks and updates.	Platform displays alerts and notification history.	“I shouldn’t have to constantly check for updates.”	Email notifications and notification preferences.
Monitor Project Dashboard	Track engagement progress	US-016	Reviews dashboards, charts and engagement metrics.	Displays project summaries, findings, outstanding tasks and	“I want a quick overview of every engagement.”	Customisable dashboards and interactive charts.

Stage	User Goal	Related User Story	User Actions	Sys-tem Re-sponse	User Ex-perience	Oppor-tunities for Im-prove-ment
				mile-stones.		
Com-plete Engage-ment	Finalise and archive the pro-ject	US-O11, US-O16	Reviews findings, exports reports, closes engage-ment and archives docu-menta-tion.	Engage-ment is marked complete and his-torical data re-mains accessi-ble.	“The project should be easy to revisit in the fu-ture.”	Project archiv-ing, re-port ver-sion his-tory and search-able archives.

5. Work Breakdown Structure

The Work Breakdown Structure (WBS) segments the project scope into hierarchical deliverables that specifies the following: task, description, expected duration, predecessors, assigned team member and the allocated resources assigned to support its completion.

The task is comprised of a WBS code with the first level representing major deliverables and its comprising sub-tasks. All sub-tasks must be completed within the expected duration for a work segment to be considered a major deliverable to be considered a success. The predecessors column indicates whether a major or sub-task is dependent on another, if this column is empty it should be assumed that the work segment is modular in nature and can be delivered independently.

5.1. Task 1

Below is the WBS for Task 1 of the project, it details all the sub-tasks for the documentation deliverable. The documentation is required before the development phase can begin to ensure alignment with stakeholder expectations namely that of Dark Development.

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member/s	Resources
I.I	Documentation	Complete project documentation	27 Days (Jul 20 - Aug 15)	None	Morné Viljoen, James Marshall, Connaire Myall, Lyle William Richards and Sky Martin	Typst
I.I.I	KAN-I Introduction	Executive summary, context and pro-	3 Days (Jul 20 - Jul 22)	None	James Marshall	Obsidian, and company provided information

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member/s	Resources
		ject justification				
I.I.2	KAN-2 Requirements	Document core functional user requirements	5 Days (Jul 23 - Jul 27)	None	Morné Viljoen	Company provided information
I.I.2.I	KAN-3 Non-functional requirements	Document system constraints and metrics	2 Days (Jul 23 - Jul 24)	None	Morné Viljoen	Company provided information
I.I.2.2	KAN-4 Functional requirements	Document core functional user requirements	3 Days (Jul 24 - Jul 27)	I.I.2.I	Morné Viljoen	Company provided information and external resources
I.I.3	KAN-5 User Roles	Define various user roles and permissions	2 Days (Jul 28 - Jul 29)	I.I.2	Morné Viljoen	Company provided information and market research
I.I.4	KAN-6 User Stories	Map functional requirements	3 Days (Jul 30 - Aug 01)	I.I.3	Connaire Myall	Company provided information and

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member/s	Resources
		to user roles in standardised format				market research
I.I.4.I	KAN-7 User Experience Journey Map	Map user navigation flows throughout the client application	3 Days (Aug 02 - Aug 04)	I.I.4	Connaire Myall	Company provided information and market research
I.I.6	KAN-8 Analysis Artifacts	Analyse technical requirements to create a domain model	3 Days (Jul 28 - Jul 30)	I.I.2	Morné Viljoen and James Marshall	Company provided information, and market research
I.I.6.I	KAN-9 Domain modeling	Design a UML domain model containing main system/s concepts	3 Days (Jul 28 - Jul 30)	I.I.2	James Marshall	Company provided information, and market research
I.I.6.2	KAN-10 Design artifacts	Design UML class diagrams of system requirements	4 Days (Jul 31 - Aug 03)	I.I.6.I	James Marshall	Company provided information, and market research

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member/s	Resources
I.I.7	KAN-11 Implementation	Document build phase guidelines and strategies	4 Days (Aug 04 - Aug 07)	I.I.6	James Marshall	Domain model, Company provided information, and market research
I.I.8	KAN-12 Data schema	Explain data storage technology selection, design ERD and JSONB schemas	3 Days (Aug 08 - Aug 10)	I.I.2, I.I.7	Lyle William Richards	PostgreSQL documentation, Mythic C2 documentation, Company provided information, and market research
I.I.9	KAN-13 Architecture artifacts	Describe architectural patterns employed in the project	3 Days (Aug 08 - Aug 10)	I.I.7	James Marshall	Design patterns: Elements of Reusable Object-Oriented Software
I.I.9.1	KAN-14 Design patterns	Document implementation of design	3 Days (Aug 08 - Aug 10)	I.I.7	James Marshall	Design patterns: Elements of Reusable

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member/s	Resources
		patterns in agent and team server				Object-Oriented Software
1.1.9.2	KAN-15 Architecture patterns	Document selection of architectural patterns in agent and team server	3 Days (Aug 08 - Aug 10)	1.1.6	James Marshall	External resources
1.1.10	KAN-16 Security	Document and discuss application of security principles and mitigations in agent and team server	2 Days (Aug 11 - Aug 12)	1.1.9.2	Sky Martin	SecOps Framework
1.1.11	KAN-17 DevOps	Define CI/CD pipelines and infrastructure	3 Days (Aug 08 - Aug 10)	1.1.7	Lyle William Richards	GitHub Actions / Docker
1.1.11.1	KAN-18 Github Actions Pipeline	Define and describe the utilisation of GitHub	3 Days (Aug 08 - Aug 10)	1.1.7	Lyle William Richards	GitHub Actions / Docker

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member/s	Resources
		actions and other relevant tests				
I.I.I.12	KAN-19 Change Management	Detail user, version control, onboarding and training plans	2 Days (Aug 11 - Aug 12)	None	Morné Viljoen	
I.I.I.13	KAN-20 Running Costs	Calculate infrastructure monthly cost projections	1 Day (Aug 11 - Aug 11)	I.I.I.11	Morné Viljoen	
I.I.I.14	KAN-21 Prototype	Develop a working Figma prototype with navigation	1 Day (Aug 05 - Aug 05)	I.I.4	Connaire Myall	Figma
I.I.I.15	KAN-22 Deployment plan	Detail release and deployment strategy pipelines	3 Days (Aug 13 - Aug 15)	I.I.I.11	Sky Martin	

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member/s	Resources
I.I.I6	KAN-23 Work Break-down Structure	Build project task break-down list and hierarchy for Task 2	3 Days (Aug 11-Aug 13)	I.I.II	James Marshall	Mark-down and Obsidian

5.2. Task 2

Below is the WBS for the development phase of the project. It has been segmented by system to improve readability, namely: Team server, Agent, and Client.

A significant number of Agent WBS tasks lack a predecessor as the methods are designed to be modular in nature. This modularity is achieved by ensuring that each method has a pre-defined signature based on the data schema of its respective command and a standardised command result data type that will be returned to the caller. This enables the orchestration of command execution without closely coupling underlying implementations that will invariably differ across command type. The importance of this is that it allows each team member to develop Agent capabilities without much concern or reliance on another's implementation enabling development to run concurrently in most cases.

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member	Resources / Tools
I.I	Agent	**Complete all agent base capabilities **	40 Days (Aug 17 - Sep 25)	Documentation	James Marshall, Lyle William Richards and Sky Martin**	Visual Studio
I.I.I	KAN-24 Keylogger	Implement key-stroke	7 Days (Aug 17 - Aug 23)	None	Lyle William Richards	Visual Studio, Microsoft

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member	Resources / Tools
		capture using callback function hooks that stores to memory buffer in agent process				Visual C++ (MSVC), Microsoft Developer Network (MSDN) documentation, Windows Subsystem and native DLLs
I.I.2	KAN-25 API Hashing	Hide plain text function names using hashes for Windows Subsystem DLLs	6 Days (Aug 17 - Aug 22)	None	Sky Martin	Visual Studio, Microsoft Visual C++ (MSVC), and Microsoft Developer Network (MSDN) documentation
I.I.3	KAN-26 Tor Support	Implement checkin/com-	7 Days (Aug 25 - Aug 31)	I.I.17	Lyle William Richards	Visual Studio, Microsoft

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member	Resources / Tools
		munication that routes through the TOR network				Visual C++ (MSVC), and Microsoft Developer Network (MSDN) documentation
I.I.4	KAN-27 Token Impersonation	Implement token impersonation commands	6 Days (Aug 23 - Aug 28)	None	Sky Martin	Visual Studio, Microsoft Visual C++ (MSVC), and Microsoft Developer Network (MSDN) documentation
I.I.5	KAN-28 Exit	Implement agent command that terminates its process	3 Days (Sep 01 - Sep 03)	None	Lyle William Richards	Visual Studio, Microsoft Visual C++ (MSVC), and Microsoft Developer

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member	Resources / Tools
						oper Network (MSDN) documentation
I.I.6	KAN-29 Configuration	Implement agent configuration: Sleep, AES256 key storage, host rotation and checkin jitter	6 Days (Aug 25 - Aug 30)	I.I.17	James Marshall	Visual Studio, Microsoft Visual C++ (MSVC), and Microsoft Developer Network (MSDN) documentation
I.I.7	KAN-30 File Transfers	Implement file transfer functionality with optional chunking for larger files	8 Days (Sep 04 - Sep 11)	None	Lyle William Richards	Visual Studio, Microsoft Visual C++ (MSVC), and Microsoft Developer Network (MSDN) documentation

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member	Resources / Tools
I.I.8	KAN-31 Direct system calls	Implement direct system calls in assembly that can be called in Cpp code base	6 Days (Aug 31 - Sep 05)	None	James Marshall	Microsoft Macro Assembler (MASM), Microsoft Developer Network (MSDN) documentation and external resources
I.I.9	KAN-32 Indirect system calls	Implement indirect system calls in assembly that can be called in Cpp code base	6 Days (Sep 06 - Sep 11)	None	James Marshall	Microsoft Macro Assembler (MASM), Microsoft Developer Network (MSDN) documentation and external resources

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member	Resources / Tools
I.I.IO	KAN-33 Kill	Implement process termination by Process Identifier (PID)	4 Days (Aug 29 - Sep 01)	None	Sky Martin	Visual Studio, Microsoft Visual C++ (MSVC), and Microsoft Developer Network (MSDN) documentation
I.I.II	KAN-34 Power-Shell	Implement PowerShell command execution	7 Days (Sep 12 - Sep 18)	None	Lyle William Richards	Visual Studio, Microsoft Visual C++ (MSVC), and Microsoft Developer Network (MSDN) documentation
I.I.I2	KAN-35 Shell	Implement shell (cmd.exe) command	5 Days (Sep 02 - Sep 06)	None	Sky Martin	Visual Studio, Microsoft Visual C++

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member	Resources / Tools
		execution				(MSVC), and Microsoft Developer Network (MSDN) documentation
I.I.I3	KAN-36 GetUID	Enumerate current User information such as Identifier and related meta-data	3 Days (Sep 19 - Sep 21)	None	Lyle William Richards	Visual Studio, Microsoft Visual C++ (MSVC), and Microsoft Developer Network (MSDN) documentation
I.I.I4	KAN-37 RunU	Run an arbitrary command or program under an arbitrary parent process.	6 Days (Sep 07 - Sep 12)	None	Sky Martin	Visual Studio, Microsoft Visual C++ (MSVC), and Microsoft Developer Network

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member	Resources / Tools
						(MSDN) documentation
I.I.I5	KAN-38 Execute-assembly	Run a local .NET program in memory on a target machine.	7 Days (Sep 12 - Sep 18)	None	James Marshall	Visual Studio, Microsoft Visual C++ (MSVC), C# and Microsoft Developer Network (MSDN) documentation
I.I.I6	KAN-39 Object file extensibility	Directs the Agent to act as an in-memory linker and loader for a compiled, unlinked C object file	9 Days (Sep 19 - Sep 25)	None	James Marshall	Visual Studio, Microsoft Visual C++ (MSVC), C and Microsoft Developer Network (MSDN) documentation

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member	Resources / Tools
I.I.I7	KAN-40 Agent Checkin	Implementation of periodic checkin to team-server and task orchestration in Agent's process memory	8 Days (Aug 17 - Aug 24)	None	James Marshall	Visual Studio, Microsoft Visual C++ (MSVC), and Microsoft Developer Network (MSDN) documentation
I.I.I8	KAN-50 UAC Bypass	Implementation company approved components for local user account elevation bypass execution context	8 Days (Sep 13 - Sep 20)	None	Sky Martin	Visual Studio, Microsoft Visual C++ (MSVC), and Microsoft Developer Network (MSDN) documentation
I.2	Team server	Complete all Team server endpoints	26 Days (Aug 17 - Sep 11)	Documentation	James Marshall, and Lyle William Richards	Visual Studio, Swagger, Docker, C#, Python,

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member	Resources / Tools
		and related logic				EF Core, MSVC (Microsoft Visual C++), CMake and Postman
I.2.1	KAN-51 Logging service	Collect, process, store, and analyze operations performed on the Team server	6 Days (Sep 04 - Sep 09)	I.2.2 - I.2.10	James Marshall	Visual Studio, C# and EF Core
I.2.2	KAN-52 Payload service	Generate an Agent payload with the defined configuration specified by the operator	6 Days (Aug 29 - Sep 03)	I.2.5, I.2.10	James Marshall	Visual Studio, C#, CMake, MSVC (Microsoft Visual C++) and EF Core
I.2.3	KAN-53 Global Exception Handler (GEH)	Handle uncaught Application Programming Interface	1 Day (Aug 17)	None	James Marshall	Visual Studio, C# and Swagger

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member	Resources / Tools
		(API) exceptions				
I.2.4	KAN-54 Authentication service	Generate an JSON Web Token upon successful operator login	5 Days (Aug 23 - Aug 27)	I.2.5	Lyle William Richards	Visual Studio, C# and EF Core
I.2.5	KAN-55 Database service	Implement database operations for all dependant services	6 Days (Aug 17 - Aug 22)	None	James Marshall, and Lyle William Richards	Visual Studio, C# and EF Core
I.2.6	KAN-56 Server service	Implement HTTP/S endpoints for Team server status and Internet Protocol (IP) retrieval	1 Day (Aug 17)	None	James Marshall	Visual Studio, C# and Swagger
I.2.7	KAN-57 Command service	Implement command endpoints and	6 Days (Aug 23 - Aug 28)	I.2.5	James Marshall, and Lyle William Richards	Visual Studio, C#, EF Core and Swagger

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member	Resources / Tools
		queuing logic for Agents				
I.2.8	KAN-58 Agent service	Implement Agent management endpoints that enable the manipulation of its configuration	6 Days (Aug 23 - Aug 28)	I.2.5	James Marshall	Visual Studio, C#, EF Core and Swagger
I.2.9	KAN-59 Task service	Implement Agent task queuing management endpoints and associated logic	6 Days (Aug 29 - Sep 03)	I.2.5, I.2.7	Lyle William Richards	Visual Studio, C#, EF Core and Swagger
I.2.10	KAN-60 Listener service	Implement endpoints for creation of HTTP/S and TCP listen-	6 Days (Aug 23 - Aug 28)	I.2.5	James Marshall	Visual Studio, C#, EF Core, Postman and Swagger

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member	Resources / Tools
		ers for Agent communication				
1.2.11	KAN-61 TOR support	Implement a re-director that forwards requests from TOR network to relevant Team server end-points	5 Days (Aug 29 - Sep 02)	1.2.8	Lyle William Richards	Python and Docker
1.2.12	KAN-62 Docker Container script	Create a Docker script that containerises the Team server project and the Python re-director	2 Days (Sep 10 - Sep 11)	1.2.1-1.2.11	James Marshall and Lyle William Richards	Docker
1.3	Client	Complete all Team server end-points and re-	30 Days (Aug 27 - Sep 25)	**Documentation, Team server **	James Marshall, Connaire Myall, and Lyle	Visual Studio, Blazor, MudBlazor, C#, Docker and Tail-

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member	Resources / Tools
		lated logic			William Richards	wind CSS
1.3.1	KAN-63 Operator management interface	Design and implement Operator management interface	6 Days (Sep 15 - Sep 20)	1.2	Con-naire Myall	Visual Studio, Blazor, Mud-Blazor, C# and Tailwind CSS
1.3.2	KAN-64 Agent Payload Builder	Design and implement Builder interface for the generation of Agent payload/s	6 Days (Aug 29 - Sep 03)	1.2.8	Con-naire Myall and Lyle William Richards	Visual Studio, Blazor, Mud-Blazor, C# and Tailwind CSS
1.3.3	KAN-65 Command Console	Design and implement command console User interface (UI) component for command input	6 Days (Sep 03 - Sep 08)	1.3.4	Con-naire Myall	Visual Studio, Blazor, Mud-Blazor, C# and Tailwind CSS
1.3.4	KAN-66 Command in-	Implement an interpreter	5 Days (Aug 29 - Sep 02)	1.2.7	James Marshall	Visual Studio, and C#

WBS Code	Task Name	Description	Duration	Predecessor/s	Team Member	Resources / Tools
	interpreter service	that parses and formats command strings into command-DTOs				
1.3.5	KAN-67 Login portal	Design and implement operator and administrator login	2 Days (Aug 28 - Aug 29)	1.2.4	Connaire Myall	Visual Studio, Blazor, Mud-Blazor, C# and Tailwind CSS
1.3.6	KAN-68 Agent management interface	Design and implement Agent management interface	6 Days (Aug 29 - Sep 03)	1.2.8	Connaire Myall	Visual Studio, Blazor, Mud-Blazor, C# and Tailwind CSS
1.3.7	KAN-69 Command console toolbar	Design and implement console toolbar that provides easy access to listener and pay-	5 Days (Sep 09 - Sep 13)	1.3.3	Connaire Myall	Visual Studio, Blazor, Mud-Blazor, C# and Tailwind CSS

WBS Code	Task Name	Descrip- tion	Duration	Prede- cessor/s	Team Member	Re- sources / Tools
		load gen- eration compo- nents				
1.3.8	KAN-70 Docker Con- tainer Con- script and or- chestra- tion	Create a Dockor script that con- tainer- ises the Client project and or- ches- trates commu- nication to Team server	4 Days (Sep 21 - Sep 25)	1.3.1-1.3.7	James Marshall	Docker

6. Non-functional Requirements

6.1. Overview

Non-functional Requirements define the quality attributes of the Trinity Command and Control (C2) Framework. Unlike functional requirements which specify what the system must do, non-functional requirements describe how the system should perform while maintaining security, reliability, usability and maintainability. These requirements ensure that the framework operates consistently under expected workloads and continues to meet the operational needs of Dark Development.

The Trinity framework has been designed to support authorised offensive security engagements through a secure, modular and scalable architecture. Consequently, the system must provide responsive performance, protect sensitive operational data, support multiple concurrent operators and agents and remain available throughout an engagement. The following sections describe the key quality attributes that will guide the implementation and future development of the framework.

6.2. Performance

Performance defines how efficiently the Trinity Command and Control (C2) Framework responds to user interactions, processes request and manages communication between operators, the C2 server and deployed agents. The framework should provide responsive performance to ensure that authorised operators can manage engagement without unnecessary delays. Efficient system performance is a key quality attribute that contributes to responsive, dependable and effective software systems [1].

The must support multiple concurrent operators, active listeners and connected agents while maintaining acceptable response times during normal operation. Operator actions such as issuing commands, generating payloads, viewing connected agents and retrieving task results will be processed promptly to provide an efficient user experience.

Communication between the C2 server and deployed agents should be optimised to minimise latency while ensuring reliable task delivery and result retrieval. As the number of connected agents increase the framework must also manage system resources efficiently to maintain stable performance.

6.3. Reliability and Availability

Reliability and availability are essential quality attributes of the Trinity Command and Control (C2) Framework. The system shall operate consistently during

authorised security engagements which ensures that operators can manage listeners, connected agents and issued tasks without unexpected failures. Reliable software minimises errors and maintains consistent functionality while availability ensures that the system remains operational whenever authorised user require access.

The Trinity framework will be designed to recover gracefully from non-critical failures and maintain the integrity of operational data. Error handling mechanisms should prevent system crashes caused by invalid input, network interruptions should be logged and appropriate feedback provided to the operator.

The framework should support continuous operation through stable server hosting and efficient management of system resources which will improve availability. Critical services which include authentication, listener management and agent communications which remains available throughout an engagement to minimise operational disruption.

6.4. Scalability

Scalability refers to the ability of the Trinity Command and Control (C2) Framework to maintain acceptable performance as the number of connected operators, listeners and deployed agents increase. A scalable architecture will ensure that the system can accommodate future growth without requiring significant redesign or negatively affecting existing functionality [2].

The Trinity framework shall support the addition of new agents, listeners and operator sessions while maintaining efficient communication and system responsiveness. The architecture must allow components such as agent management, task processing and listener services to operate independently where possible reducing resource contention and improving overall system efficiency.

To support future organisational requirements the framework should be designed to use a modular architecture that will enable new features and services to be integrated with minimal impact on existing components. Using this approach will improve the system ability to scale as operational demands increase while maintaining maintainability and performance.

6.5. Security

Security is a critical non-functional requirement of the Trinity Command and Control (C2) Framework because the system manages sensitive operational data, authentication users and communications with deployed agents. The framework shall implement appropriate security controls to ensure that only authorized operators can access system resources and perform administrative or operational

tasks. Protecting the confidentiality, integrity and availability of system data is essential for maintaining a secure and trustworthy platform [3].

The Trinity framework shall enforce user authentication and role-based access control to restrict access according to assigned permissions. Sensitive information, including operator credentials, task data and agent's communications, should be protected using secure communication protocols and appropriate encryption techniques where applicable. These measures should reduce the risk of unauthorized access, data tampering and information disclosure.

The framework must maintain detailed audit logs of authenticated attempts, issued commands and system events to support accountability and incident investigation. Regular security updates, secure coding practices to minimise vulnerability and improve the overall security of the platform [1].

6.6. Usability

Usability is a critical non-functional requirement that ensures the Trinity Command and Control (C2) Framework provides an intuitive, efficient and user-friendly interface for authorised operators. A usable system enables users to perform tasks accurately with minimal effort which reduces the likelihood of operational errors while improving productivity and overall user satisfaction [4].

The Trinity framework will provide a consistent interface with clear navigation, logical workflows and meaningful feedback to support efficient management of listeners, connected agents and issued tasks. Frequently used functions such as selecting agents, executing commands and monitoring task results will be easily accessible to minimise the time required to complete operational activities.

To further enhance usability, the framework must provide information error messages, clear status indicators and responsive feedback for user actions. The interface should present operational information in a structured and organized manner which enables operators to quickly identify connected agents, monitor task progress and interpret command output. Prioritising usability throughout the system design contributes to improved software quality and a more effective user experience.

6.7. Maintainability

Maintainability refers to the ease with which the Trinity Command and Control (C2) Framework can be modified, updated and extended throughout its lifecycle. A maintainable system enables developers to correct defects, implement new features and improve existing functionality without negatively affecting other components of the platform as operational and organizational requirements change [1].

The Trinity framework should adopt a modular software architecture in which components such as authentication, listener management, agent communication and task processing are logically separated. By using this approach it will reduce dependencies between system components, simplifies maintenance activities and enables developers to update individual modules with minimal impact on the rest of the system.

The framework should also follow consistent coding standard, maintain comprehensive documentation and implement appropriate version control practices to support collaborative development. Clear documentation and well-structured source code improve maintainability by allowing developers to understand, troubleshoot and enhance the system more efficiently throughout its development lifecycle.

6.8. Compatibility and Portability

Compatibility and portability ensure that the Trinity Command and Control (C2) Framework can operate effectively across supported environments while allowing components to be deployed, maintained and updated with minimal modification. These quality attributes improve the flexibility of the framework by enabling it to integrate with different operating environments and technologies while supporting future system enhancements [5].

The Trinity framework should be designed using widely accepted development standards and technologies to maximise compatibility between system components. The server, database and client's application should communicate through well-defined interfaces and standard communication protocols to ensure reliable interaction between different modules of the framework.

The framework should be portable across supported deployment environments with minimal configuration changes. Where practical, platform-dependent functionality should be isolated to reduce the effort required to deploy or migrate the system to different operating systems or hosting environments. Designing the framework with compatibility and portability in mind improves long-term maintainability and reduces the complexity of future system upgrades.

6.9. Accessibility

Accessibility is an important non-functional requirement that ensures the Trinity Command and Control (C2) Framework can be used effectively by authorised operators with different abilities and operating preferences. Incorporating accessibility into software design improves the overall user experience while supporting other quality attributes such as usability and functionality which result in software that is easier to operate and maintain [6].

The Trinity framework should provide an interface that is clear, consistent and easy to navigate. User interface elements should include readable fonts, appropriate colour contrast, descriptive labels and organised layouts that enable operators to identify connected agents, monitor task progress and execute commands efficiently. Important system notifications and operational information should be presented clearly to reduce the likelihood of user error during authorised security engagements.

The framework should also support keyboard navigation where appropriate and provide informative feedback for user actions and validation errors. Accessibility should be considered throughout the design process to ensure that essential functionality remains available to all authorised users while contributing to the overall quality of and usability of the Trinity framework [3].

6.10. Logging and Auditability

Login and auditability are essential non-functional requirements that support system security, accountability and operational monitoring within the Trinity Command and Control (C2) Framework. Comprehensive audit logs provide a record of system activities, enabling administrators to monitor user actions, investigate security incidents and verify that authorised operations are performed in accordance with organisational policies. Effective audit logging also improves traceability and assists in detecting abnormal or unauthorised behaviour [7].

The Trinity framework should record significant system events which includes user authentication attempts, command execution, agent registration, task assignment, configuration changes and administrative actions. Log records should contain sufficient contextual information such as timestamps, user identities and affected system components which support incident investigation and troubleshooting while maintaining the integrity of audit information.

To improve accountability audit logs must be protected against unauthorised modifications and retained according to organisational requirements. The framework should also provide mechanisms for reviewing and analysing logged events to support operational oversight, troubleshooting and security auditing. Maintaining comprehensive and reliable audit logs strengthens the overall security and quality of the Trinity framework while supporting long-term system management [3].

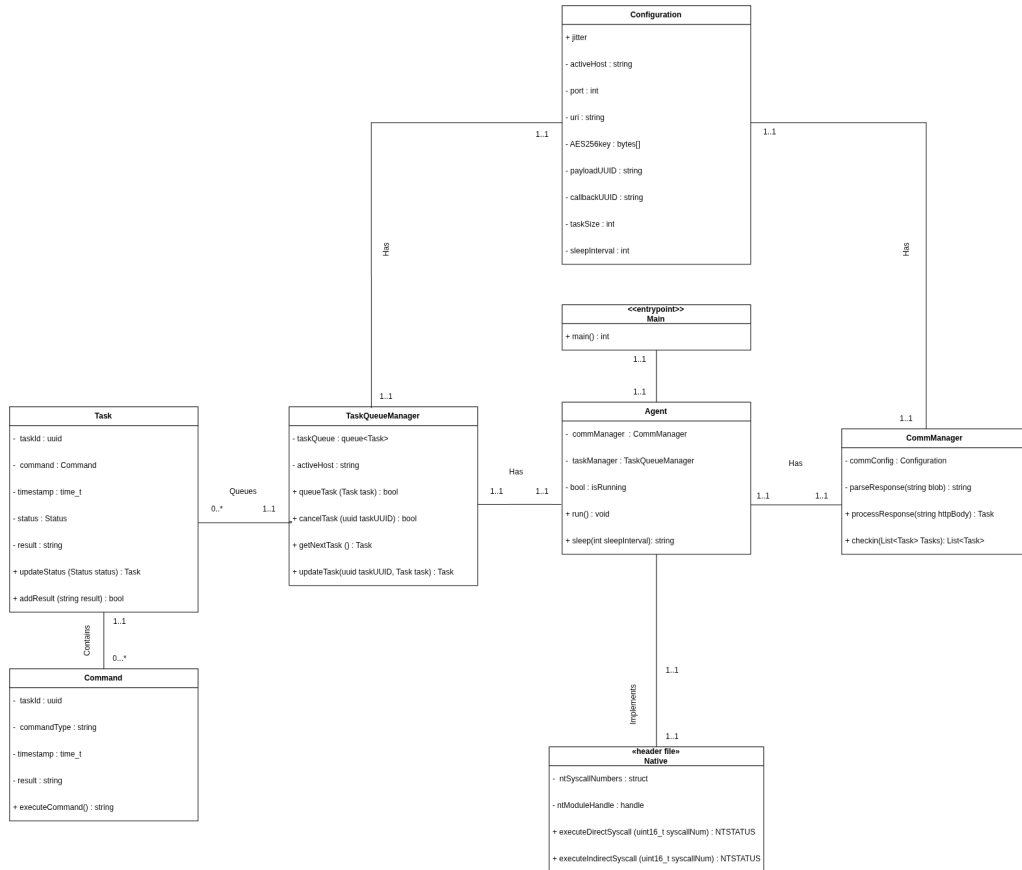
6.11. Summary

The non-functional requirements defined for the Trinity Command and Control (C2) Framework establish the quality standards necessary to support a secure, reliable and maintainable system. Attributes such as performance, scalability, se-

curity, usability, maintainability, compatibility, accessibility, and logging ensure that the framework can meet both current and future operational requirements. By incorporating these quality attributes into the system design, the Trinity framework will provide a robust and effective platform for authorised command and control operations.

7. Analysis Artifacts

7.1. Domain Models - Agent

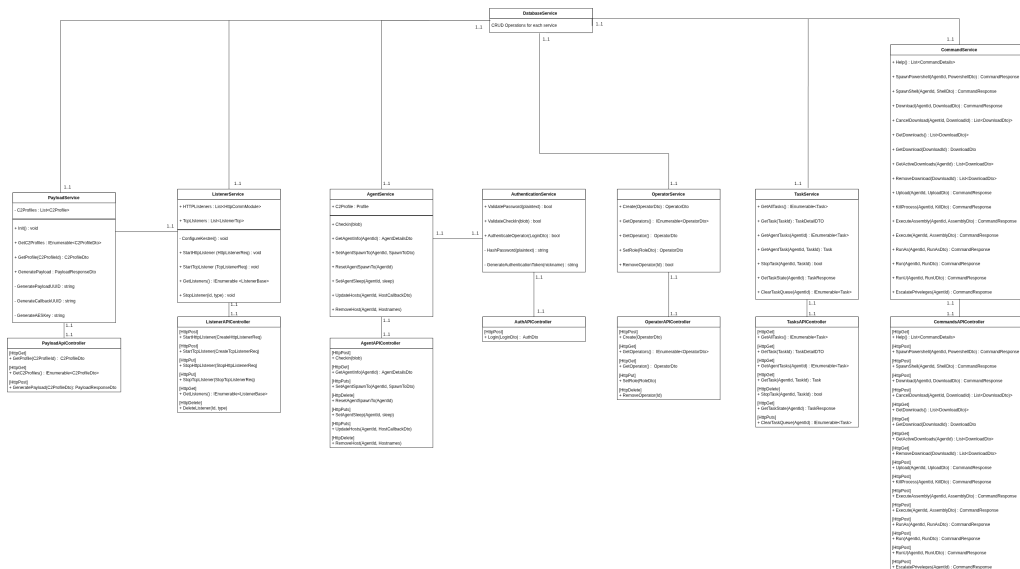


7.1.1. Agent Domain Model Explanations

Entity	Explanation
Configuration	Represents configuration data used by the Agent. There is one instance of the Configuration class referenced by the TaskQueueManager and CommManager.
CommManager	Represents the communication-management component of the Agent. It contains a Configuration reference and provides operations for parsing a response, processing a response into a Task, and sending a checkin request to the teamserver.
Agent	Represents the Agent component. It contains references to CommManager, TaskQueueManager and implements the functions stubs defined in the Native header file for the purpose of executing commands. Further-

Entity	Explanation
	more it provides maintenance methods such as run and sleep.
Main	Represents the entry point of the Agent application.
Native	Provide ntdll.dll function stubs declared in a header file.
TaskQueueManager	Represents the component responsible for managing a task queue. It contains a task queue and provides operations for adding, cancelling, retrieving, and updating tasks. There can be many Task associated with only one instance of a TaskQueueManager.
Task	Represents a task managed by TaskQueueManager. It contains a task ID, command, timestamp, status, and result. It provides operations for updating the task status and adding a result.
Command	Represents a command associated with a Task. It contains a task ID, command, timestamp, status, and result. It provides a method for executing a command that is overridden by a sub-class.

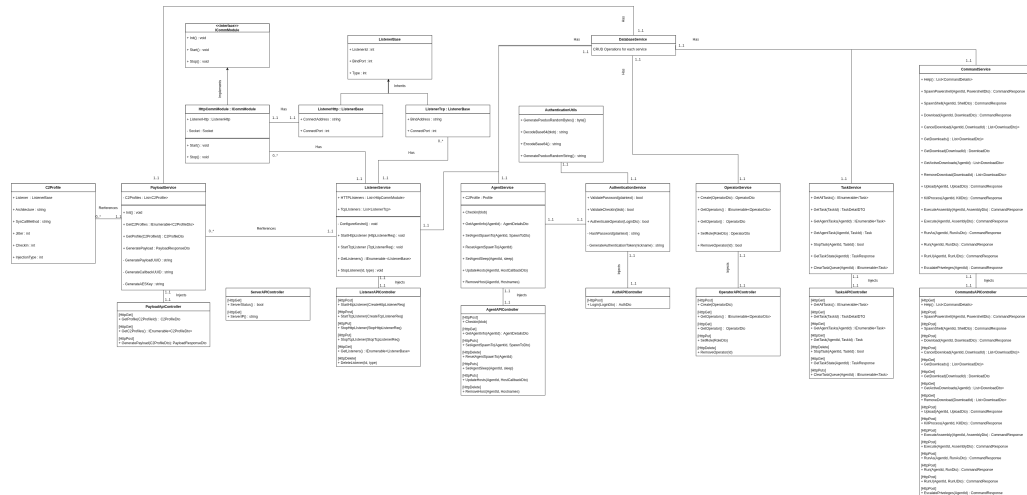
7.2. Domain Models - Team server



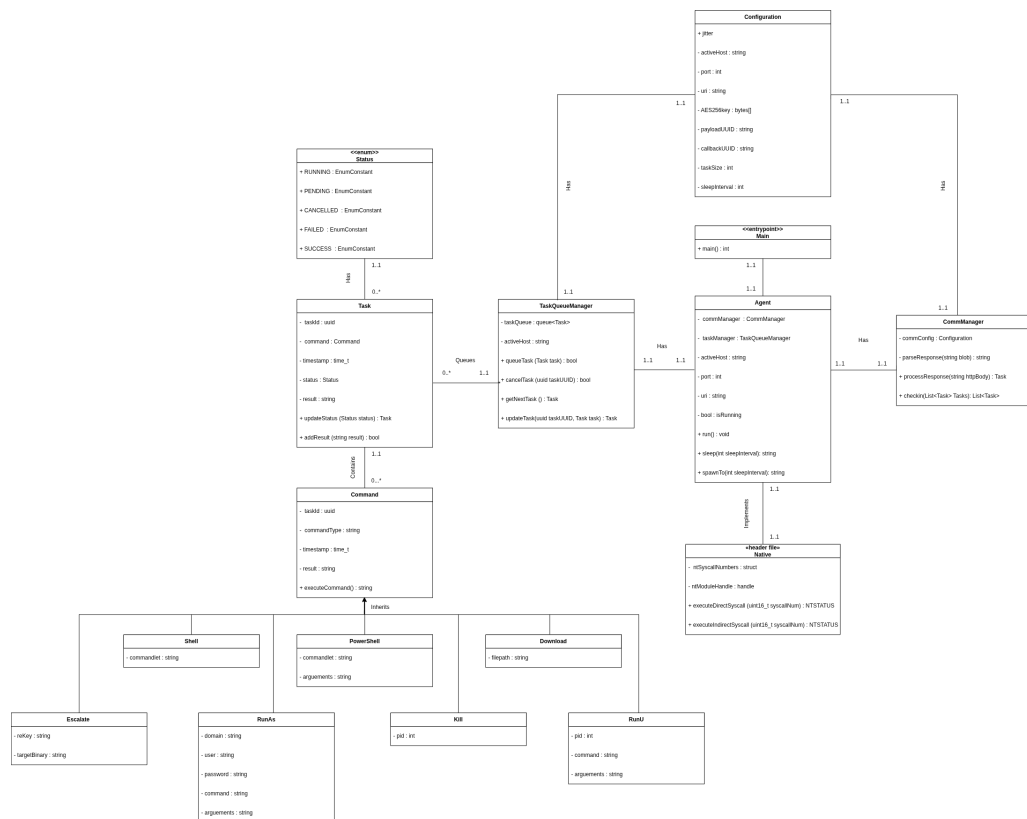
7.2.1. Team server Domain Model Explanations

System Service	Interaction Performed by the Operator
Authentication Service	Authenticates the operator before access to the Team Server is granted. It validates the operator's authentication request and ensures that only authorised users can access the system.
Operator Service	Retrieves and manages operator account information within the team server. It provides functionality for managing the accounts used by operators to access and interact with the system.
Payload Service	Generates deployable payloads using selected communication profiles and configuration. It manages the information required to create payloads that allow agents to communicate with the team server.
Listener Service	Configures and manages the communication listeners used by the team server. It provides functionality for starting, stopping, retrieving and managing listeners used for agent communication.
Agent Service	Registers, monitors and manages deployed agents connected to the team server. It provides functionality for retrieving agent information, updating Agent details and managing the operational state of connected agents.
Task Service	Creates, assigns and manages operational tasks for agents connected to the team server. It provides functionality for assigning tasks to agents, monitoring task status, stopping tasks and managing the task queue.
Command Service	It provides functionality for managing command assignment requests and handling the responses associated with those operations that are sent to the client application.
Database Service	Stores and retrieves persistent operational information used by the team server. It supports the other services by providing access to stored account, agent, task, payload, C2 profile, command, listener, and configuration information.

7.3. Design Artefacts - Team server Implementation Classes



7.4. Design Artefacts - Agent Implementation Classes



8. Implementation

This section documents the critical workflows of the operator, team server and agent at a high level. Therefore specific implementation details have been purposefully omitted or abstracted.

8.1. Operator task issuing workflow

The sequence diagram below represents the workflow of an operator issuing tasks and the broadcasting of updates upon task state change via push notifications.

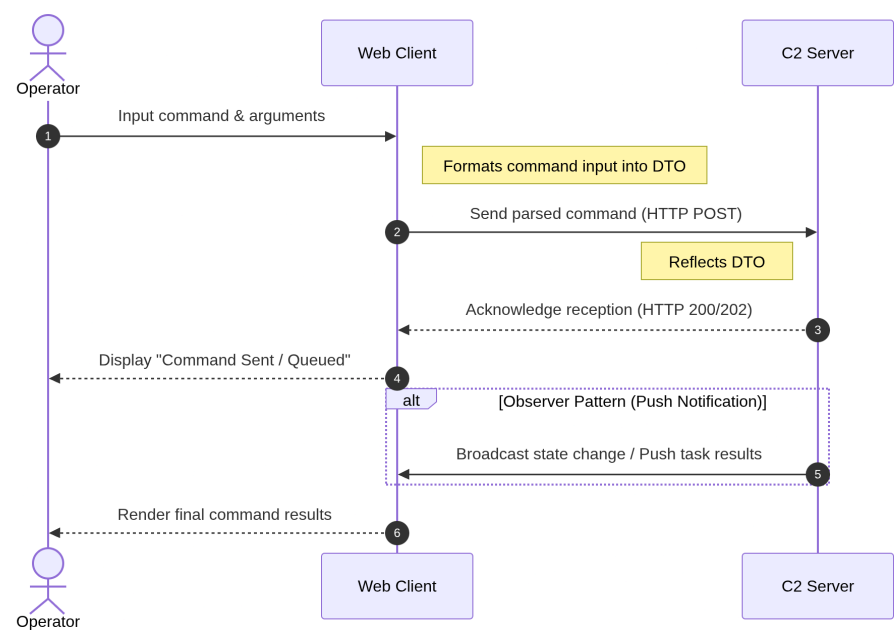
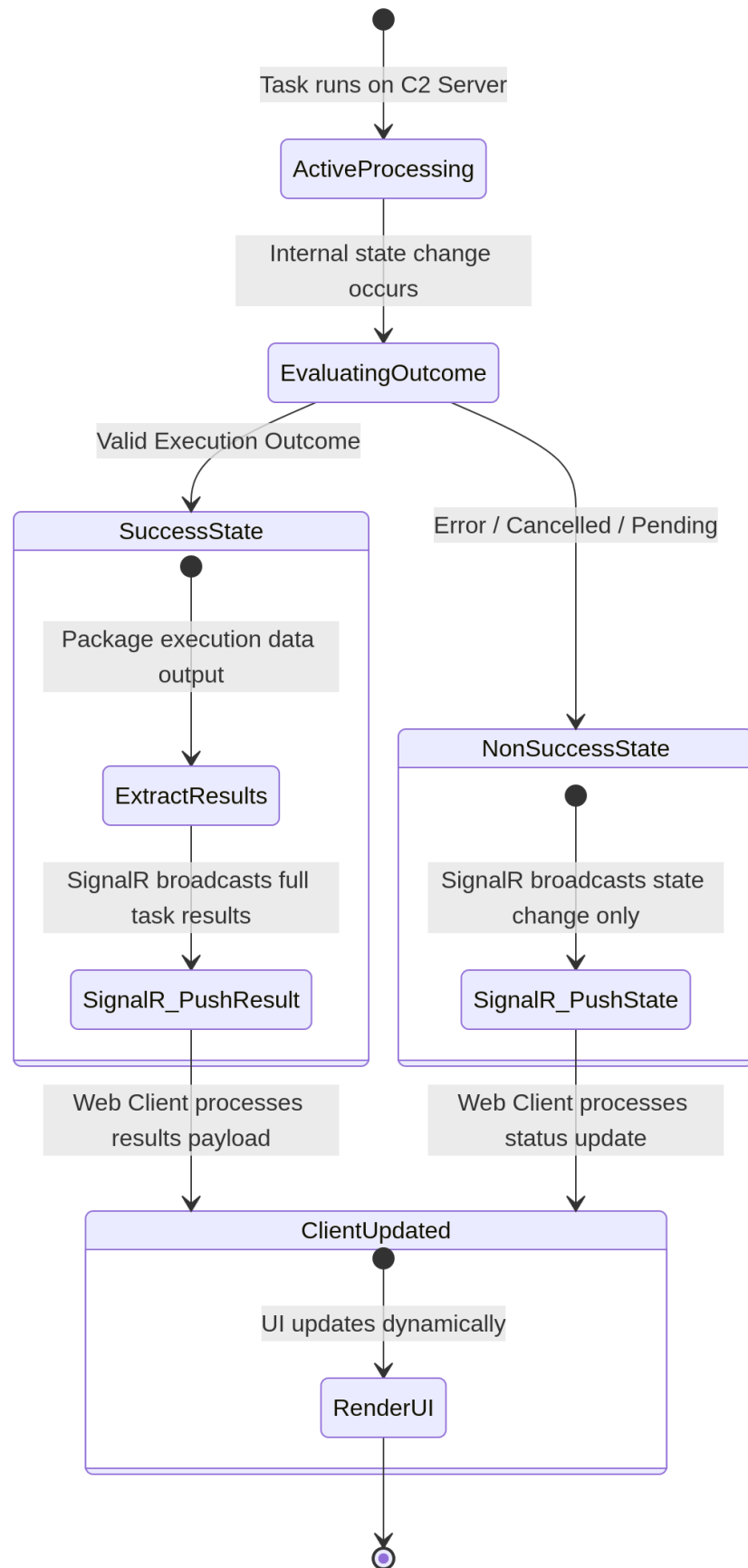


Figure 2: Operator task issuing sequence diagram. View diagram code on mermaid.live.

The state diagram outlines differing state changes and how it influences the behaviour of the push notifications implemented via the Observer Pattern.

Figure 3: Operator task issuing state diagram. View diagram code on mermaid.live.

8.2. Operator task queue retrieval workflow

The sequence diagram below represents the workflow of an operator requesting to view the task queue of an agent and the subsequent rendering logic after retrieval.

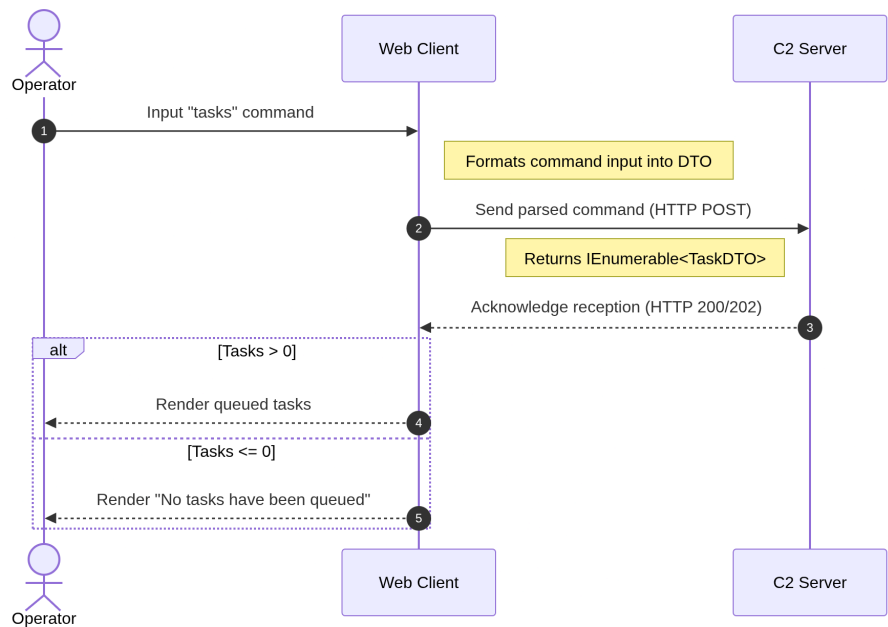


Figure 4: Operator task queue retrieval sequence diagram. View diagram code on mermaid.live.

The state diagram represents the rendering logic of the client application when evaluating the returned results retrieved from the team server.

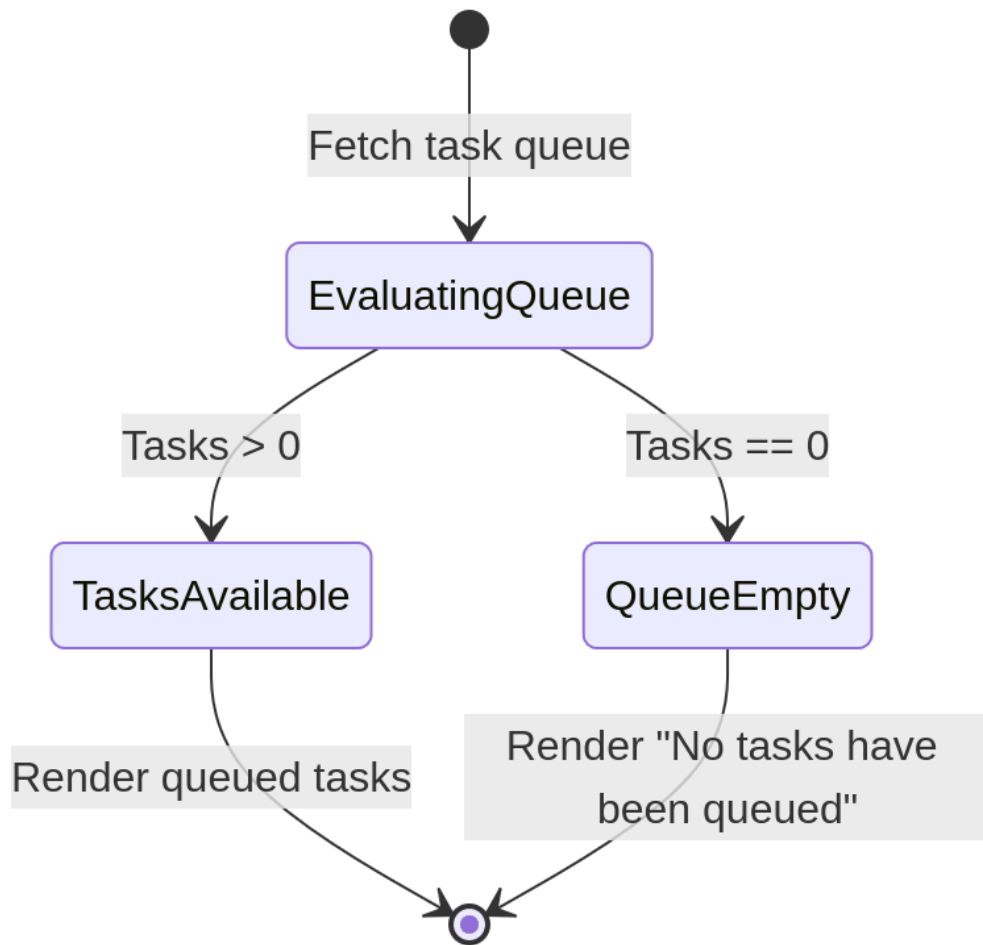


Figure 5: Operator task queue retrieval state diagram. View diagram code on mermaid.live.

8.3. Operator task cancellation workflow

The sequence diagram below represents the workflow of an operator requesting to remove a task from an agent's task queue.

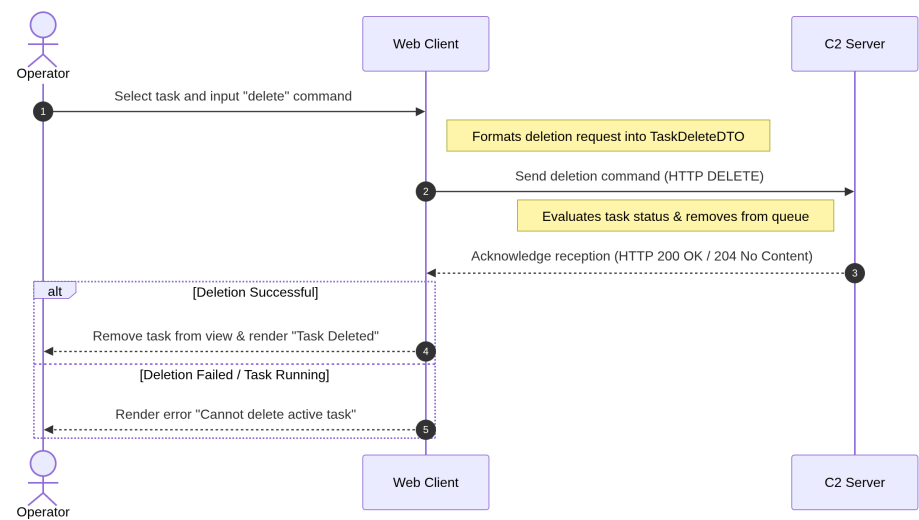


Figure 6: Operator task cancellation sequence diagram. View diagram code on mermaid.live.

The state diagram represents the evaluation of a task deletion request on the team server before updating the client UI.

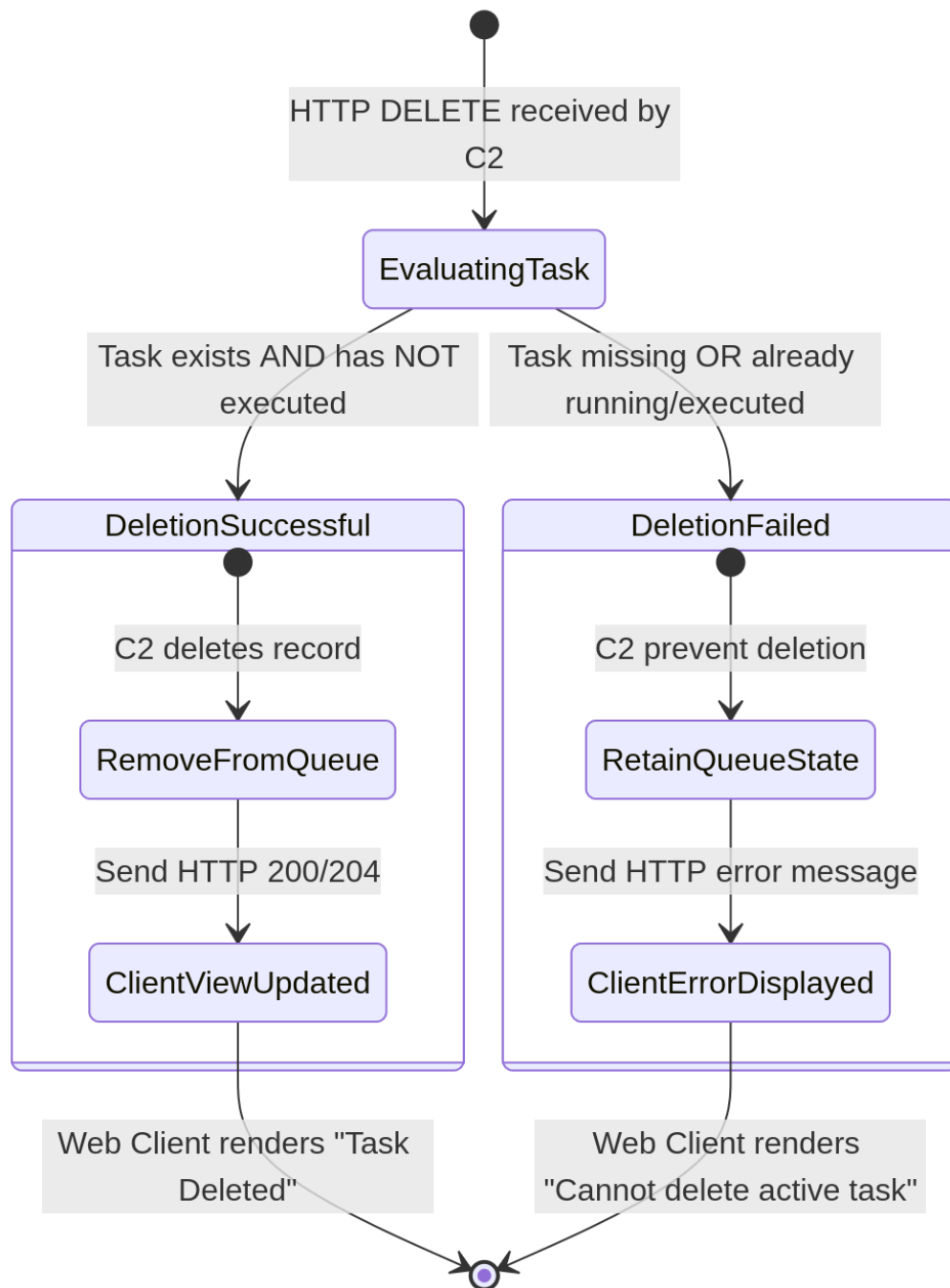


Figure 7: Operator task cancellation state diagram. View diagram code on mermaid.live.

8.4. Operator login workflow

The sequence diagram below represents the workflow of an operator attempting to authenticate to gain access to the the client dashboard.

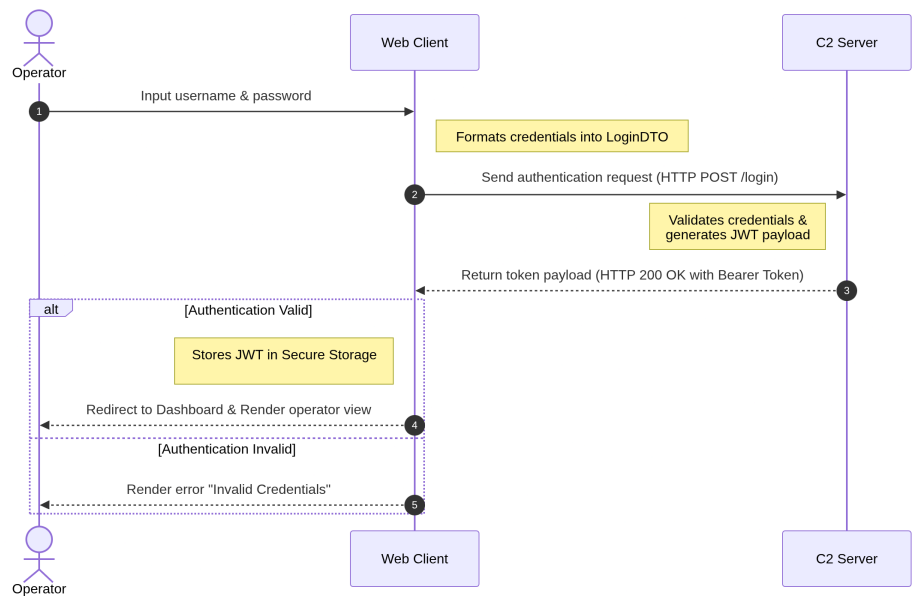
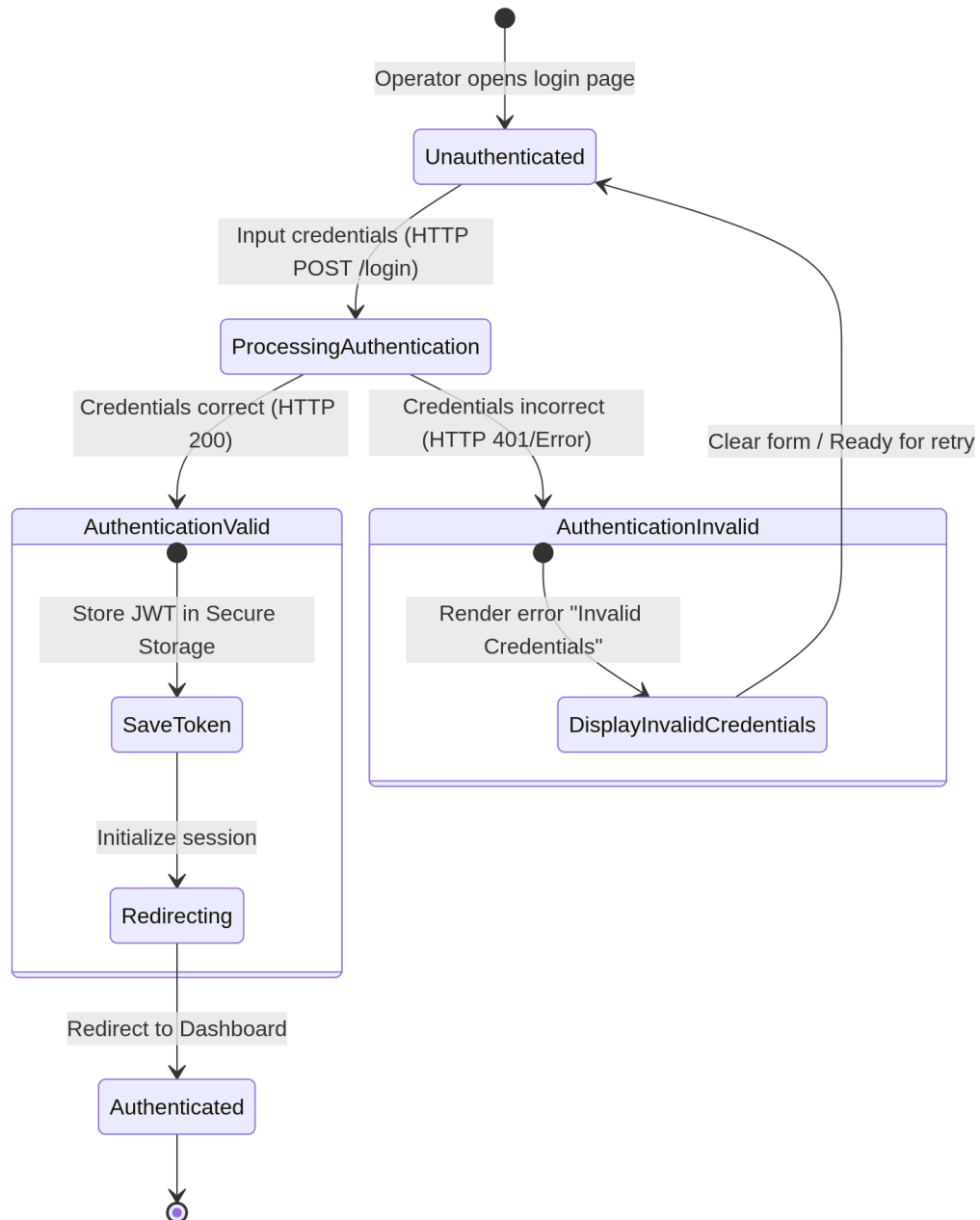


Figure 8: Operator login sequence diagram. View diagram code on mermaid.live.

The state diagram represents the evaluation of user authentication credentials on the team server application, determining whether a JWT token is successfully issued or rejected

Figure 9: Operator login state diagram. View diagram code on mermaid.live.

8.5. Agent initial checkin workflow (Static AES256 encryption)

The state diagram represents the registration process of an agent static AES256 encryption checkin request.

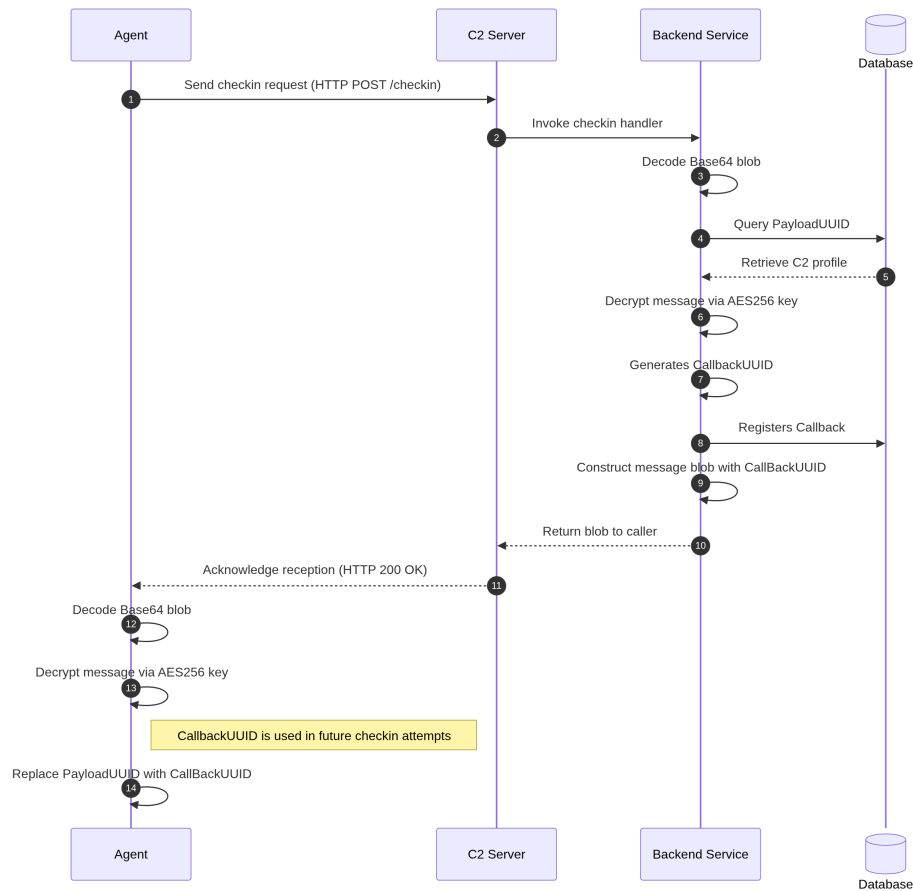


Figure 10: Agent initial checkin sequence diagram. View diagram code on mermaid.live.

8.5.1. Agent Initial Check-in Sequence Explanation

The Agent initial check-in process establishes the initial communication between the Agent and the C2 Server. The process begins when the Agent sends a check-in request to the C2 Server using an HTTP POST request to the /checkin endpoint.

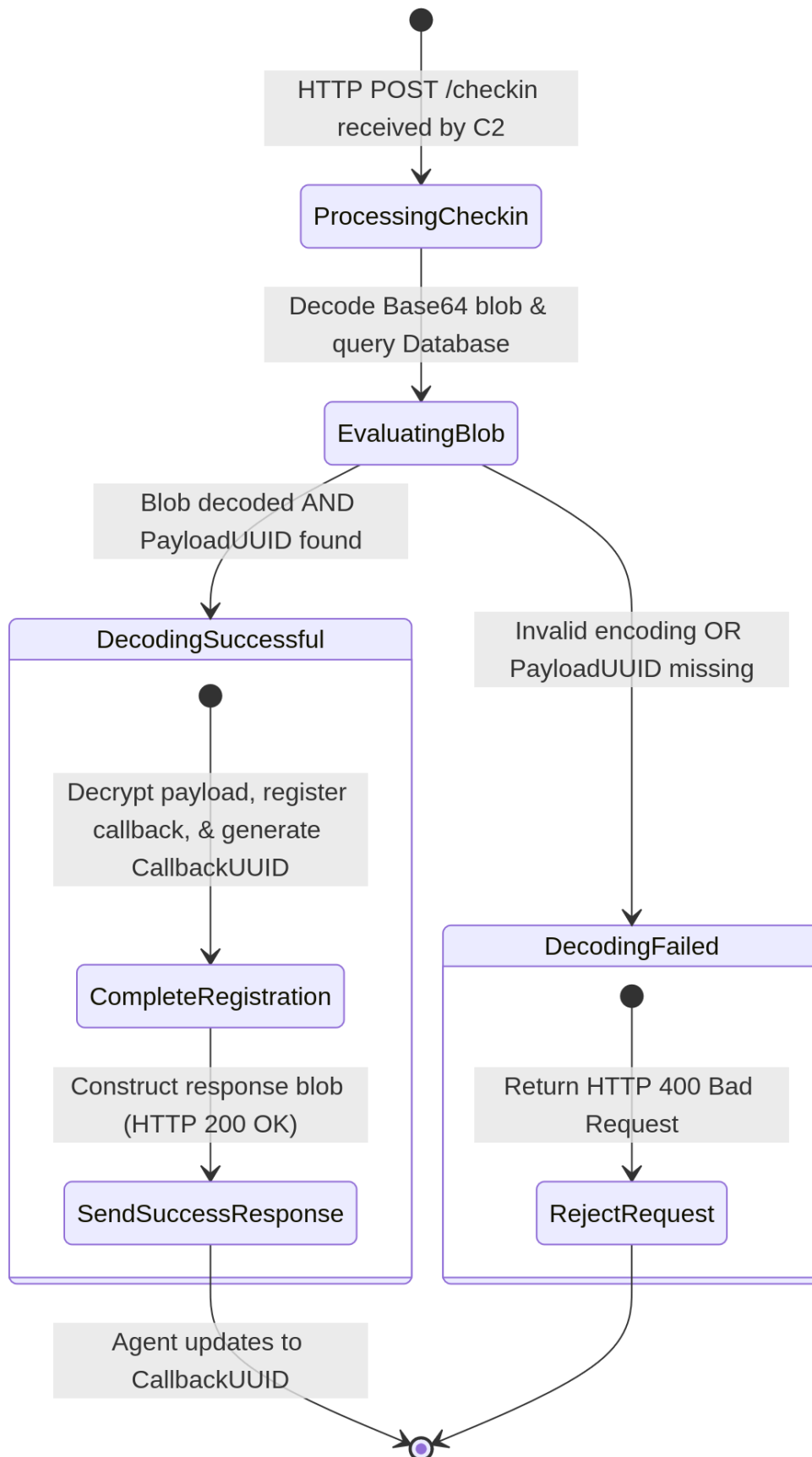
The C2 Server passes the request to the backend service responsible for processing the check-in. The backend service first decodes the Base64-encoded communication blob and extracts the PayloadUUID. The PayloadUUID is then used to query the database and retrieve the C2 profile associated with the Agent.

After retrieving the appropriate C2 profile, the backend service decrypts the communication using the associated AES-256 key. Once the initial check-in has been successfully processed, the service generates a new CallbackUUID and registers the callback information in the database.

The backend service then constructs a response containing the CallbackUUID and returns the response to the C2 Server. The C2 Server acknowledges the request with an HTTP 200 OK response and passes the returned communication blob to the Agent.

The Agent decodes the Base64-encoded response and decrypts the message using the AES-256 key. The Agent then replaces its original PayloadUUID with the newly assigned CallbackUUID. The CallbackUUID is subsequently used to identify the established Agent session during future check-in attempts.

The state diagram represents the validation and processing of an agent checkin request on the team server.

Figure 11: Agent initial checkin state diagram. View diagram code on mermaid.live.

8.6. Agent fully-established checkin workflow (Static AES256 encryption)

The sequence diagram represents the periodic fully-established checkin and task synchronisation lifecycle between an agent and team server.

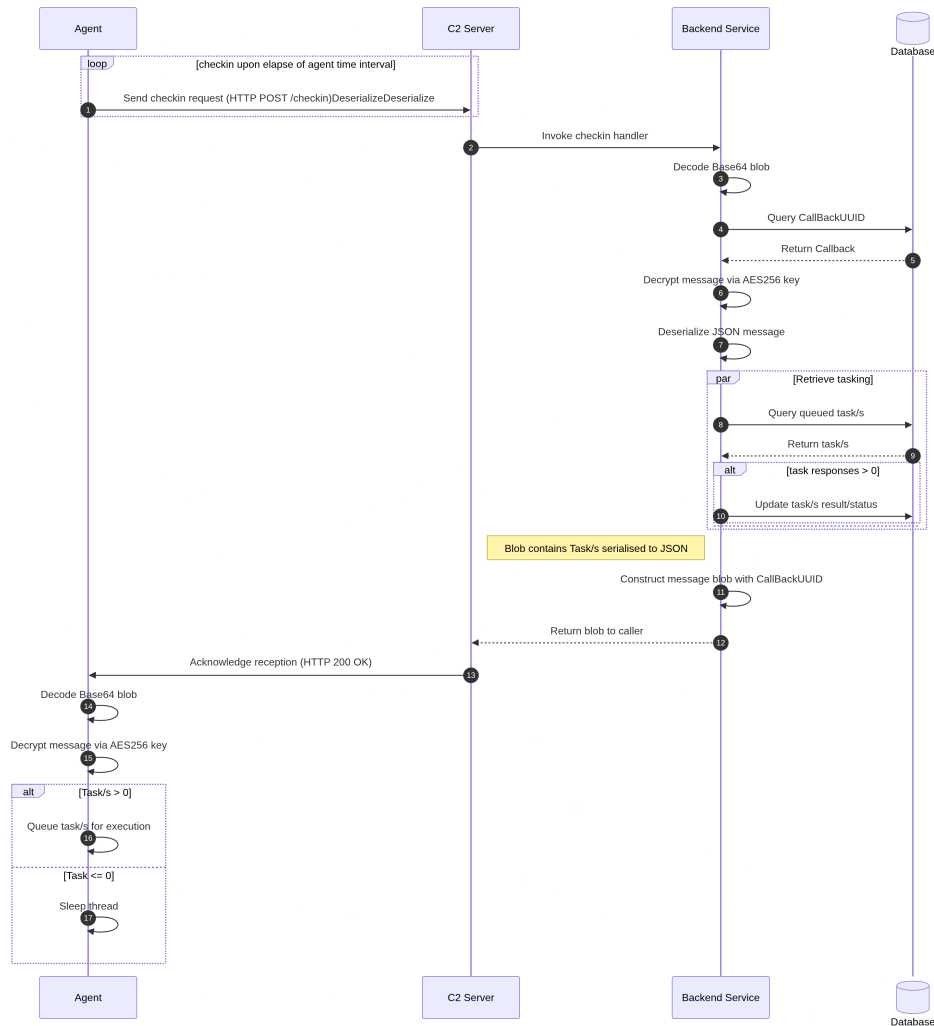


Figure 12: Agent fully-established checkin sequence diagram. View diagram code on mermaid.live.

8.6.1. Agent Fully-Established Check-in Explanation

The fully-established check-in workflow shows how the Agent communicates with the C2 Server after the initial check-in has been completed. The Agent waits for its configured check-in interval and then sends another HTTP POST request to the /checkin endpoint.

When the C2 Server receives the request, it passes it to the backend service for processing. The backend service first decodes the Base64-encoded commu-

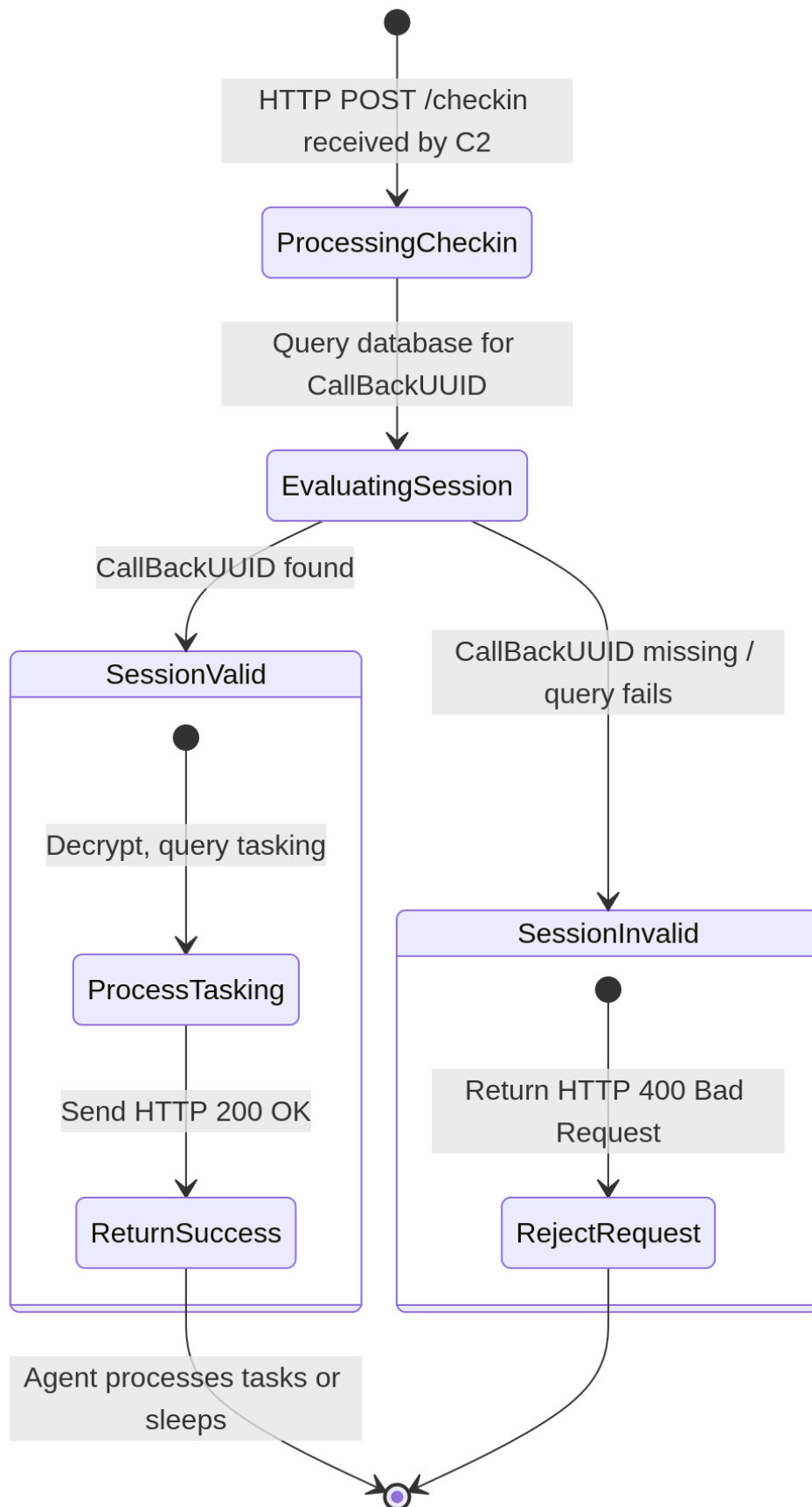
nication blob and uses the Agent's `CallbackUUID` to find the correct callback information in the database.

Once the Agent has been identified, the backend service decrypts the message using the established AES-256 key and converts the JSON data into a format that the service can process. It then checks the database to see whether there are any tasks waiting for the Agent.

If there are queued tasks, they are included in the response sent back to the Agent. If the Agent has also sent back results from tasks it previously completed, the backend service updates the relevant task results and statuses in the database. The service then creates the response containing the `CallbackUUID` and any available tasks and sends it back through the C2 Server.

The Agent receives the response, decodes the Base64 data, and decrypts it using the AES-256 key. If there are tasks in the response, the Agent adds them to its local execution queue. If there are no tasks, the Agent waits until its next scheduled check-in.

The state diagram represents the validation and processing of an agent's fully-established checkin request on the team server.

Figure 13: Agent fully-established checkin state diagram. View diagram code on mermaid.live.

8.7. Agent direct system call command execution workflow

The sequence diagram represents the workflow of an agent executing commands using direct system calls on Windows. The execution of the system call does not occur in ntdll.dll's memory address space, and in alternative implementations reference of this native library is completely unnecessary if the system call number is known.

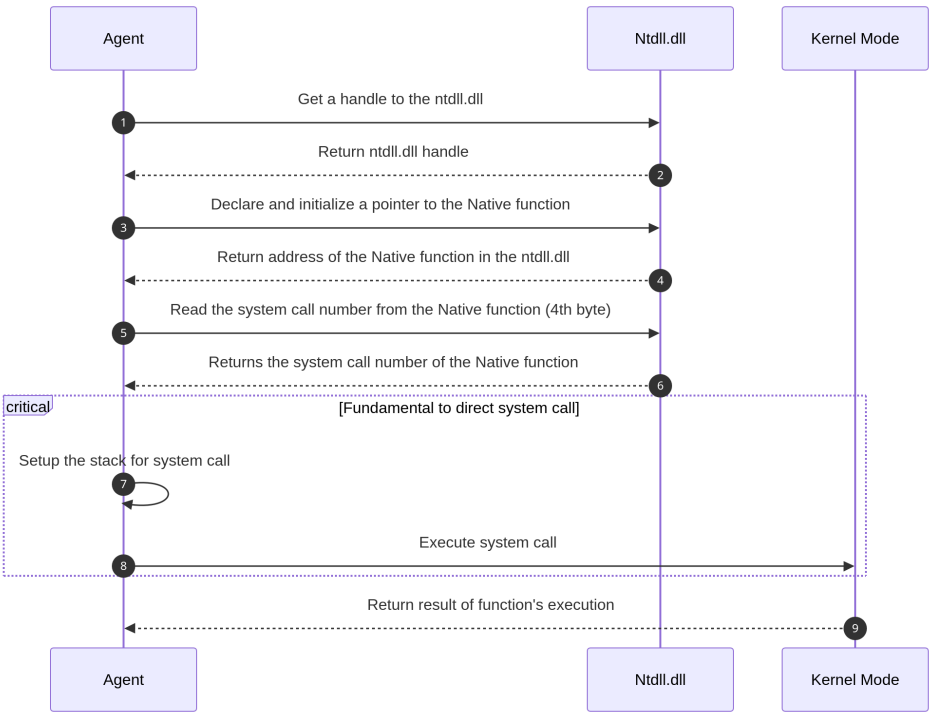


Figure 14: Agent direct system call command sequence execution diagram. View diagram code on mermaid.live.

The state diagram represents the classification of x64 registers under the Microsoft x64 calling convention, dividing them into volatile and non-volatile states before concluding at the function return execution.

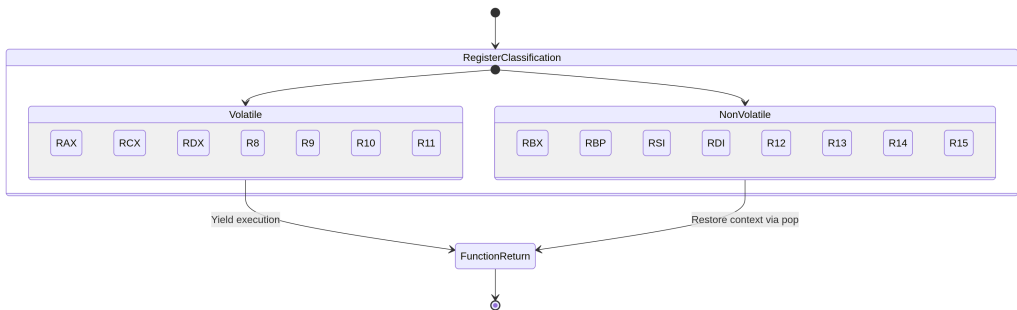


Figure 15: Agent direct system call register state diagram. View diagram code on mermaid.live.

8.8. Agent indirect system call command execution workflow

The sequence diagram represents the workflow of an agent executing commands using indirect system calls on Windows. The primary difference is the execution of the system call takes place in ntdll.dll's memory address space. The Microsoft x64 calling convention still applies to indirect system calls, therefore the state of registers remains the same as direct system calls.

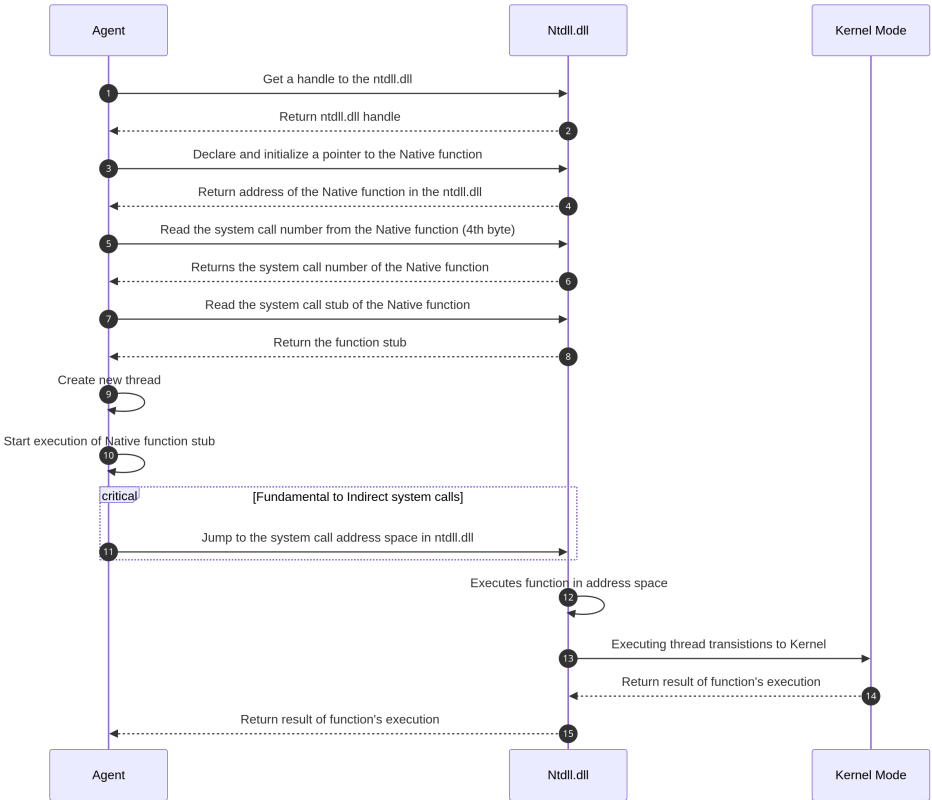


Figure 16: Agent indirect system call command sequence execution diagram. View diagram code on mermaid.live.

8.9. Team server HTTP/S listener creation workflow

The sequence diagram represents the creation and validation of an HTTP/S listener configuration on the team server. It should be noted that an HTTP/S listener within the context of the system plays an active role in the processing of HTTP/S requests sent to the team server, as it opens a port for purposes of agent communication. Additionally it acts as a configuration when building agent payloads, as a listener has to be specified to stamp/patch protocol specific configurations and to satisfy conditional preprocessing directives during compilation.

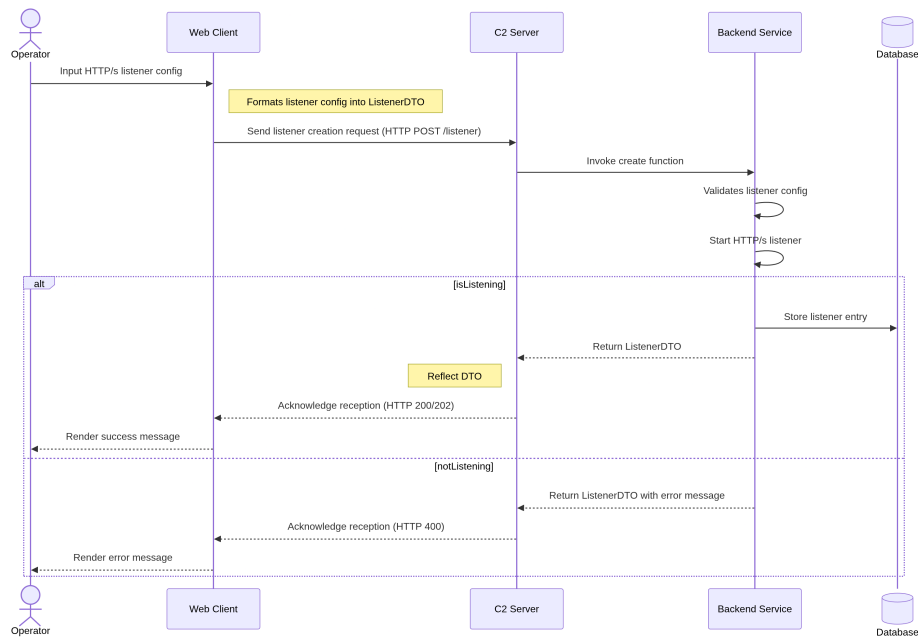


Figure 17: Team server HTTP/s listener creation sequence diagram. View diagram code on mermaid.live.

8.9.1. Team Server HTTP/S Listener Creation Explanation

The HTTP/S listener creation process starts when the operator provides the listener configuration through the web client. The web client formats the configuration into a `ListenerDTO` and using a HTTP POST request send it to the C2 Server.

The C2 Server passes the request to the backend service, where the listener configuration is validated. The backend service then attempts to start the HTTP/S listener using the supplied configuration.

If the listener is started successfully, the listener information is stored in the database. The backend service then returns the `ListenerDTO` to the C2 Server, which acknowledges the request and allows the web client to display a success message to the operator.

If the listener cannot be started, the backend service returns the `ListenerDTO` together with an error message. The C2 Server then returns an HTTP 400 response, allowing the web client to display an error message to the operator.

8.10. Team server HTTP/s listener deletion workflow

The sequence diagram represents the deletion workflow of a HTTP/S listener deletion request on the team server.

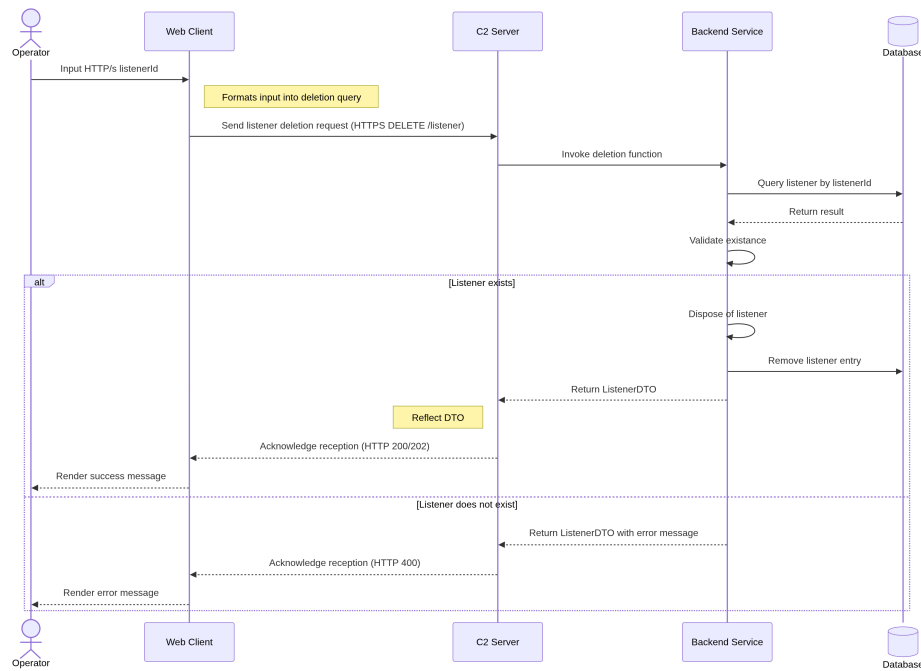


Figure 18: Team server HTTP/s listener deletion sequence diagram. View diagram code on mermaid.live.

8.10.1. Team Server HTTP/S Listener Deletion Explanation

The HTTP/S Listener Deletion process starts when the operators selects a listener and provides its listener ID through the web client. The web client uses this information to create the deletion request and sends it to the C2 Server.

The C2 Server receives the request and passes it to the backend service, which will handle the deletion process. First the backend service check the database using the provide listener ID to determine whether the listener exists.

If the listener exists the backend service disposes of the active listener and removes its corresponding entry from the database. If the listener does not exist, the backend service returns an error message to the client.

8.11. Team server HTTP/s listener update workflow

The sequence diagram represents the update lifecycle of an HTTP/S listener configuration on the team server.

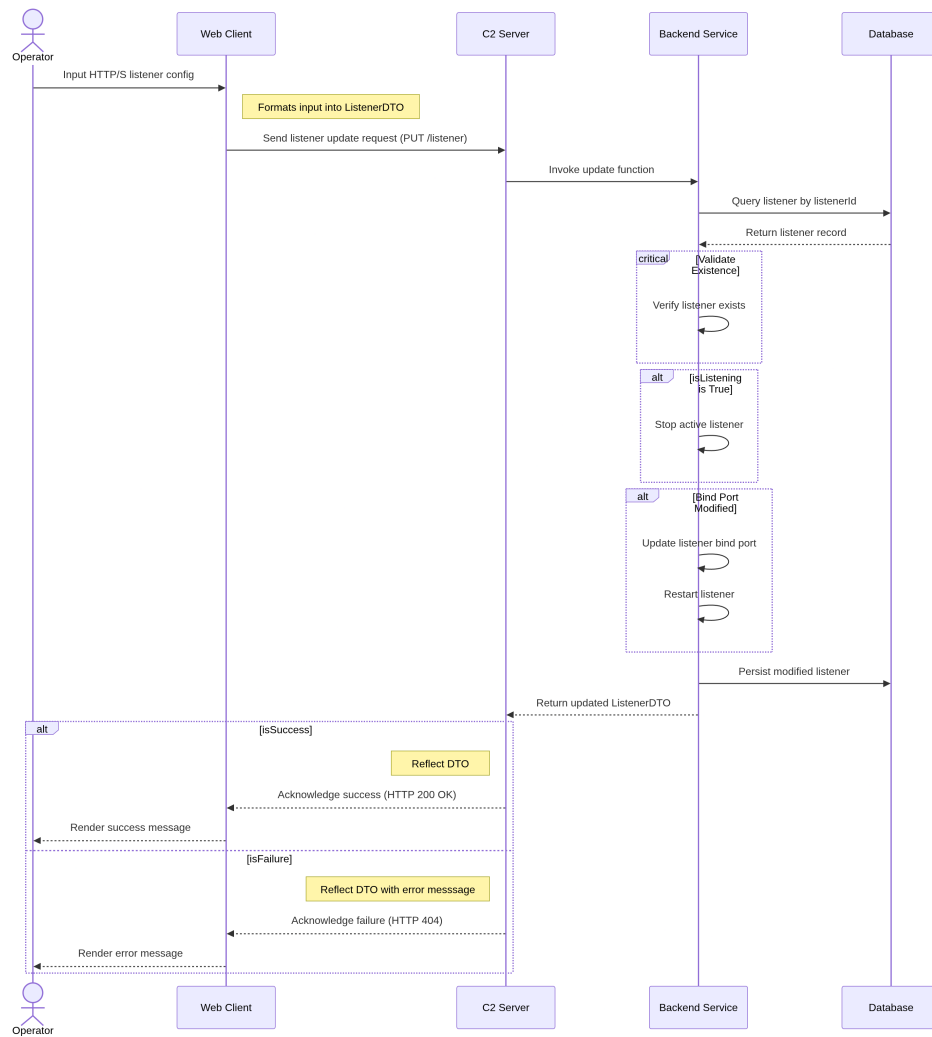


Figure 19: Team server HTTP/S listener update sequence diagram. View diagram code on mermaid.live.

8.11.1. Team Server HTTP/S Listener Update Explanation

The update process starts when the operator changes the configuration of an existing HTTP/S listener through the web client. The web client formats the updated configuration into a `ListenerDTO` and sends it to the C2 Server using an HTTP PUT request.

The C2 Server passes the request to the backend service, which first queries the database to find the listener using its `listenerId`. The backend service then checks that the listener exists before continuing with the update.

If the listener is currently running, the backend service will stop the active listener so that its configuration can be changed. If the bind port has been modified so will the service update the listener's bind port and then start the listener again using the new configuration.

Once the listener has been updated, the modified listener information is saved to the database. The backend service then returns the updated ListenerDTO to the C2 Server.

If the update is successful, the C2 Server returns an HTTP 200 response and the web client displays a success message to the operator. If the update fails, the C2 Server returns an HTTP 400 response together with an error message, allowing the web client to display the failure to the operator.

This will allow an existing HTTP/S listener to have its configuration changed while keeping the listener information stored in the database up to date.

8.12. Team server TCP listener creation workflow

The sequence diagram represents the creation and validation of a TCP listener configuration on the team server. It should be noted that a TCP listener within the context of the system acts purely as a configuration for the generation of peer-to-peer agent payloads. Unlike that of its HTTP/S listener counterpart it plays no active role on the team server with regards to agent request processing.

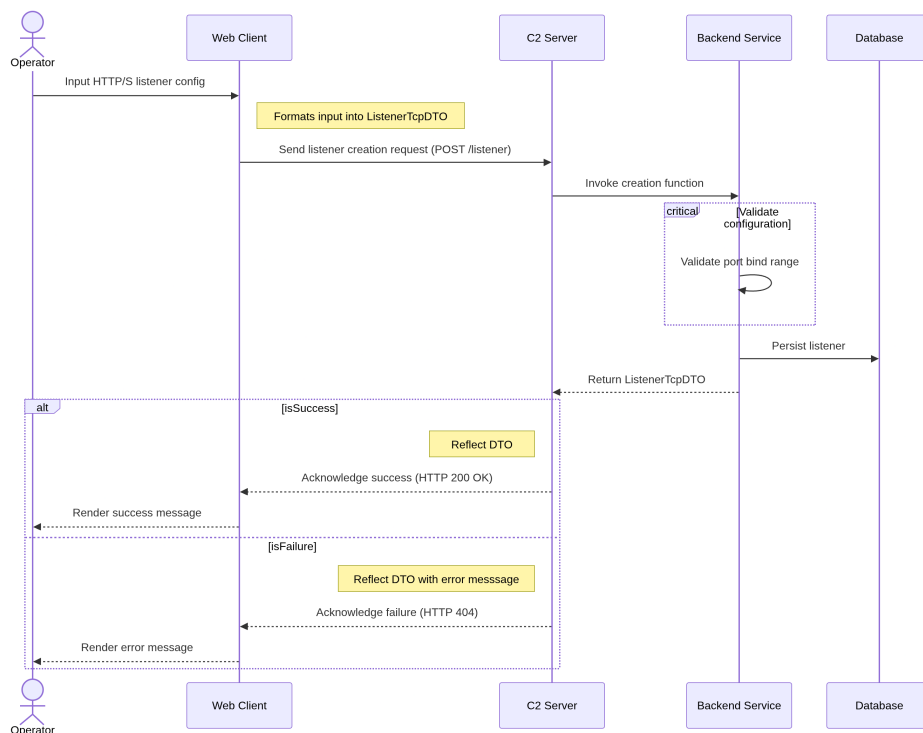


Figure 20: Team server TCP listener creation sequence diagram. View diagram code on mermaid.live.

8.12.1. Team Server TCP Listener Creation Explanation

The TCP listener creation process starts when the operator provides the required TCP listener configuration through the web client. The configuration is sent to the C2 Server, where it is passed to the backend service for processing.

The backend service validates the supplied TCP listener configuration to make sure that the required information is present and can be used. Unlike an HTTP/S listener, the TCP listener does not need to be started as an active listener on the Team Server. Instead, the configuration is used when generating peer-to-peer Agent payloads.

If the configuration is valid, the listener information is stored so that it can be used by the system when generating the appropriate Agent payload. The backend service then returns the listener information to the C2 Server, which sends the result back to the web client.

If the configuration is not valid, the backend service returns an error instead of creating the listener configuration. The C2 Server then passes the error back to the web client so that the operator can see that the creation was unsuccessful.

The TCP listener workflow is used to create and validate the configuration required for a peer-to-peer Agent communication rather than creating an active network listener on the Team Server.

8.13. Team server TCP listener update workflow

The sequence diagram represents the update lifecycle of a TCP listener configuration on the team server.

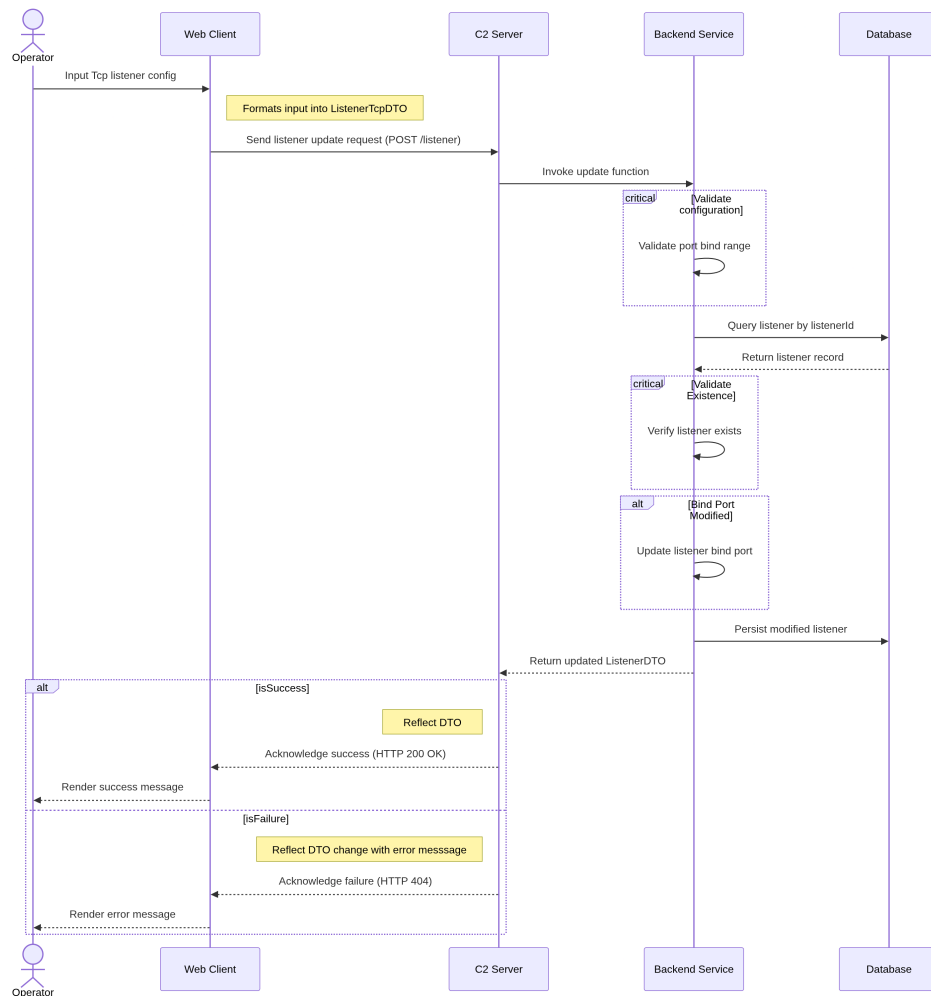


Figure 21: Team server TCP listener updation sequence diagram. View diagram code on mermaid.live.

8.13.1. Team Server TCP Listener Update Explanation

The TCP listener update process starts when the operator changes the configuration of an existing TCP listener through the web client. The updated configuration is sent to the C2 Server, which passes the request to the backend service for processing.

The backend service checks the database to find the listener using its listener ID. It then verifies that the listener exists before applying the changes to its configuration.

If the listener is found, the backend service updates the stored listener configuration with the new information provided by the operator. Since the TCP listener is used as configuration for generating peer-to-peer Agent payloads, the update changes the configuration that will be used by the system rather than starting or restarting an active listener on the Team Server.

After the update has been processed, the backend service returns the updated listener information to the C2 Server. The C2 Server then sends the result back to the web client so that the operator can see whether the update was successful.

If the listener cannot be found or the update cannot be completed, an error is returned to the web client so that the operator is informed that the update was unsuccessful.

This workflow allows the operator to modify an existing TCP listener configuration while keeping the stored configuration up to date for use when generating peer-to-peer Agent payloads.

8.14. Team server TCP listener deletion workflow

The sequence diagram represents the deletion workflow of a TCP listener deletion request on the team server.

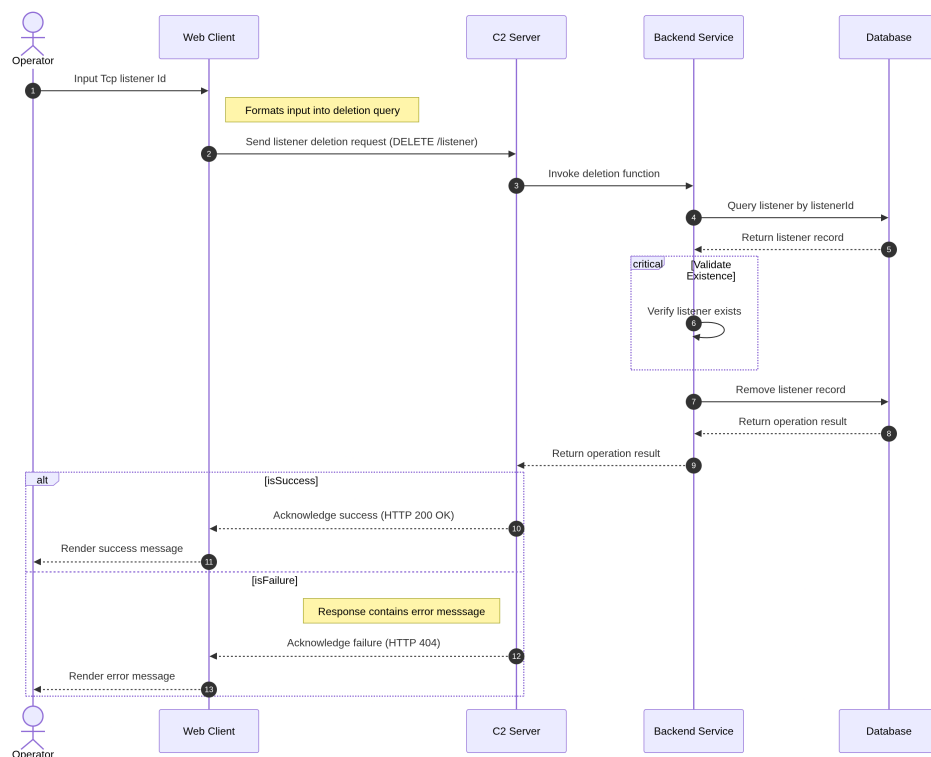


Figure 22: Team server TCP listener deletion sequence diagram. View diagram code on mermaid.live.

8.14.1. Team Server TCP Listener Deletion Explanation

The TCP listener deletion process starts when the operator selects an existing TCP listener through the web client and sends a deletion request to the C2 Server.

The C2 Server passes the request to the backend service, which checks the database using the listener information provided in the request. The backend service verifies whether the TCP listener exists before continuing with the deletion.

If the listener exists, the backend service removes the listener configuration from the database. Since TCP listeners are used as configuration for generating peer-to-peer Agent payloads and are not active listeners on the Team Server, there is no active network listener that needs to be stopped.

After the listener has been removed, the backend service returns the result to the C2 Server. The C2 Server then sends the response back to the web client, allowing the operator to see that the deletion was successful.

If the listener does not exist or the deletion cannot be completed, an error is returned to the web client so that the operator is informed that the deletion was unsuccessful.

The purpose show in this workflow is to allow the operator to remove an existing TCP listener configuration from the system when it no longer required.

8.15. Payload generation workflow

The sequence diagram represents the asynchronous compilation and registration lifecycle of a payload on the team server.

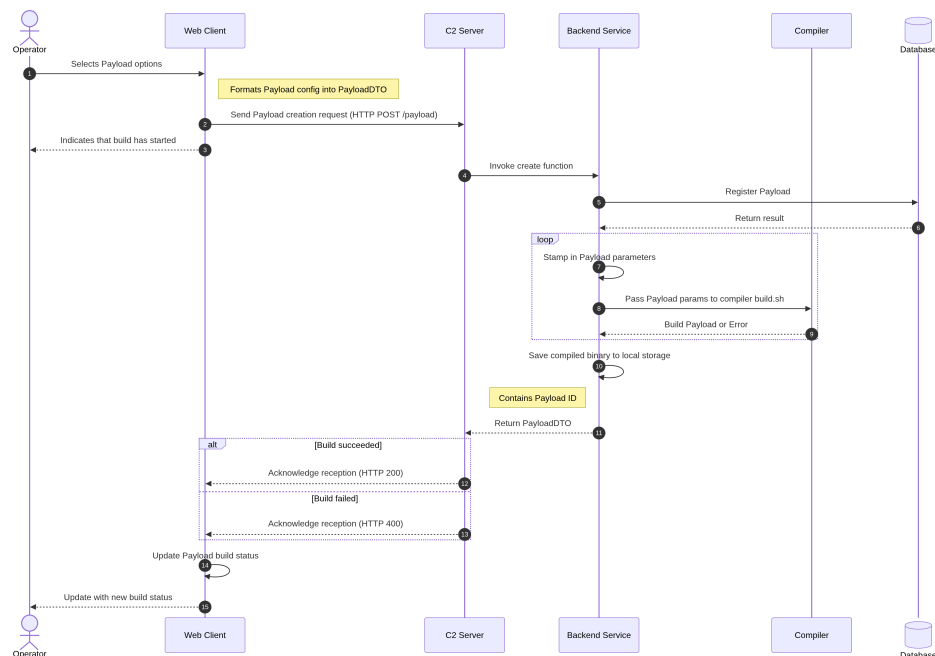


Figure 23: Payload generation sequence diagram. View diagram code on mermaid.live.

The state diagram below represents the lifecycle of a payload generation request:

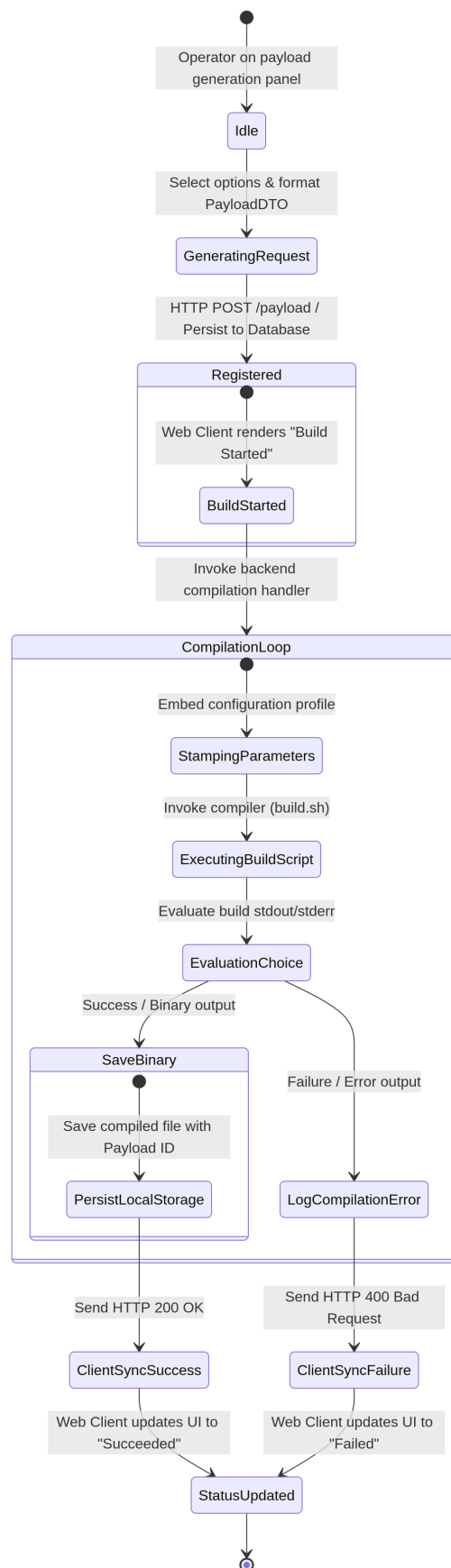


Figure 24: Payload generation state diagram. View diagram code on mermaid.live.

8.16. Deployment Plan

Docker/podman containers will be used for deployment, which may be deployed to any cloud platform or machine that can run docker/podman.

Containers will be created for the following:

- **Database**
 - Runs **PostgreSQL** [8]
- **Storage**
 - Runs **MinIO** [9]
- **Team Server API**
 - **Depends on:**
 - **Database** to *store & track database entries*
 - **Storage** to *store exfiltrated target data*
 - **Runs:** The backend ASPNET Core **Web API** managing agents, task queues, operators, compiles agent payloads, etc.
- **Client Frontend Web App**
 - **Depends on:** Team Server API
 - **Runs:** The ASPNET Core **Blazor** frontend **Web App** for operators to access Team Server functionality

9. Data Schema

9.1. Overview

The proposed C2 framework utilizes a hybrid storage architecture to separate structured operational data from binary files. **PostgreSQL** is used as the relational database management system for persistent application data, while files are stored on the local file system.

Operational data stored in PostgreSQL includes operators, deployed agents, callbacks, tasks, task results, file metadata, and audit records. Binary artifacts stored on the local filesystem include generated payloads, screenshots, downloaded documents, archives, and recordings.

9.2. Technologies

9.2.1. Relational (PostgreSQL)

Selected due to maturity, ACID compliance, and referential integrity for closely interconnected entities (Operators, Tasks, Agents, Callbacks). Key capabilities used:

- Primary and foreign key constraints
- Efficient indexing
- Concurrent access management
- Native JSONB support

9.2.2. Flexible Storage (JSONB)

Used for variable command execution outputs (e.g., process listings, directory trees, system info, registry keys) without requiring frequent schema migrations or redundant tables [10].

9.2.2.1. Alternatives

While other relational database systems such as MySQL and SQLite were considered we also have the need to manage unstructured data, this brought up the use of document-oriented database systems such as MongoDB which was also considered for its capability of being able to handle unstructured data [11]. Ultimately PostgreSQL was chosen because of the ability to handle fixed structured data and the use of JSONB allowed for the accommodation of data that does not have a fixed structure, avoiding the need of separate or multiple database technologies solely for flexible task output.

9.2.3. File Storage

Large binary files are stored separately from the relational database on the local filesystem for this current iteration. Examples of stored files would include generated payloads, screenshots, downloaded documents, archived files, recordings and other operational artifacts. Rather than storing binary content directly within PostgreSQL, the database stores metadata describing each file, including:

- Unique file identifier.
- Original filename.
- Storage path
- File type.
- File size.
- Cryptographic hash.
- Upload timestamp.
- Associated operational records.

Storing only meta data within PostgreSQL minimises database growth and improves backup and query performance while allowing binary files to be managed independently. Alternative approaches, including storing binary data directly within PostgreSQL using binary large objects (BLOBs) or integrating an object storage platform, were considered. Local filesystem storage was selected for the current implementation as it simplifies deployment, reduces infrastructure requirements, and satisfies project operational requirements until a final solution is decided upon (current suggestions would be either options like MinIO, Storage rental services like Azure blob storage or Firebase Storage). The separation of file metadata from binary content is to allow the storage subsystem to be adapted in the future should organizational requirements change, without requiring modifications to the relational database structure.

- ### 9.3. Proposed Entity Relationship Diagram (ERD)

- USER - CAMPAIGN: A user can create multiple campaigns, while each campaign records the user who created it.
- USER - PAYLOAD: A user can create multiple payloads, with each payload recording its creator.
- USER - AUDIT_LOG: A user can generate multiple audit log entries recording actions performed within the system.

9.3.2. Campaign and Operators

- CAMPAIGN - CAMPAIGN_MEMBER: A campaign can contain multiple campaign members, representing operators assigned to the campaign.
- OPERATOR - CAMPAIGN_MEMBER: An operator can be assigned to multiple campaigns through the CAMPAIGN_MEMBER junction table.

9.3.3. Listeners

- LISTENER - HTTP_LISTENER: A listener may have one HTTP-specific configuration containing HTTP listener settings.
- LISTENER - TCP_LISTENER: A listener may have one TCP-specific configuration containing TCP listener settings.
- HTTP_LISTENER - HTTP_LISTENER_HOST: An HTTP listener can have multiple associated hosts to support host configuration and rotation.

9.3.4. Host Information

- HOST - HOST_NETWORK_INTERFACE: A host can have multiple network interfaces, allowing the system to record its network configuration.
- HOST - HOST_USER_ACCOUNT: A host can contain multiple user accounts discovered on the system.

9.3.5. Campaign, Payloads and Agents

- CAMPAIGN - AGENT: A campaign can contain multiple agents, allowing agents to be associated with a specific campaign.
- LISTENER - AGENT: A listener can communicate with multiple agents that use it for communication.
- HOST - AGENT: A host can have multiple agents associated with it, representing agents running on that host.
- PAYLOAD - AGENT: A payload can be associated with multiple agents created from that payload.
- LISTENER - PAYLOAD: A listener can be used by multiple payloads for their communication configuration.
- AGENT_PROFILE - PAYLOAD: An agent profile can configure multiple payloads, allowing common agent settings to be reused.

9.3.6. Agent Information

- AGENT - PROCESS: An agent can discover multiple processes running on its associated host.
- AGENT - CREDENTIAL: An agent can collect multiple credentials during system operation.

9.3.7. Tasks and Results

- AGENT - TASK: An agent can receive multiple tasks during its operation.
- OPERATOR - TASK: An operator can create multiple tasks, with each task recording which operator issued it.
- TASK - TASK_RESULT: A task can produce multiple results, allowing responses to be stored separately from the task itself.

9.3.8. File Transfers

- AGENT - FILE_TRANSFER: An agent can perform multiple file transfers, either sending or receiving files.
 - TASK - FILE_TRANSFER: A task can initiate multiple file transfers, linking file activity to the task that caused it.
-

9.4. Checkin JSON schema

The below section outlines the request and response schema for various Checkin stages, namely: initial, and fully-established. IA request within the context of a Checkin either initial or fully-established is performed by the agent. Inversely a response is provided in relation to an agent request by the team server.

The static AES256 encryption template for initial Checkin requests and responses is in the following format:

Base64(PayloadUUID + AES256(JSON))

It should be noted that the AES256 encryption algorithm is symmetric, therefore the key does not change throughout the life cycle of an agent. Hence the use of static in the naming convention.

Alternatively for fully-established requests and responses the format differs by utilising a CallbackUUID:

Base64(CallbackUUID + AES256(JSON))

9.4.1. Initial Checkin message request

The initial Checkin request performed by the agent contains the PayloadUUID (stamped within the binary upon payload generation) which acts as an identifier and host metadata for fingerprinting purposes. Once the Checkin status of the agent changes to fully-established a CallbackUUID is used for subsequent Checkin requests.

9.4.1.1. Request

```
{
  "action": "checkin",
  "uuid": "payload uuid",

  "ips": ["127.0.0.1"],
  "external_ip": "8.8.8.8",
  "os": "macOS 10.15",
  "host": "spooky.local",
  "process_name": "osascript",
  "pid": 4444,
  "architecture": "x64",
  "motherboard": "B450 Aorus Elite V2",
  "ram": 16,
  "disk_size": 1000,
  "free_disk": 540,
  "cpu_count": 4,
  "mac": "00:1A:2B:3C:4D:5E",
  "domain": "test",
  "integrity_level": 3,
  "user": "admin",
  "users": ["admin", "george"]
}
```

9.4.1.2. Response

```
{
  "action": "checkin",
  "id": "callbackUUID",
  "status": "success"
}
```

9.4.2. Fully-established Checkin message request

Once the Checkin status has changed to full-established, there is no longer any host metadata packaged with the request. Instead a tasking size (stamped within the binary upon payload generation as part of its configuration) is specified which determines the number of tasks the team server will select from the task queue.

9.4.2.1. Request

```
{
  "action": "checkin",
```

```
    "tasking_size": 1
}
```

9.4.2.2. Response

Below is the response from the team server containing the task queued for execution on the agent's host machine:

```
{
  "action": "checkin",
  "tasks": [
    {
      "task_id": "uuid of task",
      "command": "string",
      "timestamp": 1578706611.324671
    }
  ]
}
```

In the eventuality of the team server's task queue for the specified agent not having any tasks, the response returns an empty list. This can be seen below:

```
{
  "action": "checkin",
  "tasks": []
}
```

9.4.3. Fully-established Checkin message with command execution output

The output from executed tasks is bundled within Checkin requests once execution has either succeed or failed. Otherwise the output member is omitted from the schema, leaving only the `task_id` and `status` for purposes of providing continuous feedback to the operator regarding the state of task execution. Other tasks status values such as `queued`, and `running` are considered neither failure or success states.

9.4.3.1. Request (Success state)

A success state occurs when a task has successfully been executed on the agent's host machine and the output of the operation can be parsed either in full or chunks (in the context of long running tasks).

```
{
  "action": "checkin",
  "tasking_size": 1,
  "responses": [
    {
      "task_id": "uuid of task",
      "status": "success",
      "output": "task output"
    }
  ]
}
```

```

    }
  ]
}

```

9.4.3.2. Request (Failure state)

Conversely a failure state occurs when a task has produced an error in its course of execution. The output of the operation is parsed and the output values populated regardless of the failure because it enables the further contextualisation of the operation.

```

{
  "action": "checkin",
  "tasking_size": 1,
  "responses": [
    {
      "task_id": "uuid of task",
      "status": "failure",
      "output": "task output"
    }
  ]
}

```

9.4.3.3. Request (Non-failure/Non-success)

Lastly non-failure and non-success states are fundamentally employed to provide near real-time feedback on the statuses of tasks placed within the agent's in-memory queue. This enables improved logging and auditing of task states throughout the operator's engagement independent of task execution.

```

{
  "action": "checkin",
  "tasking_size": 1,
  "responses": [
    {
      "task_id": "uuid of task",
      "status": "queued"
    }
  ]
}

```

9.5. Command JSON schema

There is a high variance in the schema of various agent commands and their respective outputs. Therefore JSONB will be used for the storage within the Task table to accommodate the unstructured nature of the schema. The agent utilises the `command_type` to determine which method should be called for execution.

9.5.1. Help command

List help for all commands available on the consoleCommand route (including possible server side aliases).

```
[
  {
    "name": "string",
    "description": "string",
    "details": "string",
    "groups": ["string"]
  }
]
```

9.5.2. Spawn PowerShell

Execute the command using PowerShell.exe.

```
{
  "task_id": "string",
  "command_type": "powershell",
  "timestamp": 1578706611.324671,
  "commandlet": "string",
  "arguments": "string"
}
```

9.5.3. Spawn Shell

Execute the command using cmd.exe.

```
{
  "task_id": "string",
  "command_type": "shell",
  "timestamp": 1578706611.324671,
  "command": "string"
}
```

9.5.4. Download

Download a file to the team server.

```
{
  "path": "string"
}
```

9.5.5. Cancel Download

Cancels a download that is currently in progress.

```
{
  "id": "string"
}
```

9.5.6. Get Downloads

Lists all downloaded files associated with specified agent from the team server.

```
[
  {
    "agent_id": "string",
    "path": "string"
  }
]
```

9.5.7. Get Active Downloads

Lists file downloads currently in progress for a given agent.

```
[
  {
    "id": "string",
    "agent_id": "string",
    "name": "string",
    "path": "string",
    "size": 0,
    "received": 0
  }
]
```

9.5.8. Upload

Upload a file to a host via the agent. The file bytes are base64 encoded.

```
{
  "task_id": "string",
  "command_type": "upload",
  "timestamp": 1578706611.324671,
  "name": "string",
  "path": "string",
  "file_bytes": "string"
}
```

9.5.9. Kill Process

Instructs the agent to terminate the specified process via Process Identifier (PID).

```
{
  "task_id": "string",
  "command_type": "kill",
  "timestamp": 1578706611.324671,
}
```

```
  "pid": 0
}
```

9.5.10. RunAs

Attempt to execute a program as another user.

```
{
  "task_id": "string",
  "command_type": "runAs",
  "timestamp": 1578706611.324671,
  "domain": "string",
  "user": "string",
  "password": "string",
  "command": "string",
  "arguments": "string"
}
```

9.5.11. Run

Execute the program. Returns output.

```
{
  "task_id": "string",
  "command_type": "run",
  "timestamp": 1578706611.324671,
  "program": "string",
  "arguments": "string"
}
```

9.5.12. RunU

Attempt to execute a program with the specified PID as its parent.

```
{
  "task_id": "string",
  "command_type": "runU",
  "timestamp": 1578706611.324671,
  "pid": 0,
  "command": "string",
  "arguments": "string"
}
```

9.5.13. Escalate

Attempts a User Access Control (UAC) Bypass with the specified registry hive key and binary.

```
{
  "task_id": "string",
  "command_type": "escalate",

```

```
"timestamp": 1578706611.324671,  
"reg_key": "string",  
"target_binary": "string"  
}
```

10. Architecture Artifacts

10.1. Design patterns

The use of the below design patterns will be described and explained within the context of the proposed system with reference to its interaction level.

10.1.1. Observer

The proposed system needs to support concurrent operation of multiple campaigns and the simultaneous management of agent task state consistency across connected clients in near real-time to ensure operators are provided accurate information pertaining to the results of their actions. The common challenge with partitioning the system into cooperating components, namely a team server and client/s, is the inability to ensure that consistency is maintained across these related components of the system. Additionally, no assumption can be made as to how many clients will need to be notified of task state changes adding to the complexity. However, the one certainty in the system is the inherent one-to-many dependency between the team server and that of the clients.

The Observer design pattern has been employed to ensure the fulfilment of this requirement so that when one object changes state (e.g. a task) all its dependants (clients interested in the state change) are notified and updated automatically [12]. The pattern operates at the interaction level, as the clients and team server communicate at runtime through the invocation of methods. This is achieved by the concrete NotificationHub class, it maintains a reference to all connected clients and implements the INotification interface that exposes a method for pushing task state changes upon modification in the system via a Signal R connection. It is the team server's responsibility to trigger notifications of state changes. The NotificationHub will be registered as a singleton, ensuring that every client will have access to the same instantiation and task state consistency throughout the system [12].

Clients (e.g. Observers) are attached upon establishment of a connection to the NotificationHub and detached when the same connection is closed or timed out. This decouples the clients from the team server, as they are able to be independently subscribed and unsubscribed from the service without conflict [12].

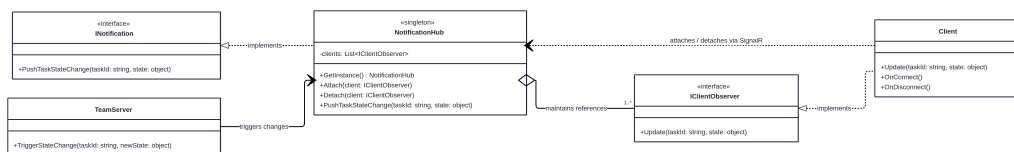
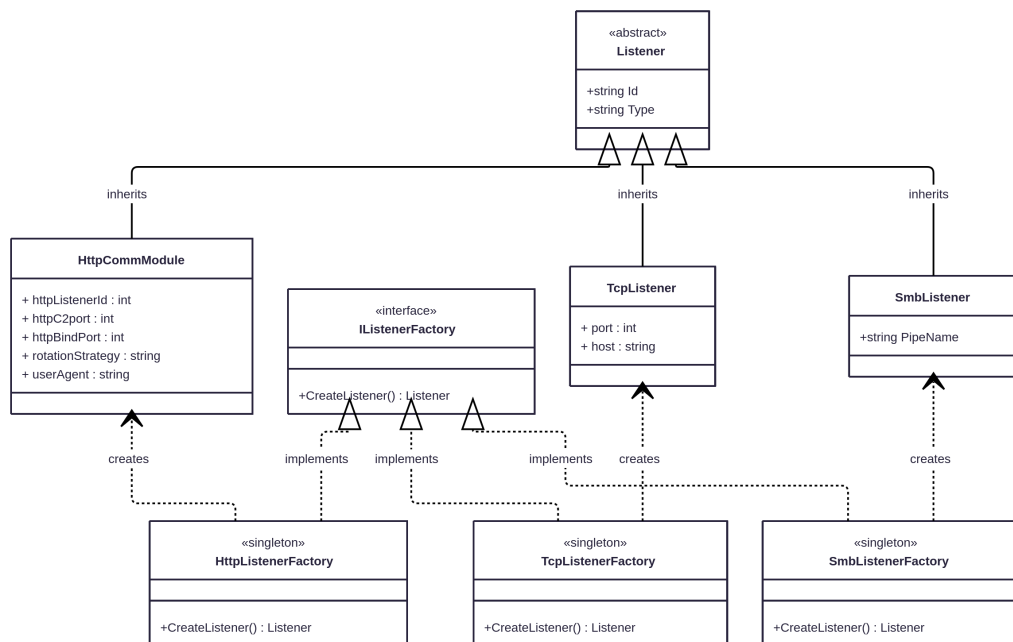


Figure 26: Observer design pattern diagram. View diagram code on mermaid.live.

10.1.2. Abstract Factory

The team server requires the creation of listener objects, these objects share a common conceptual goal within the context of the system. However, the listener configurations, behaviour and implementation logic for creation differs significantly across listener type (e.g. TCP, HTTP, and SMB). The team server requires the listener creation logic to be reusable. The conventional Factory pattern was first considered, however it relies on listener type conditional checks to determine which listener to produce. This introduces complex creation logic and unnecessarily exposes internal implementation details to the callee, as it would have to be aware of the listener type and pass this to the creation method [13].

The Abstract Factory design pattern was selected instead as the callee will only have to rely on the `IListenerFactory` interface defined for object creation. The pattern operates at the object level, as its primary responsibility is creational in nature and does not manage how those listener object instances communicate or behave after creation [14]. Each listener type will have its own factory (e.g. `HttpListenerFactory`, `TcpListenerFactory`, and `SmbListenerFactory`) that implements this interface and defines its unique creation logic for its respective listener type. This ensures that the factory encapsulates the responsibility and the process of object creation, decoupling it from the implementation classes of the various listener types. Furthermore, it improves the maintainability, as it enables internal implementation logic changes to object creation without revision to the callee. Lastly the various listener factories will be registered as singletons enabling reuse via dependency injection making it a suitable pattern for the fulfilment of the requirement [14].

Figure 27: Abstract Factory design pattern diagram. View diagram code on mermaid.live.

10.1.3. Strategy

The client needs to be able to accept a command input from the operator and format it into the appropriate command data transfer object (DTO) for communication with the team server API endpoint. The high variance in command arguments leaves little point in attempting to produce a generalised structure that is suitable for all command types. Since generalisation is not possible, the interpretation of each command needs to be specific to its structure. Therefore, the interpretation will have to employ different algorithms depending on the command type. However, hard-coding all such algorithms into all client components that require interpretation of commands is not ideal, as it will increase its complexity as it grows in the number of algorithms it supports. Furthermore, these algorithms include implementation details that client components such as the console should not know about, as they should only be responsible for data presentation and command input [15].

The Strategy design pattern was selected because it enables the a family of command interpretation algorithms to be defined, encapsulates the implementation logic of each and renders them interchangeable based on the command type [15]. The pattern operates at the interaction level, as it determines the behaviour of command processing. The command input is first parsed for a command type keyword, based on this keyword the appropriate strategy is selected by the `CommandResolver` class and used to interpret its arguments and map them to the relevant DTO.

The `IStrategy` interface defines a common `Interpret` method for all concrete strategy classes to implement. The internal implementation of the `Interpret` method is specific to the command type, mapping the arguments to its respective DTO. This enables the algorithm to vary independently from the client component that uses it, without knowledge of any internal processing [15].

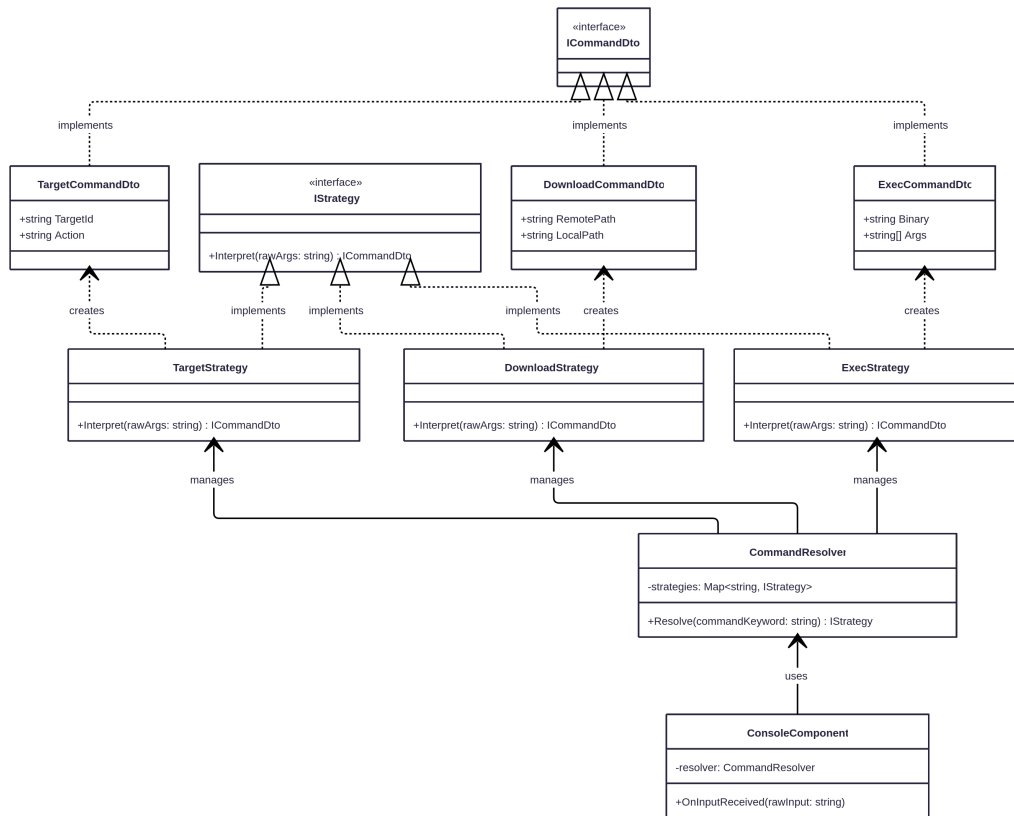


Figure 28: Strategy design pattern diagram. View diagram code on mermaid.live.

10.2. Architecture patterns

The architectural patterns will be described and explained within the context of the proposed system.

10.2.1. Peer-to-peer model

The TCP and SMB agents leverage a peer-to-peer (P2P) network architecture model for communication. Every P2P agent acts as both a client and a server [16].

This model is particularly useful for compromising the Air-Gapped Network Segment where direct external traffic requests would be flagged as anomalous. Instead of forcing every machine to talk to the internet, only an egress agent

communicates externally with the team server using standard HTTP/HTTPS protocols [17].

When the attacker issues a command, the egress receives the tasking and relays it down the chain through the Windows Named Pipes and TCP Socket Connections to the first-tier peers. These peers then forward the tasks further down the line to the grandchild peers. Once the tasks are executed, the grandchild agents forward the task results back up along the exact same chain, passing them through the intermediate peers until they reach the egress agent, which exfiltrates the data to the team server [17].

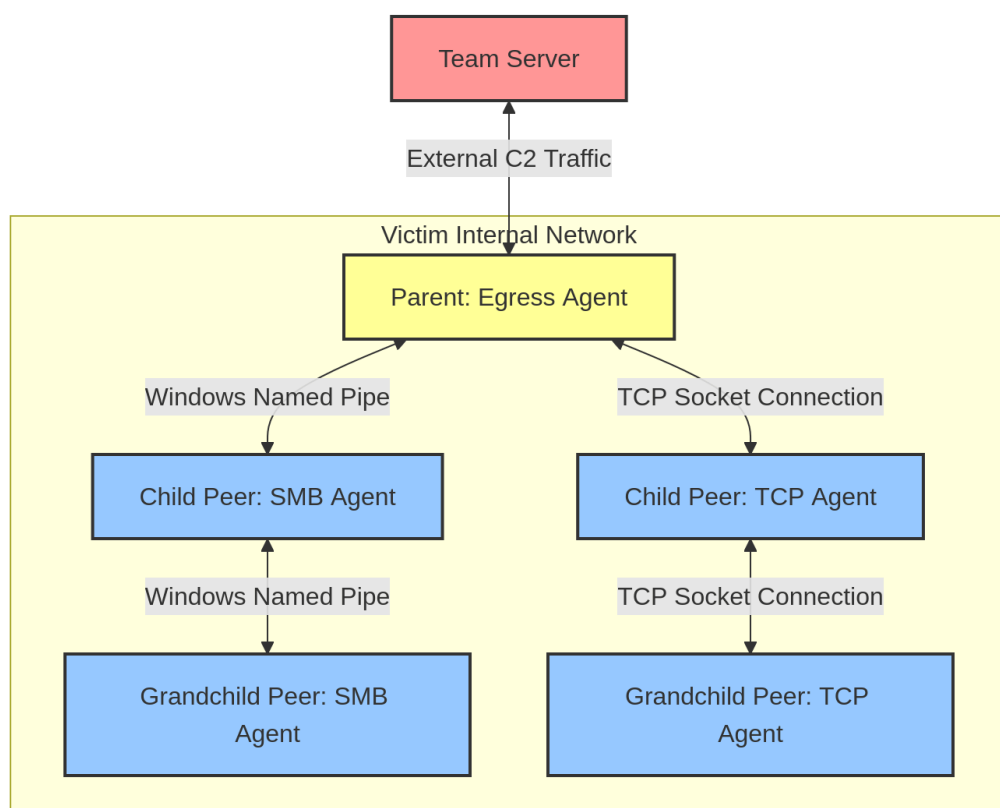


Figure 29: Peer-to-peer network architecture diagram. View diagram code on mermaid.live.

10.2.2. Client-Server model

The client-server network architecture model is implemented in the communication between HTTP/S agents (otherwise known as egress agents) and the team server. In this model, the external team server acts exclusively as the server, while the egress agent acts as the client [18].

The egress agent resides inside the victim's internal network, it must initiate all communication across the perimeter boundary to bypass inbound firewall restrictions. It acts as the client by reaching out externally to the team server using standard HTTP or HTTPS protocols [19].

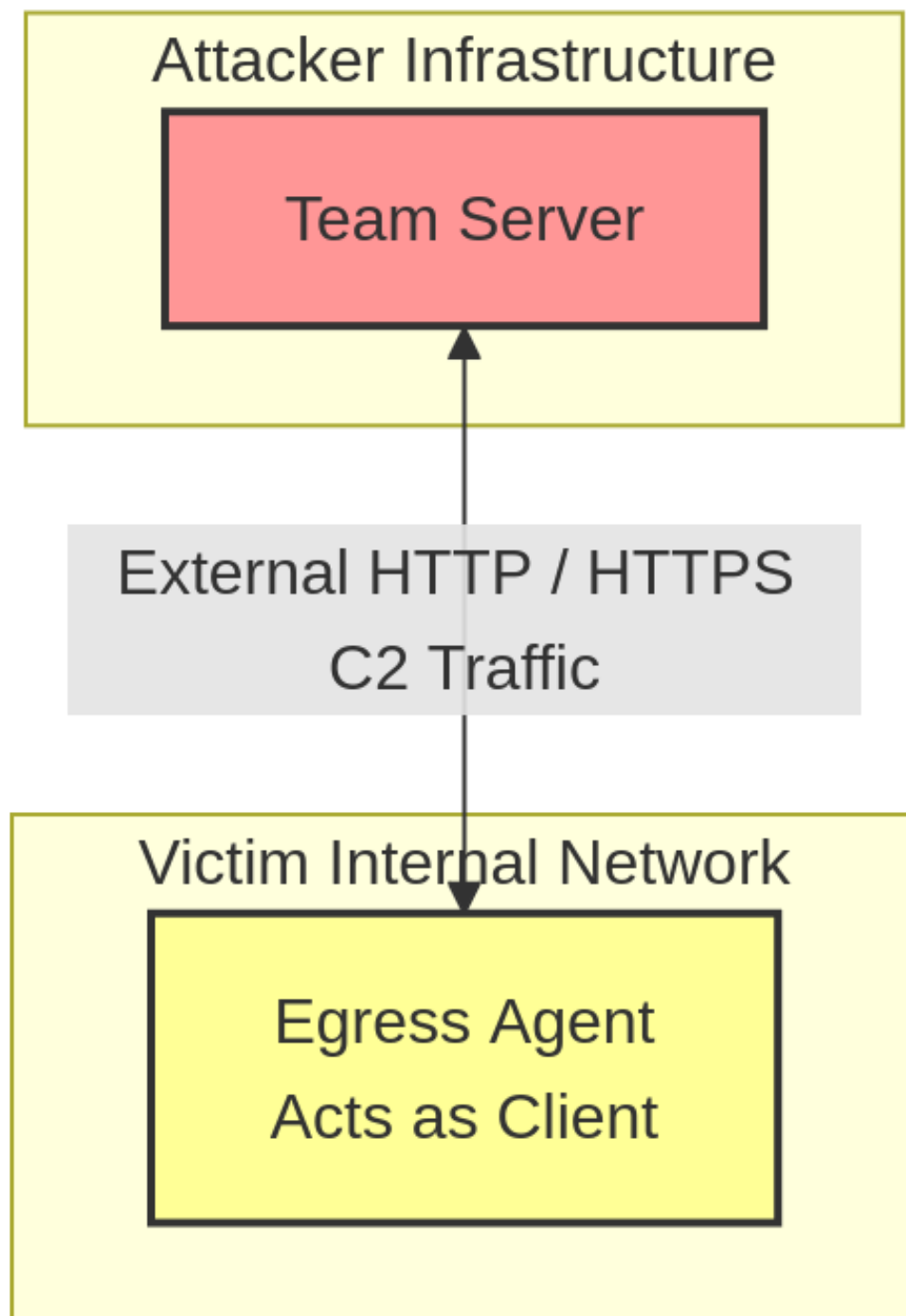


Figure 30: Egress agent client-server network architecture diagram . View diagram code on mermaid.live.

The same client-server model is applied to the client application that the operator uses to issue tasking for campaigns, as depicted below:

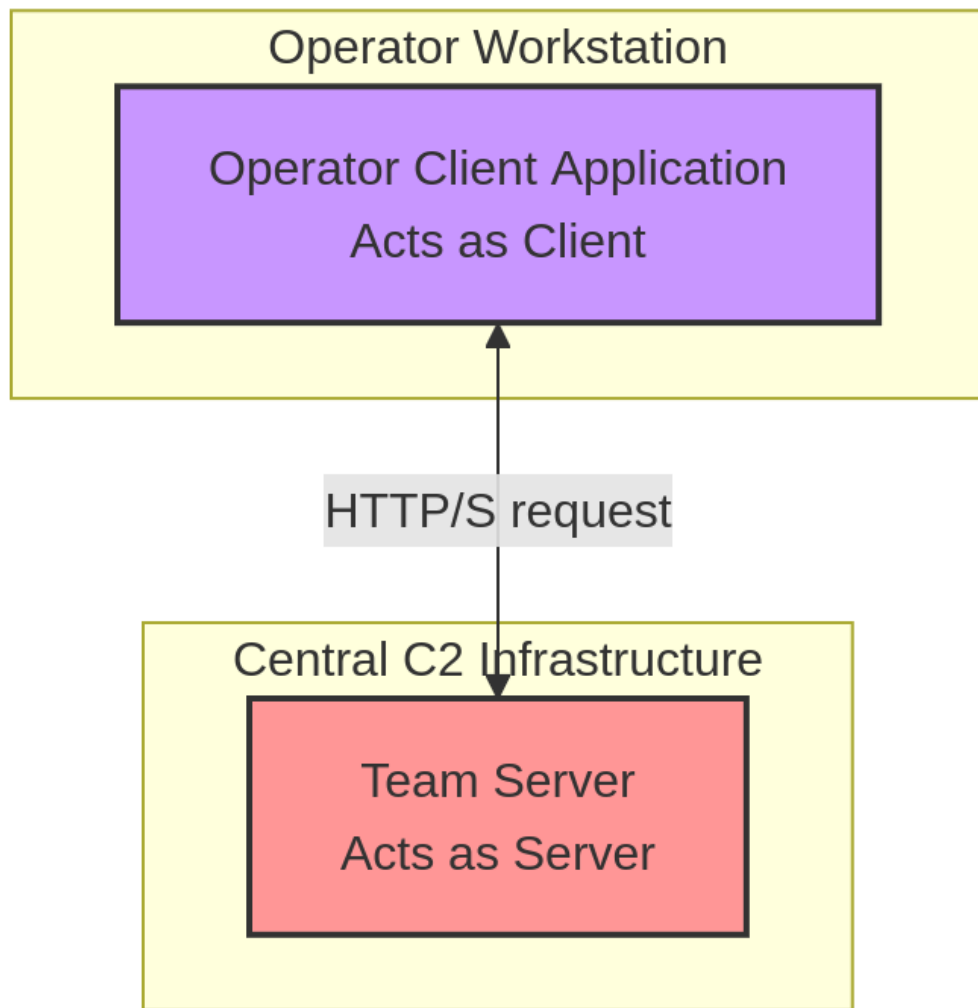


Figure 31: Client and team server network architecture diagram . View diagram code on mermaid.live.

11. Security

Security within the Trinity C2 Framework prioritises the protection of operational data, stealth-focused agent-to-server encrypted communication channels, and thorough access control policies enforced throughout the system. The architecture implements defence-in-depth principles across the Agent Payload, Team Server API, and Blazor Web Client to maintain system integrity and operational security across all cooperating components.

11.1. Component Security Architecture

11.1.1. Agent Payload Security

Agent implants operate within target environments and implement defensive and evasive mechanics in addition to core post-exploitation functionality (including exfiltration, persistence, privilege escalation, and command execution).

11.1.1.1. Security Considerations & Evasion

- **API Hashing & Dynamic Invocation:** Win32 API functions are dynamically resolved using hash algorithms at runtime to bypass static analysis and prevent Import Address Table (IAT) inspection. IAT obfuscation ensures that static binary signatures do not flag sensitive native system calls.
- **Direct & Indirect System Calls:** The agent employs direct and indirect system calls (`syscall/sysenter`) to bypass user-space API monitoring and hooks, transitioning execution into kernel mode directly to circumvent user-mode detection within `ntdll.dll`.
- **Configuration & Payload Obfuscation:** Agent payload configuration data is embedded within AES-256 encrypted blobs, which are decrypted in memory only during runtime to minimise static and dynamic detection artefacts.
- **Privilege Escalation & Persistence:** Foothold persistence is maintained using Windows task scheduling, while elevated execution privileges are obtained through custom User Access Control (UAC) bypass implementations [20].

11.1.1.2. Network Security

- **Certificate & Fingerprint Pinning:** Agent payloads are compiled with hard-coded SSL/TLS public key fingerprints based on generation configuration to prevent Man-in-the-Middle (MITM) vulnerabilities and inspection [21].
- **Traffic Masking & Beacon Jitter:** The agent sleeps for a base delay with a configured random jitter range added to prevent consistent network traffic intervals and defeat heuristic analysis.
- **Session Key Exchange:** The agent's `payloadUUID` is only used for initial check-in, with an isolated `callbackUUID` being issued and used for subsequent check-ins as per the standard established in Mythic's C2 framework [22], [23].

11.1.1.3. Vulnerabilities & Potential Threats

Potential vulnerabilities and threats include: reverse engineering of the agent payload dynamically via process memory dumps, statically reversing the behaviour of the compiled binary, file signature detection, dynamic behavioural heuristics flagged by EDR solutions [24], and agents becoming unavailable due to host endpoint failure.

11.1.2. Team Server API Security

The Team Server API operates as the core of the framework, handling agent task and command queues, supporting multi-operator collaborative campaigns (shared management of agents between multiple operators), and managing exfiltrated target data.

11.1.2.1. Security Considerations & Injection Protection

- **SQL Injection Prevention:** Database access via PostgreSQL relies strictly on Entity Framework Core Object-Relational Mapping (ORM) and fully parameterised SQL queries. The use of concatenated raw SQL queries is prohibited across all service handlers.
- **JSONB Parameterisation:** Task parameters and flexible command outputs stored in PostgreSQL JSONB structures [10] are validated and parameterised prior to query evaluation, preventing dynamic query manipulation or JSON structure injection.
- **Command Injection Prevention:** Operating system tasks or secondary tools executed by background services utilise explicit, isolated argument arrays rather than passing raw string blocks to process shells (`cmd.exe` or `powershell.exe`).
- **Cross-Site Scripting (XSS) Prevention:** Incoming data strings from deployed agents (e.g. hostnames, command execution output, system user lists) are treated as untrusted input [25]. The server sanitises input payloads prior to persistence, ensuring malicious HTML or script snippets cannot execute when retrieved by operators.

11.1.2.2. Network Security

- **Encrypted Ingress Endpoints:** External listener endpoints terminate incoming traffic over TLS 1.3 [26] or Secure WebSockets (`wss://`) [27]. Unencrypted HTTP or TCP channels are restricted strictly to isolated local development.
- **Network Segmentation:** Architectural design isolates the Team Server REST API and agent listeners from backing persistent infrastructure (PostgreSQL database and the storage subsystem) using strict firewall rules [28].
- **Tor Hidden Services:** Hidden Service listeners route agent beaconing through onion routing pathways, ensuring infrastructure anonymity and obfuscating backend server IP addresses [29], [30].

11.1.2.3. Vulnerabilities & Potential Threats

Potential vulnerabilities include: unauthorised access to active campaigns, Denial of Service (DoS) via agent beacon flooding, server-side log tampering, and credential exploitation.

11.1.3. Client Application Security (Blazor Web Interface)

The Web Client provides the graphical interface for authenticated operators to monitor campaign progress, queue operational tasks, and manage target assets.

11.1.3.1. Security Considerations & XSS Mitigation

- **HTML Auto-Escaping & Render Security:** The Blazor UI framework automatically escapes output rendered in Razor components. Dynamic command outputs, system logs, and shell responses are encoded by default to eliminate stored or reflected Cross-Site Scripting (XSS) attacks.
- **Strict HTML Sanitisation:** Where rich-text rendering or formatted terminal outputs are processed, inputs pass through a strict sanitisation pipeline enforcing explicit allow-lists for allowed tags and attributes [25].
- **Content Security Policy (CSP):** The client enforces HTTP CSP response headers, restricting script execution sources (`script-src 'self'`), prohibiting inline scripts, and disabling object embedding (`object-src 'none'`).
- **Interpreter-Based Command Parsing:** User inputs passed through the command console are validated client-side against predefined Command Data Transfer Objects (DTOs) via the Strategy pattern prior to API submission [23].

11.1.3.2. Network Security

- **Secure Session Tokens:** Operator authentication tokens (JWTs) are stored securely in memory or within `HttpOnly, SameSite=Strict` secure cookies to mitigate script-based token theft [31], [32].
- **Real-time Communication Protection:** Notification hubs and task status updates communicate over encrypted WebSockets (`wss://`) protected by Bearer token authorisation [27], [33].

11.1.3.3. Vulnerabilities & Potential Threats

Potential vulnerabilities include: session hijacking, unauthorised task issuing via client-side state manipulation, UI clickjacking, and Cross-Site Request Forgery (CSRF).

11.2. Authentication & Session Management

Operator access requires strict identity verification before access to operational resources is granted [34].

- **Credential Validation:** Authentication validates operator credentials using secure password verification pipelines [34].

- **JWT Token Architecture:** Upon successful authentication, the API issues a signed JSON Web Token (JWT) containing cryptographic claims, including user identity, role assignments, and campaign scoping [31], [32].
- **Session Lifecycle & Revocation:** Session states are tracked via the `REFRESH_TOKEN` entity in PostgreSQL [10]. Tokens enforce finite expiration windows, and explicit revocation endpoints allow administrators to terminate compromised or inactive sessions immediately.
- **Multi-Factor Authentication (MFA):** Framework architecture supports Time-Based One-Time Password (TOTP) algorithms (RFC 6238) as a planned security control for operator authentication [35].

11.3. Password Security & Credential Storage

Password processing follows standard cryptographic storage practices to protect credentials against disclosure [36]:

- **PBKDF2 Key Derivation:** User passwords are encrypted using Password-Based Key Derivation Function 2 (PBKDF2), implemented via the server's `AuthenticationUtils` helper methods (`+GeneratePBKDF2`, `+GeneratePBKDF2WithSalt`) to provide resistance against brute-force and dictionary attacks [36].
- **Per-User Salt Generation:** Every user account utilises a unique, cryptographically secure salt generated prior to hashing to prevent rainbow table attacks [36].
- **Zero Cleartext Storage:** Cleartext credentials are never written to log files, persistent databases, or volatile application buffers [34], [36].

11.4. Authorisation & Access Control

The framework implements Role-Based Access Control (RBAC) to enforce the Principle of Least Privilege [37]:

- **Role-Based Claims Enforcement:** Access to system routes and API endpoints is constrained by signed JWT role claims (`System Administrator` vs `Operator`) [32], [37].
- **Resource-Level Authorisation:** Campaign isolation ensures operators can only view or interact with hosts, tasks, and files belonging to active campaigns to which they are explicitly assigned via `CAMPAIGN_MEMBER` records [38].
- **Administrative Separation:** Administrative capabilities (e.g. user provisioning, system configuration, database backup execution) are strictly segregated from standard operational targeting functionality [37].

11.5. API Security & Input Validation

The REST API layer implements controls to resist unauthorised requests and input manipulation:

- **Endpoint Authorisation:** Protected controllers require valid Bearer <token> HTTP authorisation headers for request processing [32], [33].
- **UUID Resource Identifiers:** All entities (agents, tasks, payloads, campaign items) use Universally Unique Identifiers (UUIDv4) instead of sequential integers, preventing object enumeration and Insecure Direct Object Reference (IDOR) vulnerabilities [23].
- **Input Validation & Sanitisation:** Incoming DTO payloads undergo schema validation prior to controller execution, enforcing bounds, parameter formatting, and strict type constraints [25].
- **Rate Limiting:** API endpoints enforce request rate limits to prevent brute-force authentication attacks and resource exhaustion, returning HTTP status code 429 (Too Many Requests) when thresholds are exceeded [39], [40].

11.6. Storage Security

Data stored within persistent storage components is safeguarded throughout its lifecycle:

- **Hybrid Storage Architecture:** Operational metadata (including file hashes, sizes, paths, and associations) is managed within PostgreSQL, while large binary files (such as generated payloads, screenshots, and downloaded documents) are stored on the local filesystem to optimise query performance and minimise database growth [41].
- **Filesystem Access Control:** Binary files residing on the local filesystem are protected via strict operating system file permissions, ensuring access is restricted exclusively to the backend service account [41]. Dedicated object storage solutions (such as MinIO or cloud blob storage) remain an architectural alternative for future deployment iterations.
- **Encryption at Rest:** Sensitive database records, cryptographic keys, and stored agent binary payloads are encrypted at rest using AES-256 storage encryption standards [41].

11.7. Network Security & Evasion

Network controls secure operational control channels while allowing configurable evasion parameters:

- **TLS Transport Encryption:** Transport Layer Security (TLS 1.3) encrypts network communications between operators, listeners, and C2 services [26], [42].

- **Secure WebSockets:** Real-time data streams utilise `wss://` protocol endpoints to maintain full-duplex session encryption [27].
- **Anonymised Routing:** Tor Network integration routes C2 beaconing over SOCKS5 onion channels, obfuscating C2 server IP addresses [29], [30].
- **Network Segmentation:** Firewalls and network rules isolate operational listener infrastructure from management interfaces and database nodes [28].

11.8. Application of Security Principles (CIA Triad)

The application of fundamental security principles across the Trinity framework is summarised below:

Principle	Technical Implementation in Trinity C2
Confidentiality	• End-to-end AES-256 payload encryption and dynamic session keys [41]. • TLS 1.3 (<code>https://</code>) and Secure WebSocket (<code>wss://</code>) transport protocols [26], [27]. • PBKDF2 password hashing and JWT authorisation claims [32]. • Local filesystem permission restrictions and encrypted data-at-rest storage for exfiltrated target evidence [41].
Integrity	• Cryptographic SHA-256 checksum validation for all uploaded and exfiltrated file transfers. • Signed JWT tokens preventing client-side permission forgery [32]. • Parameterised database queries preventing payload and script injection [25]. • Centralised immutable audit logs (<code>AUDIT_LOG</code>) tracking all operator activity [7].
Availability	• Asynchronous, non-blocking task processing and background worker queues. • Resource connection pooling and PostgreSQL index optimisation [10].

Principle	Technical Implementation in Trinity C2
	• Rate limiting (HTTP 429) to mitigate agent check-in flooding or DoS conditions [39].

11.9. Threat & Risk Mitigation Matrix

The following matrix documents identified security threats across system components alongside their corresponding risk ratings and mitigation strategies:

System Component	Identified Threat / Security Concern	Severity / Risk	Applied Mitigation Strategy
Agent	Memory dumping revealing cleartext C2 endpoints and AES keys [41].	High	Store configuration data as AES-256 encrypted blobs; decrypt strings in memory only during beacon execution [41].
Agent	EDR API hooking detecting process injection or command execution.	High	Implement direct and indirect system call wrappers (syscall/sysenter) to bypass user-mode API hooks.
Agent	Man-in-the-Middle (MITM) inspection breaking beacon channel privacy [21].	High	Embed cryptographic SSL/TLS public key certificate pinning directly into agent binaries [21].
Team Server	SQL Injection via task output processing or log query routes [25].	Critical	Enforce parameterised queries via Entity Framework Core; prohibit string concatenation in PostgreSQL/JSONB routines [10], [25].
Team Server	Unauthenticated administrative	Critical	Validate JWT signatures on pro-

System Component	Identified Threat / Security Concern	Severity / Risk	Applied Mitigation Strategy
	command execution or API abuse [32].		tected routes via middleware; enforce strict RBAC claim checks [32], [37].
Team Server	Denial of Service (DoS) via agent check-in flooding [39].	Medium	Implement endpoint rate limiting (HTTP status 429) and asynchronous queue processing [39], [40].
Client UI	Stored XSS via raw agent command output rendered in the UI [25].	High	Rely on Razor HTML auto-escaping; execute untrusted terminal strings through an HTML sanitisation pipeline [25].
Client UI	Session token theft via client-side script execution [31].	High	Store operator JWTs in volatile memory or secure HttpOnly, SameSite=Strict cookies [31], [32].

12. DevOps

12.1. DevOps Approach

The development process used by the Trinity Command and Control (C2) Framework is intended to support controlled changes to the system while maintaining the quality integrity and reliability of the project. The C2 Framework utilizes GitHub as its source control and collaborative development platform. The development workflow employs branch-based development, regular commits, pull requests, code review, and automated testing to provide a controlled process for developing and integrating changes.

The development process is intended to support the non-functional requirements specified for the Trinity framework namely maintainability, reliability, availability, compatibility and portability. Version control and structured branching help with collaboration and maintainability by isolating each person's changes from the stable codebase. Pull requests are a controlled place to review changes before merging them and automated builds and tests provide consistent validation of those changes. Continuous integration decreases the time between introducing a change and detecting a defect, so problems are found sooner rather than later across multiple changes [43].

The source code is hosted on GitHub and managed using collaborative development. GitHub Actions will be used to automate the build and testing processes. GitHub Actions workflows are defined in YAML files in the repository and can be triggered by events such as pushes and pull requests [44]. The intended development and integration process is shown in Fig. 32.

12.1.1. Branching Strategy

The Trinity project uses branch based development. Contributions are developed separately from the 'main' branch. The 'main' branch will represent the integrated and stable version of the project, and the individual branches are used for specific contributions such as features, documentation, diagrams or other changes. This enables multiple team members to work in parallel without directly changing the shared stable branch.

The project uses a consistent branch naming convention:

- verb/thing-type_thing-name

The 'verb' identifies the intended action, the 'thing-type' identifies the type of contribution, and the 'thing-name' provides a concise description of the work. Examples include:

- document/devops_pipeline
- diagram/data-schema_erd
- feature/agent-management

The naming convention provides a standard way to identify the purpose of a branch. This is useful in a collaborative project as it allows the purpose of a branch to be identified without having to look at the code or documentation contained within it.

The development process follows these steps:

1. The developer updates their local copy of the 'main' branch to start working from the current state of the project.
2. A new branch is created for the particular contribution.
3. Changes are made in the branch and are committed frequently.
4. The branch is pushed to the GitHub repository.
5. A pull request is created against the 'main' branch.
6. Automated build and test workflows are run against the proposed changes.
7. Another team member reviews the contribution.
8. Once the contribution has been validated and reviewed as necessary, merge it into the "main" branch.

This separates development work from the stable project branch, and allows changes to be reviewed and validated before they are integrated. Thus pull requests provide a controlled integration point between the main code base and individual development activity [45].

12.1.2. Development Workflow

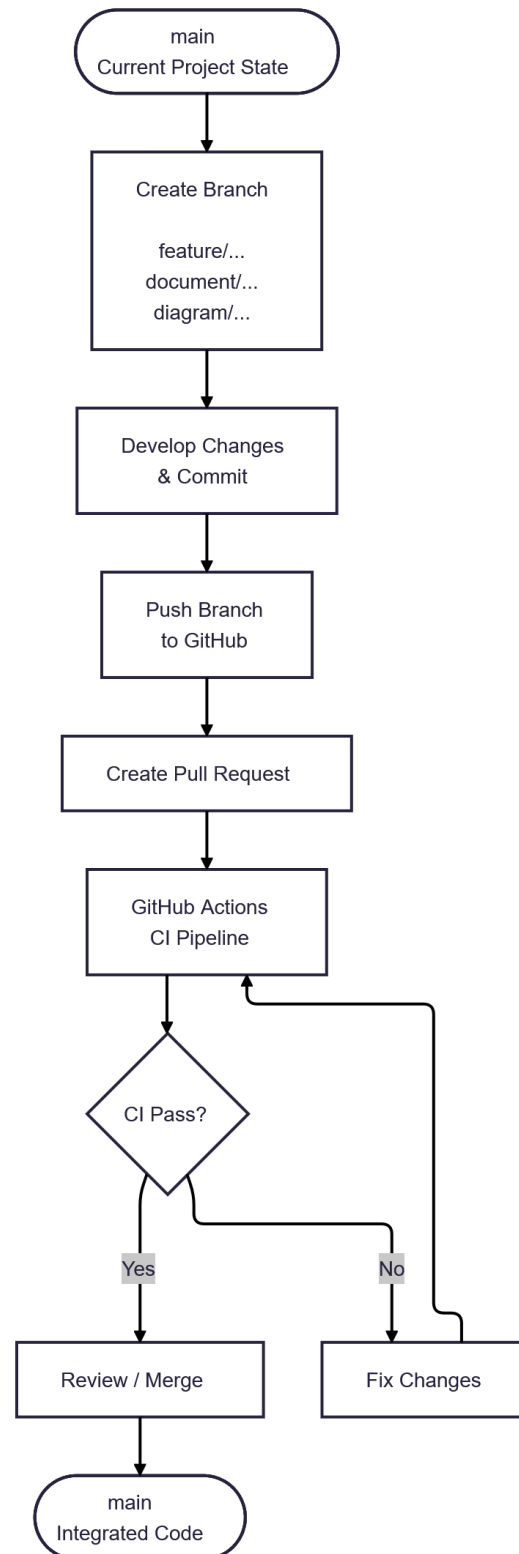


Figure 32: Trinity development and integration workflow. View diagram code on mermaid.live.

The workflow starts when a developer pulls the latest changes to their local main branch, and creates a branch for a specific contribution. Development is done in this branch so that incomplete or defected changes do not affect the integrated code base directly. Developers commit changes frequently and push their branch to GitHub. When the contribution is ready to be merged the pull request is created against the 'main' branch. The pull request allows the proposed changes to be reviewed and the automated validation pipeline to be run against the branch before it is merged [45]. The automated validation stage is divided into multiple levels of testing. Different components of the Trinity framework require different forms of validation; therefore, the pipeline does not rely on a single type of test. The agent code, server application, and server API are tested at their respective levels before the change can be considered ready for integration.

12.1.3. GitHub Actions Pipeline

GitHub Actions will be used to automate the Trinity build and quality-assurance process. A workflow consists of one or more automated jobs and can be triggered by repository events such as pushes and pull requests. This allows the same build and testing procedures to be executed consistently regardless of which developer submitted the change [44].

The pipeline will validate the different components of the Trinity framework separately before reporting the overall result of the workflow. The pipeline will perform the following validation stages where applicable:

1. Build the C/C++ agent.
2. Execute agent-level tests through CTest.
3. Build the C# server and associated services.
4. Execute C# unit tests.
5. Start the required server components and test infrastructure for integration testing.
6. Execute automated API tests using the Postman collection through the Postman CLI.
7. Report the results to the associated GitHub Actions workflow.
8. Prevent a pull request from being considered ready for integration when required validation fails.

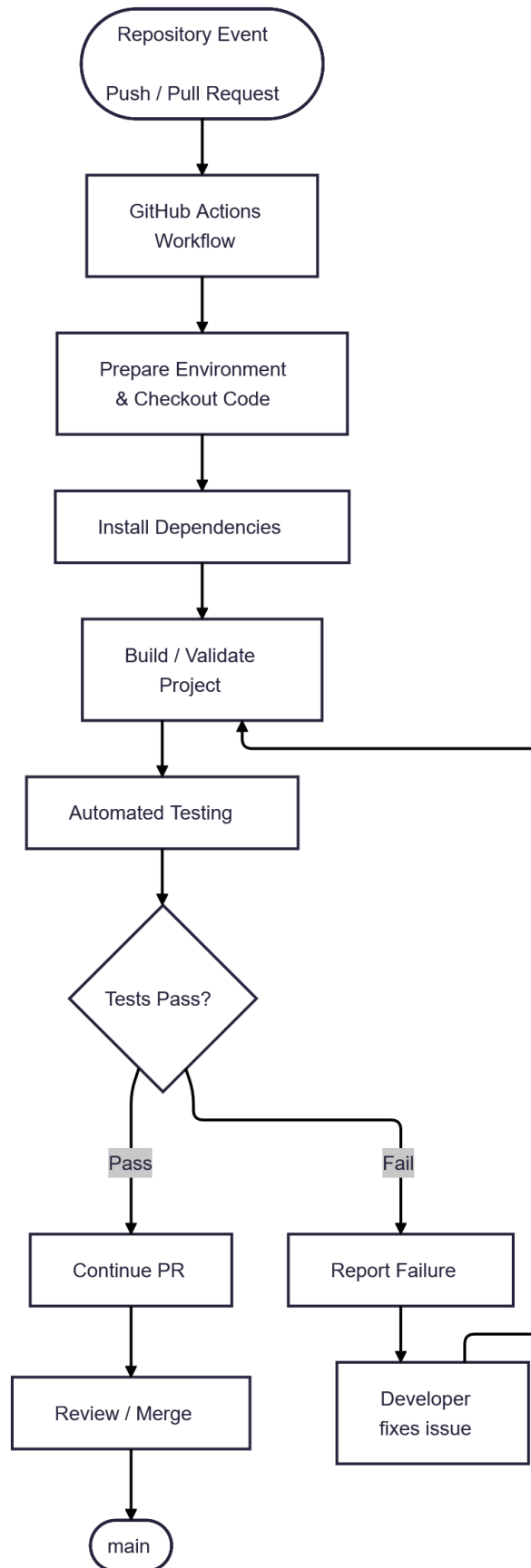


Figure 33: Trinity GitHub Actions Pipeline Flowchart. View diagram code on mermaid.live.

This separation is intended to make failures easier to diagnose. For example, a failure during the CTest stage indicates a problem within the agent component, while a failure in the C# unit-test stage indicates a problem within server-side application logic. An API test failure indicates that the externally exposed behaviour of the server does not match the expected API contract.

12.1.4. Automated Testing and Quality Assurance

The Trinity framework consists of multiple software components, including the deployed C/C++ agent, the C# server application and internal services, and the HTTP API used to communicate with the management application. Testing will therefore be performed at multiple levels to verify both individual components and their interactions.

12.1.4.1. Agent Build and Tests

The agent component will be built using CMake. The CI pipeline will configure and compile the agent as part of the automated validation process. A successful build verifies that the current source code can be configured and compiled in the CI environment. Where agent unit or component tests are implemented, CTest will be used to execute them. CTest provides a mechanism for registering and executing tests associated with a CMake build tree and reports a status result when a test fails [46].

Agent tests will focus on functionality that can be validated independently from the complete Trinity framework. Examples include:

- Command parsing and validation.
- Task deserialisation and construction.
- Agent configuration handling.
- Serialization and deserialization of agent messages.
- File handling functions.
- Response construction.
- Error handling for invalid or unexpected input.
- Individual agent functionality that can be executed without requiring a live server.

For example, if the agent receives a task containing a command and associated parameters, a unit test can verify that valid input is converted into the expected internal representation and that malformed input is rejected appropriately. Tests can also verify that the agent produces the expected response structure after executing a testable operation. The purpose of these tests is to identify defects within the agent implementation before the agent is integrated with the server. This reduces the possibility of an error in a low-level agent component appearing later as an apparently unrelated server or API failure.

12.1.4.2. C# Server Unit Tests

The C# server and internal application services will use a .NET unit-testing framework such as xUnit. Unit tests will focus on individual services, classes, and application functions without requiring the complete Trinity system to be running. xUnit provides support for automated testing within .NET projects and can be executed as part of the automated test process [47].

The server unit-test suite will ideally cover core application logic including:

- Authentication and token handling.
- User, role, and permission management.
- Campaign creation and membership rules.
- Agent registration and management.
- Listener configuration validation.
- Payload and agent-profile validation.
- Task creation and task-state management.
- Task queue behaviour.
- Task-result processing.
- File-transfer metadata handling.
- Database repository and service-layer logic where appropriate.
- Input validation and error handling.

For example, a task-management service could be tested to verify that a task created for a valid agent is placed into the expected pending state and associated with the correct operator and agent. Additional tests could verify that invalid agent identifiers are rejected and that task state transitions cannot occur when the requested transition is invalid. Authentication tests could verify that valid credentials result in the expected authentication response, while invalid credentials are rejected. Permission tests could verify that an operation requiring a particular permission cannot be performed by a user who does not possess that permission. These tests provide fast feedback on changes to individual server components without requiring a full end-to-end deployment.

12.1.4.3. Database and Persistence Tests

The server's interaction with PostgreSQL will also require testing because the database represents the persistent operational state of the framework. Persistence tests will verify that application services correctly create, retrieve, update, and delete the relevant entities and that the relationships defined by the data model behave as expected.

Tests will ideally cover relationships such as:

- Users and roles.
- Roles and permissions.

- Campaigns and campaign members.
- Campaigns and agents.
- Agents and tasks.
- Tasks and task results.
- Agents and file transfers.

For example, a task persistence test should verify that a task created for an agent is stored with the correct foreign-key relationships and can subsequently be retrieved using its associated agent. Tests should also verify that invalid relationships are rejected rather than allowing records to reference entities that do not exist. JSONB persistence will also be tested for variable task results. Different representative command-result structures can be stored and retrieved to verify that process information, system information, directory information, and other variable structures can be persisted without requiring a separate relational schema for each result type. Where database integration tests are used, the tests should execute against a controlled test database rather than the development or production database. This ensures that test data cannot affect operational data and that the database state can be reset between test runs.

12.1.4.4. Server API Tests

The Trinity server exposes functionality through API endpoints used by the management application. These endpoints will be tested using Postman collections containing requests and automated assertions. The Postman CLI can execute these collections from the command line, allowing the same API tests to be incorporated into the CI pipeline [48]. API tests will validate the behaviour visible to a client of the server rather than testing internal classes directly. This provides a higher-level test of whether the server components work together correctly.

The API test suite will ideally cover:

- Authentication and token issuance.
- Refresh-token behaviour.
- User and role management.
- Campaign creation and retrieval.
- Campaign membership.
- Agent registration and retrieval.
- Listener creation and configuration.
- Payload creation and retrieval.
- Task creation and retrieval.
- Task status updates.
- Task-result submission and retrieval.
- File-transfer metadata and associated endpoints.
- Validation of invalid requests.

- Authentication failures.
- Authorisation failures.
- Expected HTTP status codes.
- Response structure and required fields.

For example, an API test for task creation would first authenticate a test user, create or identify a test campaign and agent, submit a task through the relevant endpoint, and verify that the server returns the expected response. A subsequent request can retrieve the task and verify that it contains the expected agent, operator, status, and other metadata. Negative tests will also be included. These will send invalid or unauthorised requests and verify that the API rejects them using the expected HTTP status code and response structure. This is important because successful requests alone do not demonstrate that the API correctly enforces validation and access-control requirements. Postman collections will be executed automatically in the CI environment so that API regressions can be detected when changes are introduced. Postman documents that its CLI can run and validate API collections within CI/CD pipelines [48].

12.1.4.5. Integration Testing

Integration tests will verify interactions between individual Trinity components rather than only testing them in isolation. This is important because a component can pass its unit tests while still failing when communicating with another component.

Examples of integration scenarios include:

- The server creating a task and persisting it to PostgreSQL.
- An agent retrieving a pending task from the server.
- The agent processing the task and submitting a result.
- The server associating the returned result with the original task.
- Variable command output being persisted as JSONB.
- File-transfer activity being associated with the relevant agent and task.
- Authentication information being correctly propagated between the client and server.
- Listener configuration being used when establishing agent communication.

A representative task lifecycle integration test would create a test agent, create a task for that agent, verify that the task is persisted in the database, simulate the agent retrieving the task, submit a representative result, and finally verify that the task and result can be retrieved through the server API. This verifies the complete interaction between the API, application services, database, and task-processing components.

12.1.5. Pull Request Validation

Automated tests will be executed against pull requests before changes are merged into the 'main' branch. The purpose of this validation is to identify defects while the change is still isolated on its development branch. A pull request that introduces a compilation error will fail during the build stage. A change that breaks an existing unit test will fail during the corresponding unit-test stage. Similarly, a change that causes an API endpoint to return an incorrect response should be identified by the Postman API test stage. This creates a quality gate before integration into the main codebase. The intention is not to guarantee that every possible defect will be detected automatically, but to ensure that known and repeatable checks are performed consistently for each contribution.

12.1.6. Testing Strategy and Non-Functional Requirements

The DevOps workflow directly supports the non-functional requirements defined for the Trinity framework. Maintainability is supported through version control, structured branches, regular commits, pull requests, and automated tests. Changes remain traceable to individual contributions, while automated tests provide a regression suite that can be executed when the implementation changes. This reduces the risk that modifications to one component silently break existing functionality. Reliability is supported by requiring the framework to pass multiple levels of automated validation before integration. Unit tests verify individual functions, integration tests verify communication between components, and API tests verify externally observable server behaviour. Using multiple testing levels reduces reliance on a single form of testing and increases the likelihood that defects are detected before integration. Availability is supported by testing the application's ability to build and execute correctly in the automated environment and by identifying regressions before they are integrated into the main codebase. Although automated testing cannot guarantee runtime availability, preventing known build and functional defects from reaching the integrated version reduces avoidable causes of service failure. Compatibility is supported by executing the build and test process against the environments and toolchains targeted by the project. The agent build process verifies that the C/C++ component can be compiled using the project's configured CMake toolchain, while the server tests verify that the C# application and its dependencies operate correctly within the configured .NET environment. Portability is supported by defining the build and testing procedures as repeatable automated workflows rather than relying exclusively on developer-specific local environments. The same CMake, CTest, .NET test, and API-testing procedures can be executed by GitHub Actions, providing a consistent environment for validating changes.

12.1.7. Expected Testing Outcome

The intended outcome of the DevOps process is that every proposed contribution is subjected to an appropriate level of automated validation before being integrated into the main branch. The exact tests executed can depend on the components affected by a change, but changes to the agent should trigger agent build and test validation, changes to server functionality should trigger the relevant C# tests, and changes to API behaviour should trigger the API test suite. The overall process therefore provides a traceable path from development to integration: a developer creates an isolated branch, implements and commits a change, submits the change through a pull request, and the automated pipeline builds and tests the affected components. Only after the contribution has passed the required validation and review is it integrated into the main codebase.

13. Running Costs

The Trinity Command and Control (C2) Framework requires computing resources and supporting infrastructure to operate efficiently and reliably. The running costs include expenses associated with hosting, storage, network resources and ongoing system maintenance required to support day-to-day operations. The framework is designed to minimise operational expenses by using efficient technologies and scalable infrastructure these costs should be considered during deployment planning to ensure long-term sustainability system. By applying cost optimisation strategies enables organisations to maintain reliable system performance while controlling infrastructure and operational expenditure [49].

13.1. Infrastructure and Hosting

The Trinity Command and Control (C2) Framework requires a reliable infrastructure to support its core services which includes user authentication, agent communication, task management and secure data storage. The proposed implementation uses a PostgreSQL database to store structured operational data, while large binary files such as generated payloads, screenshots and downloaded documents are stored separately on the local file system to improve storage efficiency and maintain database performance.

The help ensure a reliable operation the framework should be deployed on infrastructure that provides sufficient processing power, memory, storage capacity and network connectivity to support multiple concurrent users and deployed agents. Selecting scalable hosting resources, the system to accommodate future growth while avoiding unnecessary operational costs. Cloud-based infrastructure or VPS (virtual private servers) can provide flexible deployment options which enables organizations to increase or decrease available resources as operational requirements change.

13.1.1. VPS Hosting Cost Comparison

The expected hosting cost will depend on the resources required by the team server, client infrastructure and redirectors. VPS providers generally offer different combinations of CPU, memory, storage and network resources, allowing the organisation to select an appropriate configuration according to the scale of the deployment. The cost should therefore be considered on a per-user basis as the number of users and deployed agents increases [50], [51].

Deployment Component	Purpose	Expected Resource Requirement	Cost Consideration
Team Server	Hosts the main C2 services, database and management functions	Higher CPU, memory and storage requirements	Highest infrastructure cost
Client Server	Supports client-side services and operational access	Moderate CPU, memory and storage requirements	Moderate infrastructure cost
Redirectors	Supports communication routing between agents and the team server	Lower resource requirements	Lower cost per instance

Real-world VPS and cloud hosting providers demonstrate different approaches to pricing and resource allocation. DigitalOcean provides Droplets with predefined combinations of CPU, memory, storage and transfer, while Amazon Web Services provides similar VPS functionality through Amazon Lightsail. Microsoft Azure provides configurable Linux Virtual Machines, allowing organisations to select resources according to their workload requirements [52].

The final VPS provider selection should consider both monthly pricing and the resources provided by each VPS configuration. Using appropriately sized VPS instances can prevent the organization from paying for resources that are not required while still providing sufficient capacity for the framework.

13.1.2. Agent Growth and Infrastructure Requirements

Infrastructure requirements are also expected to increase as the number of deployed agents and engagements grows. A smaller deployment may require only a limited number of agents, whereas multiple engagements could result in a substantially larger number of active agents communicating with the framework. The hosting environment should therefore be capable of scaling as agent numbers increase [52].

For example, an initial deployment may support a small number of agents during testing and pilot usage. As additional engagements are introduced, the number of active agents can increase, requiring additional processing, memory, storage and network resources. Monitoring agent growth allows infrastructure resources

to be increased when required rather than provisioning excessive resources from the beginning.

13.1.3. Cost at Different Deployment Scales

The expected infrastructure cost can be considered at different deployment scales to demonstrate how operational expenditure may change as the framework grows.

Expected Usage	Infrastructure Impact	Monthly Cost
Limited users and agents	Basic VPS resources and limited storage	R434/month
More users, agents and engagements	Increased CPU, memory, storage and network requirements	R835/month
Multiple engagements and many concurrent agents	Additional VPS resources, storage and redirectors	R1,269/month
<i>Prices of as of 17th August 2026</i>		

This approach allows infrastructure expenditure to remain aligned with actual system requirements. Instead of maintaining a large infrastructure environment from the beginning, resources can be expanded progressively as the number of users, engagements and agents increases [50].

13.2. Software and Development Tools

The Trinity Command and Control (C2) Framework is designed using widely adopted open-source technologies with reduces software licensing while providing a reliable and scalable development environment. The framework utilizes PostgreSQL as its relational database management system and stores large binary files separately from the database to improve storage efficiency and maintain system performance.

Development and maintenance activities require standard software engineering tools which includes version control system, database management tools, integrated development environment (IDEs) and testing utilities. Many of these tools available through open-source or community licences the primary ongoing costs are associated with infrastructure, maintenance and system administration rather than software licensing. By selecting cost-effective development tools and technologies helps reduce operational expenditure while ensuring the framework remains maintainable and scalable throughout its lifecycle [49].

13.3. Maintenance and Operational Costs

Ongoing maintenance is essential to ensure that the Trinity Command and Control (C2) Framework remains secure, reliable and available throughout its operational lifecycle. Regular maintenance activities include applying software updates, installing security patches, monitoring system performance, managing database backups and reviewing audit logs to identify potential issues before they affect system availability. These activities help to maintain system stability while reducing the risk of the operating disruptions.

Operational costs may also include server monitoring, storage expansion and periodic infrastructure upgrades as the number of users, deployed agents and stored operational data increases. By adopting proactive maintenance practices and cost optimisation strategies enables organizations to improve system reliability while controlling long-term operational expenses.

13.4. Cost Optimisation

Cost Optimisation is important considerations for ensuring the long-term sustainability of the Trinity Command and Control (C2) Framework. Infrastructure resource should be allocated according to operational requirements to avoid unnecessary expenditure while maintaining acceptable system performance. Where appropriate scalable hosting solutions and efficient resource management practices can be used to optimise computing, storage and network utilisation [49].

The framework should also utilise open-source technologies and modular system components to minimise licensing costs and simplify future upgrades. Regular monitoring of infrastructure opportunities for optimisation while ensuring that operational costs remain aligned with the framework requirements.

13.5. Summary

The running costs of the Trinity Command and Control (C2) Framework are primarily associated with infrastructure, hosting, storage and ongoing maintenance. By using scalable infrastructure, open-source technologies and cost optimisation practices, the framework can operate efficiently while keeping long-term operational costs manageable.

14. Change Management

Change management is defined as the process used to control and document changes made to the Trinity C2 Framework. It ensures that changes to requirements, source code, system architecture and documentation are properly reviewed and tested before being incorporated into the project.

Changes may be required to new requirements, identified issues, security concerns, technical limitations or stakeholder feedback. Each proposed change should be evaluated to determine its potential impact on the existing system before implementation.

The project uses version control to track changes to both the source code and documentation. Changes are developed separately, reviewed by team members and merged into the main project once they have been approved and verified.

14.1. Change Request Process

Changes to the Trinity C2 Framework should follow a structured process to minimize disruption to the existing system. This process exists with the following stages:

1. **Change Identification:** A change is identified through a new requirement, defect, security concern, technical issue or stakeholder feedback.
2. **Change Documentation:** The proposed change is documented which includes a description of the changes and the reason it is required.
3. **Impact Assessment:** The team evaluates the potential impact of the change on the system's requirements, architecture, implementation, security, performance and documentation.
4. **Review and Approval:** The proposed change reviewed by the relevant team members before development begins.
5. **Implementation:** Approved changes are implemented separately from the main version of the project to reduce the risk of affecting existing functionality.
6. **Testing and Verification:** The implemented change is tested to confirm that it functions as intended and does not negatively affect existing functionality.
7. **Merge and Documentation:** Once the change has been verified, it can be incorporated into the main project and affected documentation is updated.

14.2. Version Control and Branching Strategy

The Trinity C2 Framework uses version control to manage changes to the project's source code and documentation. Version control allows team members to work on different changes without directly modifying the main version of the project.

Each change should be developed in a separate branch before being merged into the main branch. This allows changes to be reviewed and tested independently while reducing the risk of introducing problems into the main project.

Branches should follow the naming conventions established by the project team:

Verb/thing-type_thing-name

For example a branch created to update the requirements documentation could follow a naming structure such as:

Update/docs_requirements

After the work has been completed, the changes should be reviewed and tested before being merged into the main branch. This provides a clear history of changes and allows the team to identify which changes were made and why.

14.3. Documentation Changes

Changes to project documentation should also be tracked through version control. When a requirement or design decision is changes, the affected documentation should be updated so that it remains consistent with the current system design.

14.4. Change Impact and Risk Assessment

Before an approved change is implemented, its potential impact on the Trinity C2 Framework should be assessed. The team must consider how the proposed change may affect existing requirements, system architecture, functionality, security, performance and documentation.

Change that could significantly affect the system should receive additional review before implementation. The team should also identify potential risks such as introducing new defects, affecting existing functionality or creating inconsistencies between the implementation and documentation.

The results of the impact assessment should be considered when deciding whether the change can proceed and what testing may be required after implementation. This helps the team make controlled changes while reducing the possibility of negatively affecting the existing system.

14.5. Change Implementation and Testing

Once a change has been reviewed and approved, it will be implemented in the appropriate development branch rather than directly in the main branch.

The implementation will follow the project existing development standards and should avoid unnecessary changes to unrelated functionality.

After implementation the change will be tested to verify that the intended functionality works correctly. Existing functionality should also be checked to identify an unintended effect caused by the change.

If testing identifies problems the change should be corrected and tested again before it is considered ready for merging. Once the change has been successfully verified it can proceed through the project's normal review and merge process.

14.6. Change Tracking and Review

All approved changes to the Trinity C2 Framework will be tracked through the project's version control system. This provides a record of changes made to the source code and documentation and allows the team to identify when changes were introduced and who contributed to them.

Changes should be reviewed after implementation to confirm that they meet the original requirements and that the associated documentation remains accurate. If a change affects other parts of the system all other relevant requirements must also update such as diagrams or documentation.

The change management process should be reviewed throughout the project to identify areas that can be improved. This ensures that changes remain controlled and that the project documentation continues to reflect the current state of the Trinity C2 Framework.

15. Prototype

Figma link: <https://www.figma.com/design/ZG2x3NuCFikRXIuJGUA4Qp/Trinity-C2-Prototype?node-id=0-1&p=f>

15.1. Login

The below screenshot is the login page for the Trinity C2 framework. This page will serve as the authentication screen that determines access restriction for the frameworks main interface.

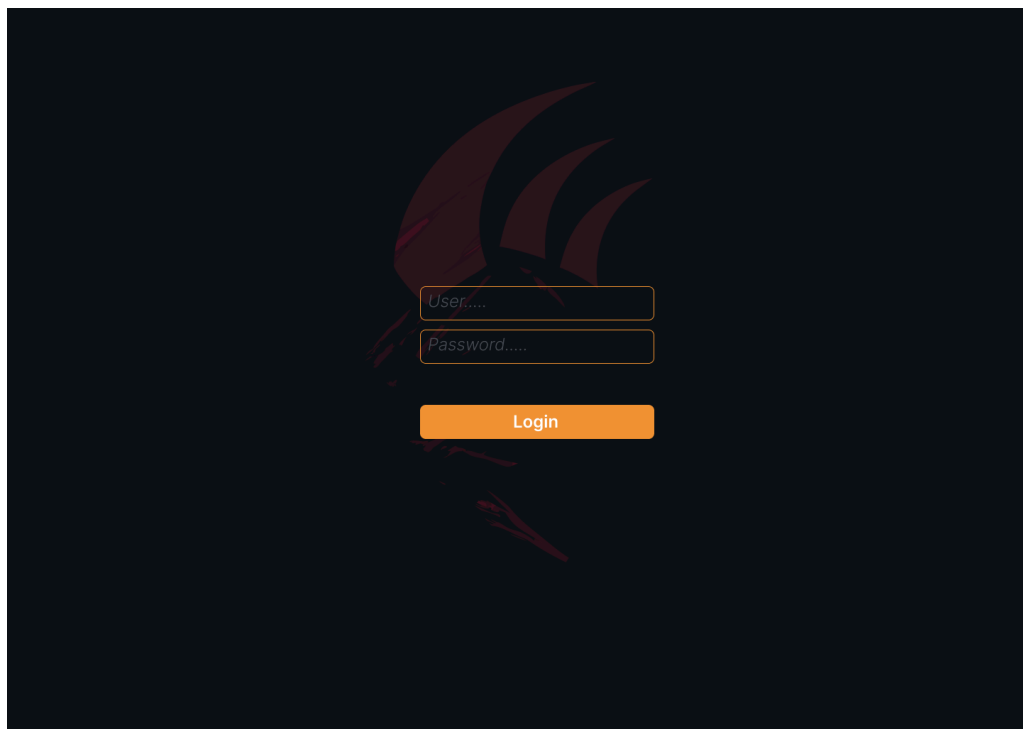


Figure 34: Login interface prototype.

15.2. Dashboard

This is the main dashboard for the Trinity C2 framework, providing an overview of system activity and quick access to core functions. It displays key statistics such as online clients, active campaigns, stored files and pending tasks. It also displays panels showing recent clients, campaigns, system activity and the overall system status. On the left is the navigation menu that allows for the user to switch between the various different pages. The Quick Actions panel allows users to add clients, create jobs, upload files, view reports and adjust settings, with the panels below the main page illustrates the pop ups that will appear when these quick action buttons are pressed.

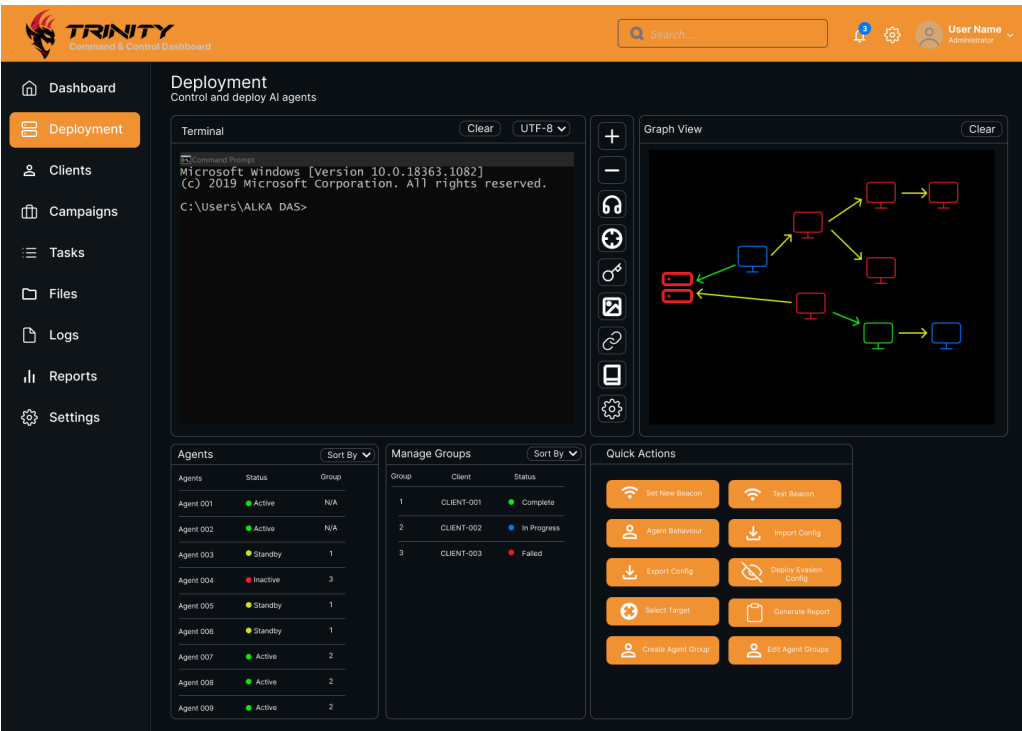


Figure 35: Deployment interface prototype.

15.3. Deployment

The Deployment page serves as the central interface for managing and controlling deployed agents. It provides a terminal for executing commands, a graphical view for monitoring agent connections and communication paths and an agents list showing the status of currently deployed agents. This page also includes quick action controls for common deployment and management operations as displayed by the pop ups that can be seen displayed below the main deployment page in the image below.

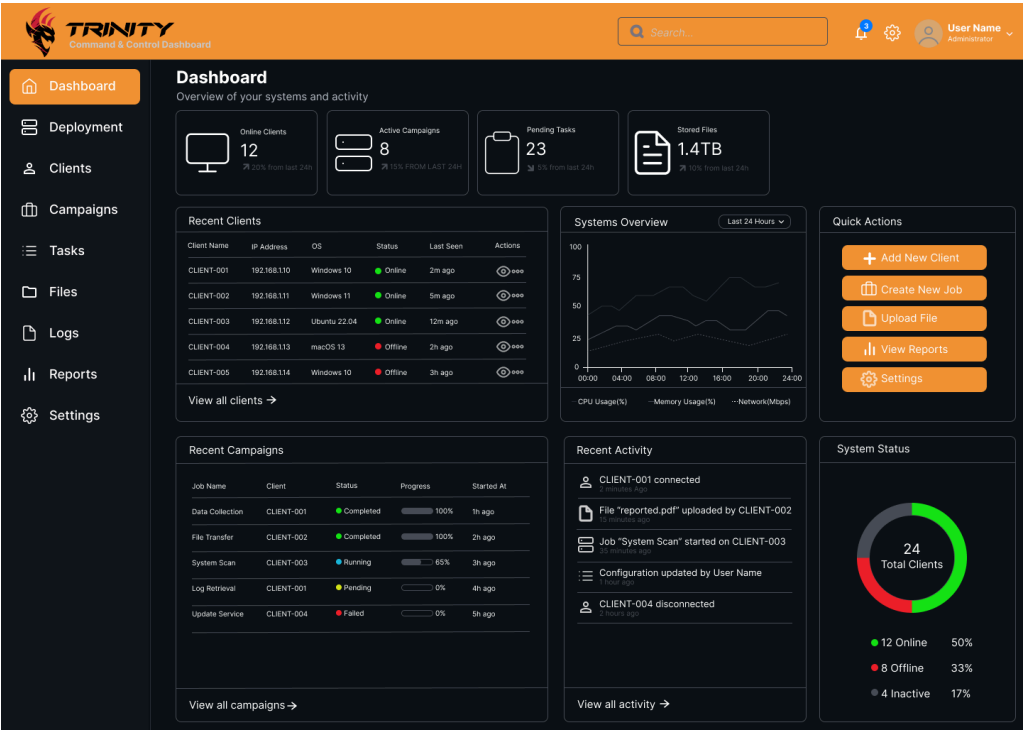


Figure 36: Dashboard interface prototype.

15.4. Clients

The Clients page offers a centralised interface for managing and monitoring all connected clients within Trinity C2. It presents an overview of key details such as client statistics, total clients, online and offline clients and client activity status. Shown below is also a detailed table containing information such as client ID, hostname, operating system, status and client activity. A map and client distribution chart serve to provide additional visual insight into where clients are located and their current status. The quick actions shown below the main page illustrate the different tools available on this page such as announcements, job execution, locking clients, accessing command shell and viewing individual clients.

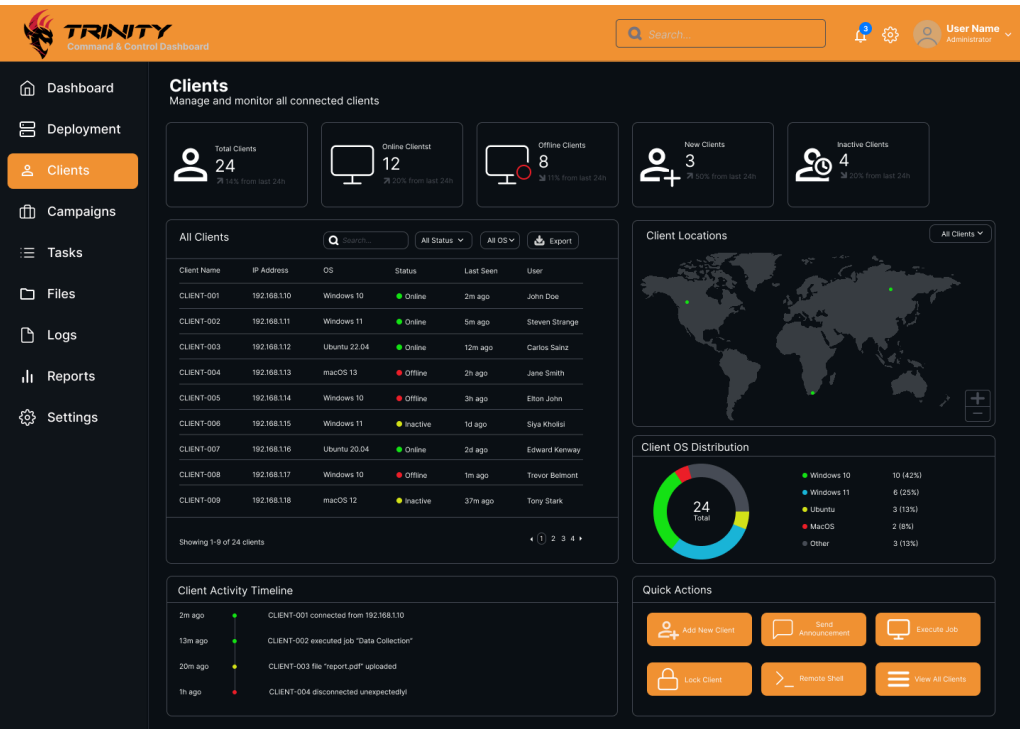


Figure 37: Clients interface prototype.

15.5. Campaigns

The Campaigns page provides a centralised interface for creating, managing and monitoring campaigns. It displays key campaign statistics, including the number of active campaigns, pending tasks and completed campaigns. Detailed tables provide information on individual campaigns, their status and client progress, with visual charts showing campaign status, activity distribution and overall performance, which allows users to quickly assess campaign activity. Below the main page are the quick action pop ups showing campaign creation, campaign management, templates, importing existing campaigns, cloning campaigns and viewing individual campaigns. This page overall serves as the primary workspace for organising, configuring and monitoring campaigns.



Figure 38: Campaigns interface prototype.

15.6. Tasks

The Tasks page acts as a centralised interface for creating, managing and monitoring tasks within Trinity C2. It displays statistics such as completed tasks, active tasks, pending tasks and failed tasks, with detailed tables providing information on individual tasks, their status, associated campaigns, progress and priority. The visual charts summarize task priorities and overall status, allowing users to quickly assess activity and workload. The quick actions control creating tasks, scheduling tasks, importing tasks, managing templates, performing bulk updates, viewing reports and accessing individual tasks.

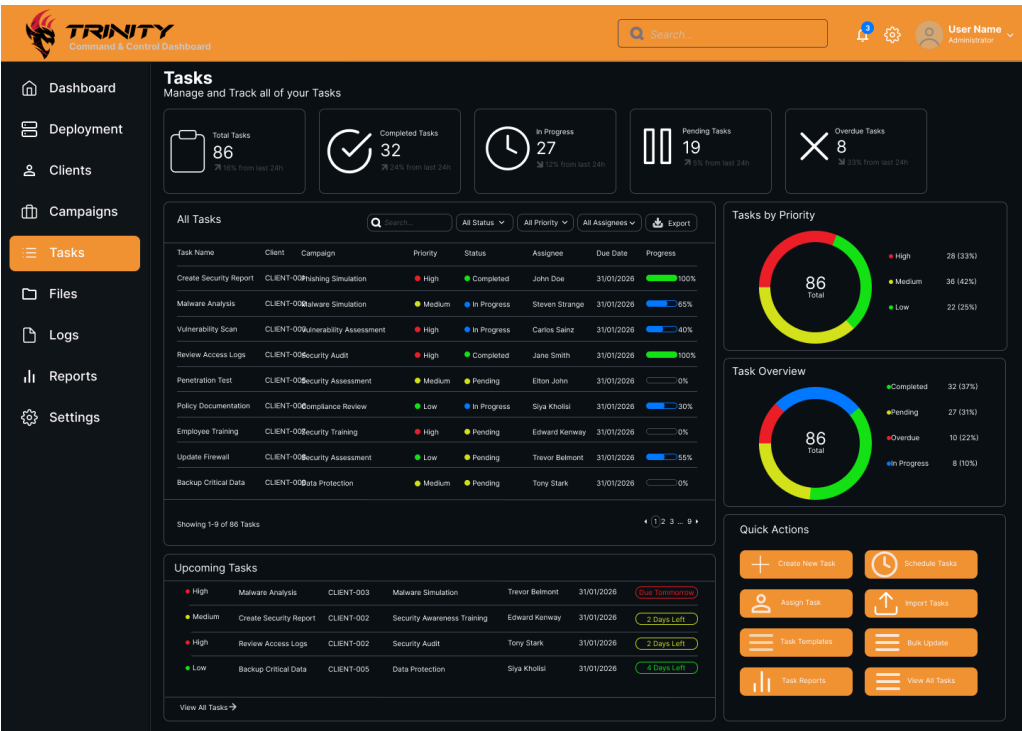


Figure 39: Tasks interface prototype.

15.7. Files

The Files page acts as the central interface for managing, monitoring and transferring files. Key statistics include total number of files, folders, storage capacity and upload and download activity, while the main file table provides details about individual files, their size, location and status. Visual charts serve as representation for overall storage usage and file type distribution, allowing users to quickly understand how the available is being utilised. The quick actions facilitate uploading, scanning, downloading and sharing of files, folder creation, link generation, file viewing and performing bulk uploads.

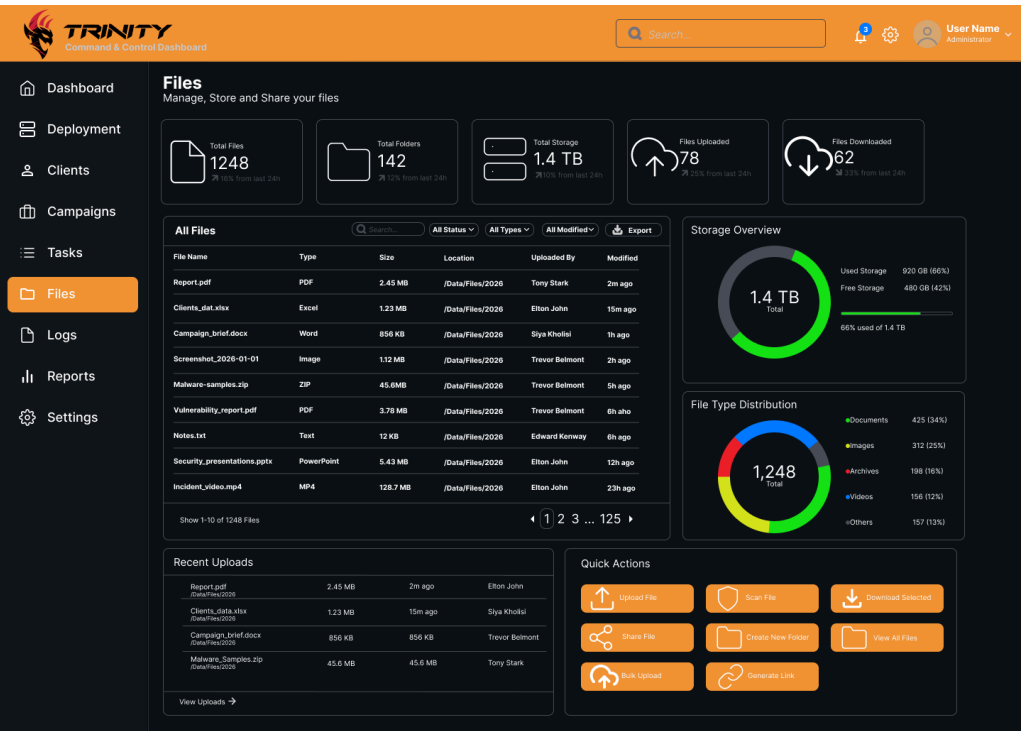


Figure 40: Files interface prototype.

15.8. Logs

The Logs page acts as the interface for monitoring and reviewing system and user activity, the key statistics include total logs, warnings, errors, critical events and alerts, with the main logs table providing information about individual events, timestamps, severity, source, event type and user or system logs. The visual chart summarises the log severity and events distribution, while the additional panels highlight top clients by log count and recent critical activity. The quick actions control searching, filtering, log management, creating alerts, exporting logs, clearing logs and log configuration settings.

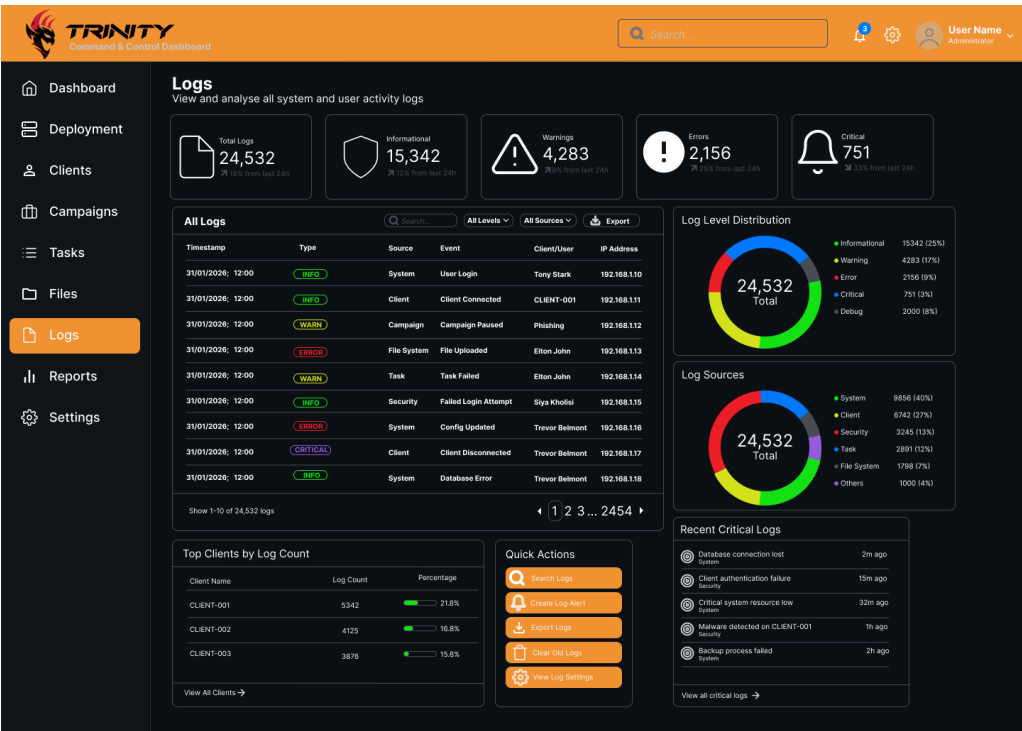


Figure 41: Logs interface prototype.

15.9. Reports

The Reports page provides the interface for creating, scheduling and reviewing reports for Trinity C2. The key statistics displayed are total number of reports, completed reports, pending reports, generated data and failed reports. The main report table provides details about individual reports, their status, their type, associated user and creation date. The charts summarise the report statuses and their distribution by type. The quick actions for this page allow for report creation, report scheduling, managing report templates, exporting reports and viewing existing reports.



Figure 42: Files interface prototype.

15.10. Settings

The settings page offers a centralised interface for configuring and managing the system and its underlying framework settings. It presents system information, C2 framework configuration, active policies, authentication, logging and other operational preferences. This page also includes quick actions that control configuration and policy management, C2 profile creation, exporting and importing configurations, resetting C2 settings, adjusting evasion options, testing connections and running operational security (OPSEC) checks.

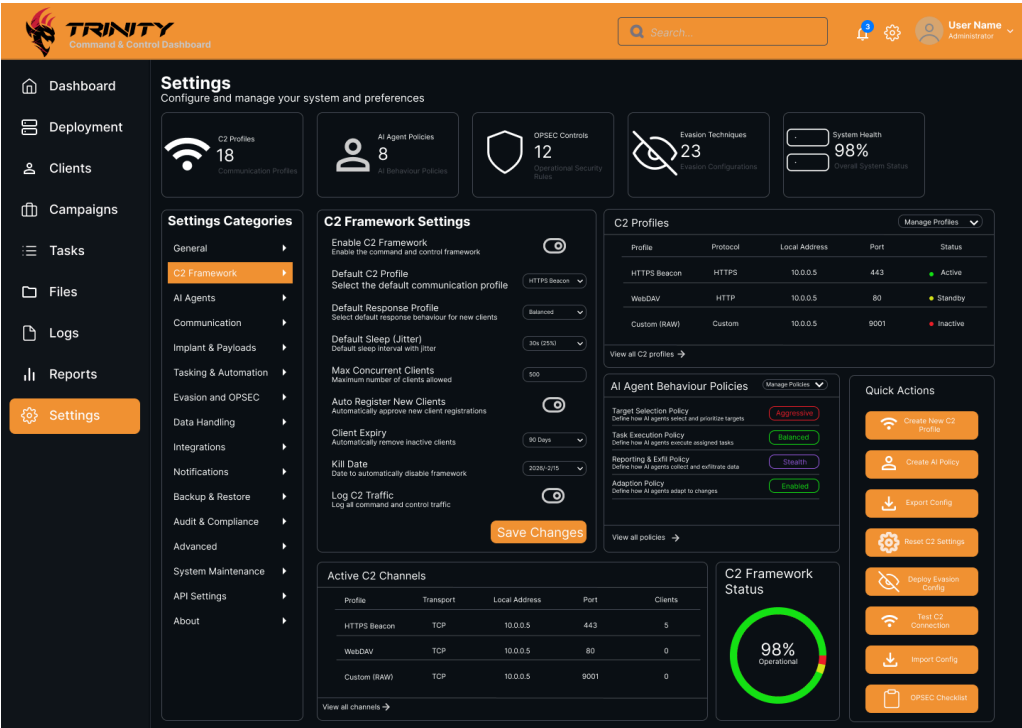


Figure 43: Settings interface prototype.

16. Appendix A - Declaration of Authenticity

Plagiarism occurs in a variety of forms. Ultimately, though, it refers to the use of the words, ideas, or images of another person without acknowledging the source using the required conventions. The IIE publishes a Quick Reference Guide (available on The IIE Library website) that provides more detailed guidance, but a brief description of plagiarism and referencing is included below for your reference. It is vital that you are familiar with this information and the Intellectual Integrity Policy before attempting any assignments.

The IIE respects the intellectual property of other people and requires its students to be familiar with the necessary referencing conventions. Please ensure that you seek assistance in this regard before submitting work if you are uncertain.

If you fail to acknowledge the work or ideas of others or do so inadequately this will be handled in terms of the Intellectual Integrity Policy (IIE023 – available in the library) and/or the Student Code of Conduct policy (IIE026)– depending on whether or not plagiarism and/or cheating (passing off the work of other people as your own by copying the work of other students or copying off the Internet or from another source) is suspected.

Reiteration of the Declaration you have signed:

1. I have been informed about the seriousness of acts of plagiarism
2. I understand what plagiarism is.
3. I am aware that The Independent Institute of Education (IIE) has a policy regarding plagiarism and that it does not accept acts of plagiarism.
4. I am aware that the Intellectual Integrity Policy and the Student Code of Conduct prescribe the consequences of plagiarism.
5. I am aware that referencing guides are available in my student handbook or equivalent and in the library and that following them is a requirement for successful completion of my programme.
6. I am aware that should I require support of assistance in using referencing guides to avoid plagiarism, I may speak to the lecturers, the librarian, or the campusADC/Campus Co-ordinator for clarification.
7. I am aware of the consequences of plagiarism.

Please ask for assistance prior to submitting work if you are at all unsure.

I, James Marshall, ID Number, 0111285323082, hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: ST10434366

STUDENT NAME: James Marshall

STUDENT SIGNATURE:

A handwritten signature in black ink, appearing to be 'J. Marshall', written over a horizontal line.

DATE: 2026-08-17

17. Appendix B - Declaration of Authenticity

Plagiarism occurs in a variety of forms. Ultimately, though, it refers to the use of the words, ideas, or images of another person without acknowledging the source using the required conventions. The IIE publishes a Quick Reference Guide (available on The IIE Library website) that provides more detailed guidance, but a brief description of plagiarism and referencing is included below for your reference. It is vital that you are familiar with this information and the Intellectual Integrity Policy before attempting any assignments.

The IIE respects the intellectual property of other people and requires its students to be familiar with the necessary referencing conventions. Please ensure that you seek assistance in this regard before submitting work if you are uncertain.

If you fail to acknowledge the work or ideas of others or do so inadequately this will be handled in terms of the Intellectual Integrity Policy (IIE023 – available in the library) and/or the Student Code of Conduct policy (IIE026)– depending on whether or not plagiarism and/or cheating (passing off the work of other people as your own by copying the work of other students or copying off the Internet or from another source) is suspected.

Reiteration of the Declaration you have signed:

1. I have been informed about the seriousness of acts of plagiarism
2. I understand what plagiarism is.
3. I am aware that The Independent Institute of Education (IIE) has a policy regarding plagiarism and that it does not accept acts of plagiarism.
4. I am aware that the Intellectual Integrity Policy and the Student Code of Conduct prescribe the consequences of plagiarism.
5. I am aware that referencing guides are available in my student handbook or equivalent and in the library and that following them is a requirement for successful completion of my programme.
6. I am aware that should I require support of assistance in using referencing guides to avoid plagiarism, I may speak to the lecturers, the librarian, or the campusADC/Campus Co-ordinator for clarification.
7. I am aware of the consequences of plagiarism.

Please ask for assistance prior to submitting work if you are at all unsure.

I, Lyle William Richardson, ID Number, 9802125027086, hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NAME: Lyle William Richardson

STUDENT NUMBER: ST10322521

STUDENT SIGNATURE:

A handwritten signature in black ink, appearing to be 'Lyle William Richardson', with a large, stylized initial 'L' and a wavy line extending to the right.

DATE: 2026-08-17

18. Appendix C - Declaration of Authenticity

Plagiarism occurs in a variety of forms. Ultimately, though, it refers to the use of the words, ideas, or images of another person without acknowledging the source using the required conventions. The IIE publishes a Quick Reference Guide (available on The IIE Library website) that provides more detailed guidance, but a brief description of plagiarism and referencing is included below for your reference. It is vital that you are familiar with this information and the Intellectual Integrity Policy before attempting any assignments.

The IIE respects the intellectual property of other people and requires its students to be familiar with the necessary referencing conventions. Please ensure that you seek assistance in this regard before submitting work if you are uncertain.

If you fail to acknowledge the work or ideas of others or do so inadequately this will be handled in terms of the Intellectual Integrity Policy (IIE023 – available in the library) and/or the Student Code of Conduct policy (IIE026)– depending on whether or not plagiarism and/or cheating (passing off the work of other people as your own by copying the work of other students or copying off the Internet or from another source) is suspected.

Reiteration of the Declaration you have signed:

1. I have been informed about the seriousness of acts of plagiarism
2. I understand what plagiarism is.
3. I am aware that The Independent Institute of Education (IIE) has a policy regarding plagiarism and that it does not accept acts of plagiarism.
4. I am aware that the Intellectual Integrity Policy and the Student Code of Conduct prescribe the consequences of plagiarism.
5. I am aware that referencing guides are available in my student handbook or equivalent and in the library and that following them is a requirement for successful completion of my programme.
6. I am aware that should I require support of assistance in using referencing guides to avoid plagiarism, I may speak to the lecturers, the librarian, or the campusADC/Campus Co-ordinator for clarification.
7. I am aware of the consequences of plagiarism.

Please ask for assistance prior to submitting work if you are at all unsure.

I, Sky Martin, ID Number, 9704275431086, hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: ST10286905

STUDENT NAME: Sky Martin

STUDENT SIGNATURE:

A handwritten signature in black ink, appearing to be 'Sky Martin', written in a cursive style with a large loop at the top and a long horizontal stroke at the bottom.

DATE: 2026-08-17

19. Appendix D - Declaration of Authenticity

Plagiarism occurs in a variety of forms. Ultimately, though, it refers to the use of the words, ideas, or images of another person without acknowledging the source using the required conventions. The IIE publishes a Quick Reference Guide (available on The IIE Library website) that provides more detailed guidance, but a brief description of plagiarism and referencing is included below for your reference. It is vital that you are familiar with this information and the Intellectual Integrity Policy before attempting any assignments.

The IIE respects the intellectual property of other people and requires its students to be familiar with the necessary referencing conventions. Please ensure that you seek assistance in this regard before submitting work if you are uncertain.

If you fail to acknowledge the work or ideas of others or do so inadequately this will be handled in terms of the Intellectual Integrity Policy (IIE023 – available in the library) and/or the Student Code of Conduct policy (IIE026)– depending on whether or not plagiarism and/or cheating (passing off the work of other people as your own by copying the work of other students or copying off the Internet or from another source) is suspected.

Reiteration of the Declaration you have signed:

1. I have been informed about the seriousness of acts of plagiarism
2. I understand what plagiarism is.
3. I am aware that The Independent Institute of Education (IIE) has a policy regarding plagiarism and that it does not accept acts of plagiarism.
4. I am aware that the Intellectual Integrity Policy and the Student Code of Conduct prescribe the consequences of plagiarism.
5. I am aware that referencing guides are available in my student handbook or equivalent and in the library and that following them is a requirement for successful completion of my programme.
6. I am aware that should I require support of assistance in using referencing guides to avoid plagiarism, I may speak to the lecturers, the librarian, or the campusADC/Campus Co-ordinator for clarification.
7. I am aware of the consequences of plagiarism.

Please ask for assistance prior to submitting work if you are at all unsure.

I, Connaire Myall, ID Number, 9807155167082, hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: ST10297849

STUDENT NAME: Connaire Myall

STUDENT SIGNATURE:

A handwritten signature in black ink, appearing to be 'Connaire Myall', written in a cursive, stylized script.

DATE: 2026-08-17

20. Appendix E - Declaration of Authenticity

Plagiarism occurs in a variety of forms. Ultimately, though, it refers to the use of the words, ideas, or images of another person without acknowledging the source using the required conventions. The IIE publishes a Quick Reference Guide (available on The IIE Library website) that provides more detailed guidance, but a brief description of plagiarism and referencing is included below for your reference. It is vital that you are familiar with this information and the Intellectual Integrity Policy before attempting any assignments.

The IIE respects the intellectual property of other people and requires its students to be familiar with the necessary referencing conventions. Please ensure that you seek assistance in this regard before submitting work if you are uncertain.

If you fail to acknowledge the work or ideas of others or do so inadequately this will be handled in terms of the Intellectual Integrity Policy (IIE023 – available in the library) and/or the Student Code of Conduct policy (IIE026)– depending on whether or not plagiarism and/or cheating (passing off the work of other people as your own by copying the work of other students or copying off the Internet or from another source) is suspected.

Reiteration of the Declaration you have signed:

1. I have been informed about the seriousness of acts of plagiarism
2. I understand what plagiarism is.
3. I am aware that The Independent Institute of Education (IIE) has a policy regarding plagiarism and that it does not accept acts of plagiarism.
4. I am aware that the Intellectual Integrity Policy and the Student Code of Conduct prescribe the consequences of plagiarism.
5. I am aware that referencing guides are available in my student handbook or equivalent and in the library and that following them is a requirement for successful completion of my programme.
6. I am aware that should I require support of assistance in using referencing guides to avoid plagiarism, I may speak to the lecturers, the librarian, or the campusADC/Campus Co-ordinator for clarification.
7. I am aware of the consequences of plagiarism.

Please ask for assistance prior to submitting work if you are at all unsure.

I, Morné Viljoen, ID Number, 0211175083081, hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: ST10374994

STUDENT NAME: Morné Viljoen

STUDENT SIGNATURE:

A handwritten signature in black ink, appearing to be 'MV', written in a cursive, stylized manner.

DATE: 2026-08-17

21. AI Declaration

21.1. Disclosure of AI Usage

AI usage will be documented as entries using the format below:

Sections: Section(s) within the assessment in which generative AI was used, e.g. Conclusion, `source.cs` lines x - y, etc.

Name of AI tool(s) used: Name of AI tool(s) used.

Purpose/intention behind use: Purpose/intention behind use, e.g.

Brainstorming; Meaning making; Initial investigation; Correcting grammar and spelling; Sourcing ideas for xyz (structure; design, etc.)

Date(s) in which generative AI was used: yyyy-MM-dd

A link to the actual generative AI chat: <https://generic.ai/share/xyz>

Disclosure of AI Usage in my Assessment.

Sections: Security

Name of AI tool(s) used: Gemini

Purpose/intention behind use: Sourcing ideas for expanding and fleshing out the Security section

Date(s) in which generative AI was used: 2026-08-10

A link to the actual generative AI chat: [<https://gemini.google.com/share/709f0c4b69cc?skid=9951586c-201e-4b93-b55f-36fe25a6a0ca>] (<https://gemini.google.com/share/709f0c4b69cc?skid=9951586c-201e-4b93-b55f-36fe25a6a0ca>)

Disclosure of AI Usage in my Assessment.

Sections: Data Schema

Name of AI tool(s) used: ChatGPT

Purpose/intention behind use: I felt like I was repeating myself and needed to be sure what I could just cut

Date(s) in which generative AI was used: 2026/08/10

A link to the actual generative AI chat: <https://chatgpt.com/share/6a7a1250-e4a0-83ea-a0fe-eda43a6ef576>

Disclosure of AI Usage in my Assessment.

Sections: DevOps

Name of AI tool(s) used: ChatGPT

Purpose/intention behind use: Modified my write up to match my team leads more technical writing style.

Date(s) in which generative AI was used: 2026/08/15

A link to the actual generative AI chat: <https://chatgpt.com/share/6a80721a-1294-83ea-920f-2ac0f669792a>

Disclosure of AI Usage in my Assessment.

Sections: Running Costs and Change Management

Name of AI tool(s) used: ChatGPT

Purpose/intention behind use: To help quickly setup tables, cash convergence to African currency, reformatting, idea generation and writing assistance.

Date(s) in which generative AI was used: 2026/08/13

A link to the actual generative AI chat: <https://chatgpt.com/c/6a8352d6-6310-83ea-8ele-bdda67eb09b0>

Disclosure of AI Usage in my Assessment.

Sections: Security

Name of AI tool(s) used: Gemini

Purpose/intention behind use: Validating alignment and consistency of Security.md with the overall documentation

Date(s) in which generative AI was used: 2026-08-17

A link to the actual generative AI chat: <https://share.gemini.google/T85UBw35ejUp>

Disclosure of AI Usage in my Assessment.

Sections: Security

Name of AI tool(s) used: Gemini Purpose/intention behind use: Rewriting the Security draft to align vocabulary, tone, and documentation style with colleague writing in Architecture Artifacts

Date(s) in which generative AI was used: 2026-08-17

A link to the actual generative AI chat: <https://share.gemini.google/JtmEfesfcQxK>

Sections: User Roles

Name of AI tool(s) used: Gemini

Purpose/intention behind use: Identifying Unicode values and HTML entities for check mark and cross mark symbols, including clarification of emoji color representations.

Date(s) in which generative AI was used: 2026-08-17

A link to the actual generative AI chat: <https://share.google/aimode/tTfNZzPtcV6C9f90u>

References

- [1] L. Chung, B. A. Nixon, E. Yu, and J. Mylopoulos, *Non-Functional Requirements in Software Engineering*. Springer Science & Business Media, 2012.
- [2] A. Mishra, Y. I. Alzoubi, and N. Gavrilovic, "Quality Attributes of Software Architecture in IoT-based Agricultural Systems," *Smart Agricultural Technology*, vol. 8, p. 100523, Aug. 2024, doi: 10.1016/j.atech.2024.100523.
- [3] M. Yaseen, W. Mehmood, F. Hameed, and M. A. Nauman, "Scalability and Limitations of Existing Software Requirements Prioritization Techniques: A Systematic Literature Review," *Journal of Software: Evolution and Process*, vol. 37, no. 8, p. e70039, 2025, doi: 10.1002/smr.70039.
- [4] K. Sagar and A. Saha, "A Systematic Review of Software Usability Studies," *International Journal of Information Technology*, Dec. 2017, doi: 10.1007/s41870-017-0048-1.
- [5] A. Abran, K. T. Al-Sarayreh, and J. J. Cuadrado-Gallego, "A Standards-Based Reference Framework for System Portability Requirements," *Computer Standards & Interfaces*, vol. 35, no. 4, pp. 380–395, Jun. 2013, doi: 10.1016/j.csi.2012.11.003.
- [6] Y. Zhao, L. Gong, W. Yang, and Y. Zhou, "How Accessibility Affects Other Quality Attributes of Software? A Case Study of GitHub," *Science of Computer Programming*, vol. 231, p. 103027, Jan. 2024, doi: 10.1016/j.scico.2023.103027.
- [7] B. Alexander and M. Denis, "SECURITY AUDIT LOGGING IN MICROSERVICE-BASED SYSTEMS: SURVEY OF ARCHITECTURE PATTERNS," *Вопросы кибербезопасности*, no. 2 (42), pp. 71–80, 2021, Accessed: Aug. 09, 2026. [Online]. Available: <https://cyberleninka.ru/article/n/security-audit-logging-in-microservice-based-systems-survey-of-architecture-patterns>
- [8] "13.1. Introduction." Accessed: Aug. 12, 2026. [Online]. Available: <https://www.postgresql.org/docs/18/mvcc-intro.html>
- [9] "Managing Objects." Accessed: Aug. 10, 2026. [Online]. Available: <https://docs.min.io/aistor/administration/console/managing-objects/>
- [10] "Chapter 8. Data Types." Accessed: Aug. 15, 2026. [Online]. Available: <https://www.postgresql.org/docs/18/datatype.html>
- [11] "Data Modeling in MongoDB - Database Manual - MongoDB Docs." Accessed: Aug. 15, 2026. [Online]. Available: <https://www.mongodb.com/docs/manual/data-modeling/>

- [12] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, "Observer," in *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison Wesley, pp. 293–305.
- [13] "Factory Method Design Pattern." Accessed: Aug. 17, 2026. [Online]. Available: <https://www.geeksforgeeks.org/system-design/factory-method-for-designing-pattern/>
- [14] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, "Abstract Factory," in *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison Wesley, pp. 87–97.
- [15] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, "Strategy," in *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison Wesley, pp. 315–325.
- [16] "What Is P2P (Peer-to-Peer Process)?" Accessed: Aug. 16, 2026. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/what-is-p2p-peer-to-peer-process/>
- [17] "Beacon Covert Peer-to-Peer Communication." Accessed: Aug. 16, 2026. [Online]. Available: https://hstechdocs.helpsystems.com/manuals/cobaltstrike/current/userguide/content/topics/listener-infrastructure_peer-2-peer.htm
- [18] "Client-Server Model." Accessed: Aug. 16, 2026. [Online]. Available: <https://www.geeksforgeeks.org/system-design/client-server-model/>
- [19] "HTTP Beacon and HTTPS Beacon." Accessed: Aug. 16, 2026. [Online]. Available: https://hstechdocs.helpsystems.com/manuals/cobaltstrike/current/userguide/content/topics/listener-infrastructure_beacon-http-https.htm
- [20] S.-ox12D. Development, "User Access Control Bypass via CMSTP." Accessed: Aug. 17, 2026. [Online]. Available: <https://medium.com/@s12deff/user-access-control-bypass-via-cmstp-07bf78beoce2>
- [21] "Pinning - OWASP Cheat Sheet Series." Accessed: Aug. 15, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Pinning_Cheat_Sheet.html
- [22] "2. Checkin." Accessed: Aug. 17, 2026. [Online]. Available: https://docs.mythic-c2.net/customizing/payload-type-development/create_tasking_agent-side-coding/initial-checkin
- [23] K. Peabody and K. Davis, "Universally Unique IDentifiers (UUID)," Request for Comments RFC 9562, May 2024. doi: 10.17487/RFC9562.

- [24] “What Is Endpoint Detection and Response (EDR)?.” Accessed: Aug. 17, 2026. [Online]. Available: <https://safe.security/resources/insights/what-is-endpoint-detection-and-response-edr/>
- [25] “Input Validation - OWASP Cheat Sheet Series.” Accessed: Aug. 15, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
- [26] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” Request for Comments RFC 9846, Jul. 2026. doi: 10.17487/RFC9846.
- [27] I. Fette and A. Melnikov, “The WebSocket Protocol,” Request for Comments RFC 6455, Dec. 2011. doi: 10.17487/RFC6455.
- [28] K. Scarfone and P. Hoffman, “Guidelines on Firewalls and Firewall Policy,” Special Publication NIST SP 800-41 Rev. 1, Sep. 2009. doi: 10.6028/NIST.SP.800-41r1.
- [29] R. Dingledine, N. Mathewson, and P. Syverson, “Tor: The Second-Generation Onion Router,” in *Proceedings of the 13th USENIX Security Symposium*, 2004, pp. 303–320. [Online]. Available: <https://www.usenix.org/conference/13th-usenix-security-symposium/tor-second-generation-onion-router>
- [30] SI2cybersecurity, “SI2cybersecurity/OnionC2Implant.” Accessed: Aug. 10, 2026. [Online]. Available: <https://github.com/SI2cybersecurity/OnionC2Implant>
- [31] Auth0, “JSON Web Token Introduction - Jwt.Io.” Accessed: Aug. 06, 2026. [Online]. Available: <https://jwt.io/introduction>
- [32] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Request for Comments RFC 7519, May 2015. doi: 10.17487/RFC7519.
- [33] M. Jones and D. Hardt, “The OAuth 2.0 Authorization Framework: Bearer Token Usage,” Request for Comments RFC 6750, Oct. 2012. doi: 10.17487/RFC6750.
- [34] D. Temoshok *et al.*, “Digital Identity Guidelines,” NIST Special Publication (SP) 800-63-4, Jul. 2025. doi: 10.6028/NIST.SP.800-63-4.
- [35] D. M'Raihi, S. Machani, M. Pai, and R. Wallner, “TOTP: Time-Based One-Time Password Algorithm,” Request for Comments RFC 6238, May 2011. doi: 10.17487/RFC6238.

- [36] "Password Storage - OWASP Cheat Sheet Series." Accessed: Aug. 09, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [37] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-Based Access Control Models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996, doi: 10.1109/2.485845.
- [38] OWASP Foundation, "Authorization Cheat Sheet - OWASP Cheat Sheet Series." Accessed: Aug. 10, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
- [39] M. Nottingham and R. Fielding, "Additional HTTP Status Codes," Request for Comments RFC 6585, Apr. 2012. doi: 10.17487/RFC6585.
- [40] "OWASP API Security Project | OWASP Foundation." Accessed: Aug. 15, 2026. [Online]. Available: <https://owasp.org/www-project-api-security/>
- [41] K. Scarfone, M. Souppaya, and M. Sexton, "Guide to Storage Encryption Technologies for End User Devices," Special Publication NIST SP 800-111, Nov. 2007. doi: 10.6028/NIST.SP.800-111.
- [42] A. Freier, P. Karlton, and P. Kocher, "The Secure Sockets Layer (SSL) Protocol Version 3.0," Request for Comments RFC 6101, Aug. 2011. doi: 10.17487/RFC6101.
- [43] M. Shahin, M. Ali Babar, and L. Zhu, "Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices," *IEEE access : practical innovations, open solutions*, vol. 5, pp. 3909–3943, 2017, doi: 10.1109/ACCESS.2017.2685629.
- [44] "Understanding GitHub Actions." Accessed: Aug. 14, 2026. [Online]. Available: <https://docs.github.com/en/actions/get-started/understand-github-actions>
- [45] "Pull Requests." Accessed: Aug. 15, 2026. [Online]. Available: <https://docs.github.com/en/pull-requests/reference/pull-requests>
- [46] "Testing With CMake and CTest — Mastering CMake." Accessed: Aug. 12, 2026. [Online]. Available: <https://cmake.org/cmake/help/book/mastering-cmake/chapter/Testing%20With%20CMake%20and%20CTest.html>
- [47] ardalis, "Unit Testing C# Code in .NET Using Dotnet Test and xUnit - .NET." Accessed: Aug. 12, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-csharp-with-xunit>

-
- [48] “Run and Test Collections from the Command Line Using Newman CLI | Postman Docs.” Accessed: Aug. 12, 2026. [Online]. Available: <https://learning.postman.com/docs/reference/newman-cli/command-line-integration-with-newman>
- [49] A. K. Gupta and A. S. Kushwaha, “Cost Optimization Techniques for SAP Cloud Infrastructure in Enterprise Environments,” *International Journal of Research Radicals in Multidisciplinary Fields*, vol. 3, no. 2, 2024, doi: 10.64284/ijrrmf.v3i2.164.
- [50] DigitalOcean, “Droplet Pricing,” 2026. Accessed: Aug. 17, 2026. [Online]. Available: <https://www.digitalocean.com/pricing/droplets>
- [51] M. Azure, “Linux Virtual Machines Pricing,” 2026. Accessed: Aug. 17, 2026. [Online]. Available: <https://azure.microsoft.com/en-us/pricing/details/virtual-machines/linux/>
- [52] A. W. Services, “Amazon Lightsail Pricing,” 2026. Accessed: Aug. 17, 2026. [Online]. Available: <https://aws.amazon.com/lightsail/pricing/>